MCSP-232

ONLINE EXAMINATION SYSTEM

by

Sushmita Mishra

Enrolment No.: 2350025885

Under Guidance of

Shiv Pratap Singh

Submitted to the School of Computer and Information Sciences,
IGNOU in partial fulfilment of the requirements for the award of the degree

Master of Computer Applications (MCA)

Year of Submission: 2026

Indira Gandhi National Open University
Maidan Garhi
New Delhi – 110068

Approved Proforma, Project Proposal and Guide Bio-data

Attach (1) the original approved project-proposal proforma, (2) the approved project proposal/synopsis, and (3) the signed guide bio-data at this location before binding.

Student: Sushmita Mishra | **Enrolment No.:** 2350025885 | **Guide:** Shiv Pratap Singh

Study centre: _____ | Regional centre: _____

Guide qualification / designation / organisation: _____

Project Proposal Number / approval date / evaluator stamp: _____

Attach the signed originals behind this sheet before regional-centre submission. The technical chapters that follow document the implemented software, not those administrative records.

Certificate of Originality

This is to certify that the project report titled **ONLINE EXAMINATION SYSTEM** submitted to Indira Gandhi National Open University in partial fulfilment of the requirement for the award of the degree of MASTER OF COMPUTER APPLICATIONS, is an authentic and original work carried out by Ms. **Sushmita Mishra** with Enrolment No. **2350025885** under my guidance.

The matter embodied in this project is genuine work done by the student and has not been submitted whether to this University or to any other University / Institute for the fulfilment of the requirements of any course of study.

……

(Signature of the Student)

Date: _____

Name and Address of the student

Sushmita Mishra

Enrolment No. 2350025885

Address: _____

……

(Signature of the Guide)

Date: _____

Name, Designation & Address of the Guide

Shiv Pratap Singh

Designation: _____

Address: School of Computer and Information Sciences, IGNOU

_____

Declaration

I, **Sushmita Mishra**, Enrolment No. **2350025885**, declare that the project titled **ONLINE EXAMINATION SYSTEM** submitted in partial fulfilment of MCSP-232 for the MCA degree of Indira Gandhi National Open University is my original work, carried out under the guidance of **Shiv Pratap Singh**.

I further declare that this report describes the software that was implemented and executed for this submission. Features that are not present in the source—webcam or AI proctoring, OTP, CAPTCHA, two-factor login, IP allow-lists, browser lockdown, and an automated coding judge—are identified as future scope and are not claimed as implemented.

Software evidence (automated tests, screenshots, and demonstration reports) was obtained from a local run on 5 October 2026.

Date: _____ Signature of the student: ……

Acknowledgement

I thank my project guide, **Shiv Pratap Singh**, for guidance on this MCSP-232 project. I thank the School of Computer and Information Sciences, Indira Gandhi National Open University, Maidan Garhi, for the MCA programme and the MCSP-232 project framework.

Abstract

Need. Institutions still set, conduct and mark many internal tests on paper. That process does not freeze a per-student randomized paper, enforce a server deadline, score objective items immediately, hide results until a deliberate publish step, or produce participation and topic analytics from the same transactional records.

Method. ONLINE EXAMINATION SYSTEM is a three-tier web application for Student, Faculty and Admin actors. The presentation tier is React 18 (Vite). The application tier is Python 3 / Django / Django REST Framework with SimpleJWT and Argon2 password hashes (PBKDF2 still accepted). The data tier is MySQL 8. Exam rules live in transactional service functions: paper freeze, timer, save, objective scoring, descriptive marking, and result publication. Reports are SQL aggregates; there is no report table.

Result. The evidence host executed 35 passed pytest cases in 6.57 seconds on 5 October 2026, plus a separate three-role browser journey whose screenshots appear in Chapters 4 and 8. Demonstration Operating Systems Midterm data used in layout tables was read from the live API the same day.

Limits. The build is a single-institution academic system on HTTP for local demonstration. JWT access tokens are stored in localStorage. There is no per-exam enrolment list, no React component test suite, and no production TLS, rate-limiting, or operational monitoring in the repository. Those limits are restated in Chapter 9 rather than being hidden.

Table of Contents

Approved Proforma, Project Proposal and Guide Bio-data 2
Certificate of Originality 3
Declaration 4
Acknowledgement 5
Abstract 6
Chapter 1 Introduction and Objectives 11
1.1 Introduction 11
1.2 Problem statement 11
1.3 Need for the project 11
1.4 Objectives 11
1.5 Scope, assumptions and exclusions 12
1.6 Target users 12
1.7 Organisation of the report 12
Chapter 2 System Analysis 14
2.1 Identification of need 14
2.2 Existing versus proposed system 14
2.3 Feasibility study 14
2.4 Function-point cost and effort estimate 14
2.5 Project planning — PERT 15
2.6 Gantt schedule 16
2.7 Software engineering paradigm applied 16
2.8 Conflicts in the synopsis and the resolution used 17
2.9 What the literature in the synopsis does and does not support 18
2.10 Disposition of each synopsis section 18
2.11 Feasibility rationale by implementation component 19
2.12 Risk register 21
2.13 Stakeholder workflow and acceptance responsibilities 21
Chapter 3 Software Requirements Specification 23
3.1 Purpose 23
3.2 Scope 23
3.3 Definitions, acronyms and references 23
3.4 Overall description 23
3.5 Hardware and software requirements 23
3.6 Functional requirements catalogue 24
3.7 Detailed specifications (core FRs) 26
3.8 Specifications for the catalogue items not opened in Section 3.7 28
3.9 Non-functional requirements 35
3.10 Validation rules 35
3.11 Detailed non-functional specifications 36
Chapter 4 System Design 39
4.1 Architecture 39
4.2 Modularization 39
4.3 Data flow diagrams 39
4.4 ER model 42
4.5 Data dictionary 42
4.6 Normalization and data integrity 45
4.7 UML views 45
4.8 Procedural design 49
4.9 User interface design 52
4.10 Design-time test cases 58
4.11 Process logic of each module 58
4.12 Edge cases called out by the design 59
4.13 Field validation on the screens 60
4.14 Entity-by-entity design notes 60
4.15 Normalization derivation 62
4.16 Referential actions and index rationale 63
4.17 Algorithm specifications in pseudocode 64
4.18 Module interfaces: inputs, processing, outputs and stores 66
4.19 End-to-end data-flow narratives 68
4.20 Diagram interpretation and consistency checks 68
4.21 Design decisions and rejected alternatives 70
Chapter 5 Implementation and Coding 71
5.1 Development environment and repository 71
5.2 Database implementation, seed and grants 71
5.3 Annotated excerpts 71
5.4 Standardization: efficiency, errors, parameters, validation 72
5.5 API catalogue 72
5.6 How parameters move through one attempt 72
5.7 Error behaviour actually returned 73
5.8 Constraint excerpts (not the full DDL) 73
5.9 Where each requirement lives 74
5.10 Endpoint contracts by resource 74
5.11 Runtime configuration and deployment boundary 76

5.12 Representative JSON contracts....77
5.13 Repository and configuration map....79
Chapter 6 Testing....81
6.1 Testing techniques and strategies....81
6.2 Test plan....81
6.3 Executed unit and integration reports....81
6.4 System / browser verification....83
6.5 Debugging and code improvement....83
6.6 What the browser run and the later API read actually showed....84
6.7 Checks that were not given a Pass row....84
6.8 Detailed executed-case report....85
6.9 Test environment, fixtures and reproducibility....90
6.10 Interpreting the executed result....91
6.11 Test strategy rationale by level....91
6.12 Requirement coverage summary....92
Chapter 7 System Security....94
7.1 Authentication....94
7.2 Authorization and access-rights matrix....94
7.3 Database and examination controls....94
7.4 Production controls still required....94
7.5 One request, followed through the layers....94
7.6 Threat, control and residual-risk review....95
7.7 Security verification scenarios for deployment review....95
7.8 Layered security configuration....96
7.9 Access review checklist....97
Chapter 8 Results and Reports....98
8.1 Student history layout....98
8.2 Faculty exam analysis layout....99
8.3 Admin summary layout....101
8.4 Audit extract layout....102
8.5 Reading the two observed totals....104
8.6 Question-level arithmetic on the captured FAIL paper....104
8.7 Screen-by-screen acceptance notes....105
8.8 Report query definitions....106
8.9 User operating guide....107
8.10 Administrator maintenance guide....108
Chapter 9 Limitations, Future Scope and Conclusion....110
9.1 Current limitations....110
9.2 Future scope....110
9.3 Conclusion — objectives versus evidence....110
9.4 Objective-by-objective validation....111
Bibliography....19
Appendix A — Traceability....114
Appendix B — Observed pytest transcript....115
Appendix C — CD handover....116
Appendix D — Glossary....117
Appendix E — Viva preparation (project-specific)....118

List of Figures

Figure 2.1 — PERT network for ONLINE EXAMINATION SYSTEM (A1–A15)....16
Figure 2.2 — Fifteen-week Gantt derived from Table 2.5....16
Figure 4.1 — Three-tier architecture: React, DRF/Django services, MySQL 8, Gunicorn....39
Figure 4.2 — Context-level DFD....40
Figure 4.3 — Level 1 DFD....41
Figure 4.4 — Level 2 DFD: conduction....41
Figure 4.5 — Level 2 DFD: evaluation....42
Figure 4.6 — ER diagram of the implemented schema....42
Figure 4.7 — Use-case diagram....46
Figure 4.8 — Class diagram of principal models and services....47
Figure 4.9 — Combined activity of attempt through publication....48
Figure 4.10 — Sequence among browser, API, services and MySQL....49
Figure 4.11 — Exam, attempt and result states....49
Figure 4.12 — Procedural design: freeze_paper (question shuffle and option shuffle)....50
Figure 4.13 — Procedural design: score_mcq and objective floor....51

Figure 4.14 — Login. Email/password form and demonstration account reminder.....52
Figure 4.15 — Student registration. No role picker; server assigns STUDENT.....53
Figure 4.16 — Student dashboard with exam state and Start.....53
Figure 4.17 — Exam room: frozen paper, server timer, palette filters and mark-for-review state. 54
Figure 4.18 — Submission confirmation after the server accepts the paper.....54
Figure 4.19 — Faculty exam list (draft and published).....55
Figure 4.20 — Question bank with keyword, topic, difficulty and type filters.....55
Figure 4.21 — Published paper and lifecycle controls.....56
Figure 4.22 — Manual evaluation of a descriptive answer.....56
Figure 4.23 — Evaluated result ready to publish (publish is a distinct action).....57
Figure 4.24 — Admin user management.....57
Figure 8.1 — Published student result list (demo).....98
Figure 8.2 — Published result detail with answer review after publish.....99
Figure 8.3 — Faculty report screen.....100
Figure 8.4 — Operating Systems Midterm analytics.....100
Figure 8.5 — Admin summary.....101
Figure 8.6 — Admin summary detail.....102
Figure 8.7 — Admin audit log (demo).....103
Figure 8.8 — Evaluation working lists (operational, not a stored report file).....103

List of Tables

Table 0.1 — Abbreviations used in this report.....10
Table 1.1 — Identification of need by actor.....11
Table 1.2 — Project objectives.....11
Table 1.3 — User classes.....12
Table 2.1 — Existing versus proposed system.....14
Table 2.2 — Feasibility.....14
Table 2.3 — Internal logical files.....15
Table 2.4 — Transactional function points.....15
Table 2.5 — PERT activities.....15
Table 2.6 — Gantt occupancy of the same activities.....16
Table 2.7 — Agile increments mapped to OES modules.....16
Table 2.8 — Synopsis section disposition.....18
Table 2.9 — Project risk register.....21
Table 3.1 — SRS definitions.....23
Table 3.2 — Hardware and software.....23
Table 3.3 — Functional requirements catalogue.....24
Table 3.4 — Non-functional requirements (implemented only).....35
Table 3.5 — Serializer / screen validation.....35
Table 4.1 — Modules, tables and process logic.....39
Table 4.2 — Level 1 processes.....40
Table 4.3 — Principal relationships.....42
Table 4.4 — users.....42
Table 4.5 — subjects.....43
Table 4.6 — topics.....43
Table 4.7 — questions.....43
Table 4.8 — question_options.....43
Table 4.9 — exams.....43
Table 4.10 — exam_questions (master paper).....43
Table 4.11 — exam_attempts.....44
Table 4.12 — attempt_questions (frozen paper).....44
Table 4.13 — attempt_option_orders.....44
Table 4.14 — attempt_answers.....44
Table 4.15 — results.....44
Table 4.16 — audit_logs.....44
Table 4.17 — Integrity types applied to OES.....45
Table 4.18 — UI inventory.....52
Table 4.19 — Design-time test cases (not execution results).....58
Table 4.20 — Edge cases and the observed check.....59
Table 4.21 — Screen fields and the server rule.....60
Table 4.22 — Functional dependencies used in normalization.....63
Table 4.23 — Design decisions and rejected alternatives.....70
Table 5.1 — Implemented API catalogue.....72
Table 5.4 — Error map used by the API.....73
Table 5.5 — Requirement groups and the files that implement them.....74
Table 5.6 — Repository ownership and change impact.....79
Table 6.1 — Techniques applied to this project.....81
Table 6.2 — Test plan.....81

Table 6.3 — Executed automated cases (35 collected, including four MCQ parameter rows) 81
Table 6.4 — Not marked Pass.....84
Table 6.10 — Requirement coverage and named gaps....92
Table 7.1 — Access-rights matrix from permissions.py and views....94
Table 7.3 — Access review checklist....97
Table 8.1 — Student history (demo student@oes.local, 5 Oct 2026)....98
Table 8.2 — Faculty exam report cards (live GET /api/reports/faculty/exams/1/)....99
Table 8.3 — Faculty topic/difficulty analysis (live GET …/analysis/)....99
Table 8.4 — Admin summary rows (live GET /api/reports/admin/summary/). Institution totals: published_results=1, pass=1, fail=0, average=7.50....101
Table 8.5 — Audit layout (latest 500; filters action/user/entity)....102
Table 8.6 — Awards read from Figure 8.2....104
Table 9.1 — Objectives mapped to evidence....110
Table A.1 — Requirement traceability....114
Table D.1 — Glossary....117

Table 0.1 — Abbreviations used in this report

Abbreviation	Meaning in this project
API	Application Programming Interface — here, DRF JSON routes under /api/
Argon2	Password hasher configured first in Django PASSWORD_HASHERS
Attempt	One exam_attempts row; unique per student per exam
DFD	Data Flow Diagram (context, Level 1, Level 2 conduction/evaluation)
DRF	Django REST Framework
Frozen paper	attempt_questions plus attempt_option_orders created at start
Gunicorn	Production WSGI server: gunicorn config.wsgi:application
JWT	JSON Web Token issued by SimpleJWT (access + refresh)
MCQ	Multiple-choice question with exactly four options and one key
MCSP-232	IGNOU MCA project course for this report
NFR	Non-functional requirement
OES	Short name for ONLINE EXAMINATION SYSTEM used in prose
RBAC	Role-based access control (STUDENT / FACULTY / ADMIN)
SRS	Software Requirements Specification (IEEE 830 shape)
UFP	Unadjusted function points used in the projected effort estimate
3NF	Third normal form, applied to the 13 domain tables

Chapter 1 Introduction and Objectives

1.1 Introduction

Institutions still run many internal tests as paper booklets: seating, invigilation, collection, and hand marking. That process does not scale cleanly when the same question bank must serve several sittings, when objective items could be scored at once, and when faculty still need a queue for descriptive answers.

ONLINE EXAMINATION SYSTEM is a browser application for one institution's own users. It does not connect to IGNOU's examination platform, take payment, or issue certificates outside this database. The product title is taken from the approved synopsis and is not renamed.

The product is a three-tier system. Students, faculty, and administrators use a React client. The client calls JSON APIs on Django REST Framework. Django ORM writes MySQL 8. Seven logical modules sit on four Django apps: accounts, catalog, examination, and auditlog.

1.2 Problem statement

Paper conduction and ad-hoc spreadsheets leave five gaps that this project addresses in software, not by adding invigilation hardware:

Identity is informal. Faculty and admin accounts must be created by an administrator; students may self-register only as STUDENT.

Questions are copied into each paper. The bank stores MCQ and descriptive items by subject, topic, difficulty, and marks; an exam selects them through exam_questions.

Timing is a wall clock. The server computes a deadline as the earlier of started_at + duration_minutes and exam.end_at, auto-saves answers, and auto-submits when that instant is reached.

Marking is delayed for every item type. MCQ scores run on submit, including optional negative marking floored at zero; descriptive scores wait for faculty; the student does not see marks until a separate publish step.

Reports drift if they are stored as files. Participation, pass/fail, averages, and topic/difficulty rows are queries over attempts, answers, and published results. There is no reports table.

1.3 Need for the project

The synopsis names a web-based examination system for institutions and test bodies. The need is operational and actor-specific:

Table 1.1 — Identification of need by actor

Actor	Pain in the existing paper process	Need satisfied in OES
Student	One booklet, no resume of a frozen electronic paper, delayed objective marks	Register, start one sitting, stable shuffled paper, 15-second save, server timer, published result only
Faculty	Retype questions, mark every item by hand, rebuild analytics in a spreadsheet	Reusable bank, draft/publish/close, auto MCQ score, descriptive queue, exam report and topic analysis
Admin	Account lists and audit trails live in email or paper	Create Faculty/Admin, activate/deactivate, institution summary, last-500 audit filter
Institution	Duplicate attempts and result leaks depend on invigilation only	Unique exam+student attempt, unpublished results, DML-only database login, role checks on every mutating API

1.4 Objectives

Table 1.2 — Project objectives

ID	Objective	Where it lands in the code

OBJ-01	Register students; authenticate all three roles	POST /api/auth/register/, POST /api/auth/login/
OBJ-02	Enforce RBAC	HasRole / IsAdmin / IsFacultyOrAdmin / IsStudent
OBJ-03	Maintain a question bank (MCQ, descriptive)	catalog models and /api/questions/
OBJ-04	Create, schedule, draw or fix, publish, and close exams	/api/exams/ and related actions
OBJ-05	Conduct a timed attempt with shuffle and auto-save	start_attempt, freeze_paper, save_answers
OBJ-06	Score MCQ automatically; mark descriptive manually	score_mcq, award_descriptive
OBJ-07	Publish results; hide them until then	results.published; student GET returns 403 if unpublished
OBJ-08	Produce student, faculty, and admin reports as queries	examination/reports.py
OBJ-09	Record selected security events	audit_logs

1.5 Scope, assumptions and exclusions

In scope (implemented). English-language browser UI; Student self-registration; admin-managed Faculty and Admin users; Argon2 hashing with PBKDF2 still accepted; SimpleJWT; subjects, topics and questions; exactly four MCQ options with one correct option; keyword/topic/difficulty/type question-bank filters; exam draft/publish/close; fixed paper and rule-based draw; one attempt per student per exam; frozen question and option order; server remaining-seconds; auto-save; answered/not-answered/marked palette filters; persistent mark for review; submit and expiry auto-submit; negative marking floored at zero; unpublished then published results; faculty/admin CSV exports; query-based reports; admin audit list; oes_migrator versus oes_app.

Assumptions. One academic project deployment; Chrome or Firefox; users exist only inside this database; transport encryption is a reverse-proxy concern; Faculty/Admin accounts are provisioned by an administrator.

Out of scope (must not be described as built). Payments; IGNOU or Moodle integration; native mobile or offline clients; email/SMS; OTP; CAPTCHA; 2FA; IP allow-lists; camera/microphone/screen surveillance; face recognition; browser lockdown; AI malpractice detection; field-level AES; an executable coding sandbox.

1.6 Target users

Table 1.3 — User classes

Role	How the account is created	Principal tasks
STUDENT	Self-registration (RegisterSerializer forces STUDENT)	Register, sign in, start or continue, auto-save, submit, view published results
FACULTY	Admin POST /api/users/ with role FACULTY	Question bank, exam draft and paper, publish/close, evaluation, exam reports
ADMIN	Admin create, or first seed	All faculty functions plus user activate/deactivate, institution summary, audit

1.7 Organisation of the report

Chapter 2 analyses the existing paper process, feasibility, function-point effort, PERT/Gantt, and Agile sprints. Chapter 3 is an IEEE 830-style SRS. Chapter 4 is design: architecture, modules, DFD/ER/UML, data dictionary, 3NF, integrity, procedures, and annotated screens. Chapter 5 documents the coding environment, DDL, grants, and selected excerpts — not a dump of the repository. Chapter 6 records executed tests. Chapter 7 is security as coded. Chapter 8 gives

report layouts with live demonstration values. Chapter 9 maps objectives to evidence and states limits and future work.

Chapter 2 System Analysis

2.1 Identification of need

Section 1.3 stated actor pain. This section is the analysis view: the organisation needs a single application in which a question is authored once, an exam is a scheduled window with a paper, a student may sit once, the clock is authoritative on the server, objective items are scored without waiting for the whole pack, descriptive items have a faculty queue, and reports cannot drift from the mark list because they are queries.

The need is *not* for biometric invigilation. The synopsis lists monitoring and anti-cheating as requirements; this implementation satisfies the subset that can be encoded as application rules (one attempt, frozen order, hidden keys, result embargo, audit). Camera, lockdown and AI detection remain future work after legal review.

2.2 Existing versus proposed system

Table 2.1 — Existing versus proposed system

Concern	Existing paper / spreadsheet process	Proposed ONLINE EXAMINATION SYSTEM
Question reuse	Retype into each booklet	Bank in questions; exam selects via exam_questions
Identity	Hall ticket + invigilator	Email login; Student cannot self-elevate; inactive users rejected
Attempt control	Invigilation only	Unique (exam_id, user_id); second start HTTP 409
Paper stability	Printed order	Shuffled then frozen per attempt, including MCQ option order
Timer	Hall clock	min(started_at + duration, end_at); auto-submit
Objective marking	After collection	score_mcq at submit; negative floor at 0
Descriptive marking	Loose scripts	Evaluation queue; marks in [0, question.marks]
Result visibility	Notice board / email	Hidden until POST /api/results/{id}/publish/
Analytics	Rebuilt by hand	Faculty participation/topic analysis; admin summary — queries only
Evidence	Paper logs	audit_logs for LOGIN, START, SAVE, SUBMIT, EVALUATE, PUBLISH

2.3 Feasibility study

Table 2.2 — Feasibility

Aspect	Finding
Technical	Python 3, Django, DRF, React and MySQL 8 are listed in MCSP-232 section VIII and are the implemented stack. The evidence host ran Python 3.12, Django 5.2.17 (requirements.txt pins 5.1.4), DRF 3.18.1, Node 22, MySQL 8.0, Chrome.
Economic	All selected components are open-source. Cost in §2.4 is **projected effort**, not money spent. No rupee rate is claimed.
Operational	Three roles match the synopsis. Faculty is a first-class actor in permissions and UI.
Schedule	PERT critical path is about 75 working days (§2.5), which is more realistic than a one-line eight-week synopsis phase list.
Legal / ethical	The current scope avoids biometric and surveillance collection. Production still needs a privacy notice, retention policy, backups, and institutional authorization. Those are not claimed as deployed.

2.4 Function-point cost and effort estimate

Method: simplified IFPUG. Value Adjustment Factor is taken as **1.00** because the fourteen General System Characteristics were not independently scored. All figures below are **projected academic estimates**, not timesheets and not expenditure.

Internal Logical Files are grouped as user-identifiable files (not one ILF per SQL table). Django auth / admin / sessions / contenttypes tables are framework storage and are not counted. EIF = 0 (no external system).

Table 2.3 — Internal logical files

ILF	Tables	Complexity	Weight
User accounts	users	Low	7
Subject catalogue	subjects, topics	Low	7
Question bank	questions, question_options	Average	10
Exam definition	exams, exam_questions	Average	10
Attempt (frozen paper + answers)	exam_attempts, attempt_questions, attempt_option_orders, attempt_answers	High	15
Published outcome	results	Average	10
Audit	audit_logs	Low	7
ILF total			**66**

Table 2.4 — Transactional function points

Type	Count basis (from actual APIs)	FP
EI	Register, login, refresh, admin user write, catalog write, exam write, draw, publish/close, start, save, submit, mark, publish result (high complexity on question write, draw, start, submit)	80
EO	Revealed student result, history, faculty exam report, topic analysis, admin summary, evaluation queue, in-exam payload with remaining seconds	41
EQ	/me, user list, subject/topic/question lists, exam lists, attempt list, evaluation detail, audit, health	38
Unadjusted FP	66 + 0 + 80 + 41 + 38	**225**

Productivity assumption for this academic build: **5 person-hours per unadjusted function point** (inside a 4–6 hour band). Projected effort = 225 × 5 = **1,125 person-hours**, or about **7.0 person-months** at 160 hours/month for one developer.

A future monetary estimate should multiply 7.0 person-months by a rate the student or institution actually states. No rupee figure is invented here.

2.5 Project planning — PERT

Expected time TE = (O + 4M + P) / 6. Unit: working days. One student developer.

Table 2.5 — PERT activities

ID	Activity	O	M	P	TE	Pred.
A1	Requirements gathering	3	5	8	5.2	—
A2	Analysis and conflict resolution	3	5	8	5.2	A1
A3	SRS	4	6	10	6.3	A2
A4	Architecture	2	4	6	4.0	A3
A5	Database design	3	5	8	5.2	A4
A6	UI design	3	5	8	5.2	A4
A7	API design	2	3	5	3.2	A4
A8	Backend	12	18	28	18.7	A5, A7
A9	Frontend	10	16	24	16.3	A6, A7
A10	Integration	3	5	9	5.3	A8, A9
A11	Functional testing	4	6	10	6.3	A10
A12	Security testing	2	3	6	3.3	A10
A13	Debugging	2	4	7	4.2	A11, A12
A14	Report	8	12	18	12.3	A13
A15	Demo package	1	2	4	2.2	A14

Critical path: A1 → A2 → A3 → A4 → A5 → A8 → A10 → A11 → A13 → A14 → A15. Sum of expected times on that path: 5.2 + 5.2 + 6.3 + 4.0 + 5.2 + 18.7 + 5.3 + 6.3 + 4.2 + 12.3 + 2.2 = **74.9 working days** (about 15 five-day weeks).

Slack (this network, not the Job Portal sample). Frontend A9 versus backend A8 is about 2 working days because A8's TE is 18.7 against A9's 16.3 after they share A7. Security testing A12 (3.3 days) sits parallel to functional testing A11 (6.3 days), so A12 has about 3 days of slack before A13. UI design A6 and API design A7 finish inside the database-design window of A5. This path is an OES 15-week academic plan; it is not the sample report's 32-day T1–T9 network.

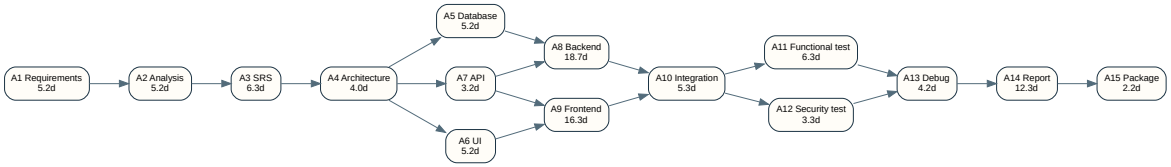


Figure 2.1 — PERT network for ONLINE EXAMINATION SYSTEM (A1–A15).

2.6 Gantt schedule

Table 2.6 — Gantt occupancy of the same activities

Week	Activities
1–2	A1 requirements, A2 analysis
2–4	A3 SRS, A4 architecture
4–5	A5 database, A6 UI, A7 API (overlap)
5–9	A8 backend and A9 frontend in parallel
9–10	A10 integration
10–12	A11 functional tests; A12 security tests in parallel
12	A13 debugging
12–15	A14 report
15	A15 package

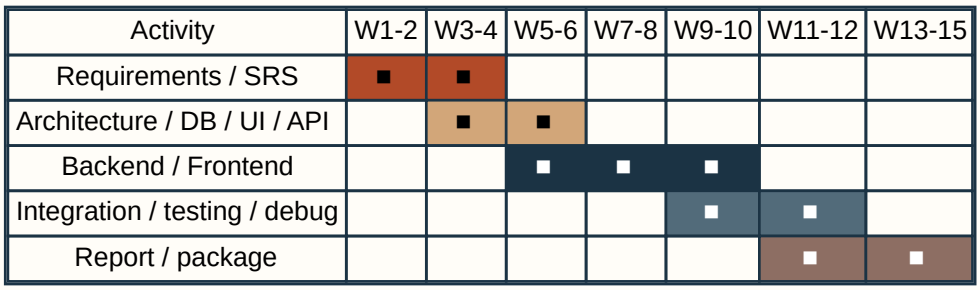


Figure 2.2 — Fifteen-week Gantt derived from Table 2.5.

2.7 Software engineering paradigm applied

The synopsis names Agile. Work is therefore organized as short vertical increments (sprints) rather than a single coding phase after a frozen design. Each sprint delivers a slice of model, API, permission, React screen, and test. Database migrations preserve schema history. The final evidence run follows integration rather than being assumed from unit-level development.

This is Scrum-like iteration for one student, not a multi-team board and not a claim of GitHub/Render delivery from the sample report.

Table 2.7 — Agile increments mapped to OES modules

Sprint	Module	Primary tables / APIs
1	User and authentication	users, /api/auth/*, /api/users/*
2	Question bank	subjects, topics, questions, question_options
3	Exam management	exams, exam_questions
4	Conduction	exam_attempts, frozen paper tables, answers
5	Evaluation and results	scoring services, results
6	Reports and audit	examination/reports.py, audit_logs

Sprint	Module	Primary tables / APIs
7	Test and fix	tests/, screenshot evidence

2.8 Conflicts in the synopsis and the resolution used

The 25-page synopsis is a proposal. It is not internally consistent, and several sentences describe features as if they already exist. The implemented design records one resolution for each conflict. Later chapters follow these resolutions. They do not follow the contradictory sentence.

Six named modules versus eight process descriptions. Section 9 of the synopsis lists User, Admin, Exam, Evaluation, Report and Security. Section 9.2 then describes question-bank maintenance, scheduling, conduction, results and analytics as separate processes. Treating Admin and Faculty as modules would duplicate the user table. The implementation therefore has seven process areas — authentication, question bank, exam management, conduction, evaluation, results and reports, and audit — while Student, Faculty and Admin remain roles on users.role.

A question owned by one exam versus a reusable bank. The synopsis ER text says each question belongs to one exam (Exam 1:N Question) and stores option_a through option_d on the question row. The process text says questions are tagged by topic and difficulty and then randomly selected. Those cannot both be true. The database stores questions in a bank (questions, question_options). An exam selects a subset through exam_questions. An attempt copies that subset, shuffled, into attempt_questions and attempt_option_orders. Deleting a subject or a question that is already on an attempt is restricted.

“User attempts Exam” is not an attempt entity. The synopsis draws User 1:N Exam. An exam definition is not a student’s sitting. exam_attempts is its own table, with UNIQUE (exam_id, user_id). A second start returns HTTP 409. One result row per user per exam is a separate uniqueness rule on results.

Result 1:1 Report would store a drifting copy. The synopsis proposes a Reports table of average score and pass percentage. Those numbers change when another result is published. The implementation does not create that table. Faculty and admin figures are queries over attempts, answers and published results (examination/reports.py). Chapter 8 shows both a layout and the query rule.

Faculty is missing from some synopsis diagrams. The use-case text names only Admin and Student, while the role list and the question-bank steps include Faculty. Faculty is a first-class role and actor in the use-case diagram, the permission classes, and the React navigation.

Eight-week synopsis schedule versus the documentation and build. The synopsis Gantt is one week each for analysis, testing and documentation, two weeks for design and three for implementation. That list is kept as the phase order only. The PERT in Section 2.5, about 74.9 working days on the critical path, is the schedule used for this academic build. The sample Job Portal path of 32 days is not used.

Security sentences that name unbuilt controls. The synopsis introduction, scope and security section name camera and microphone monitoring, screen watching, browser lockdown, facial recognition, AI cheating detection, OTP, CAPTCHA, IP whitelist, field-level AES, and a proctoring server. The same document later lists several of those items as future scope. The resolution is the future-scope reading. The running code has hashed passwords, role checks, randomization, a server timer, audit rows and TLS as a deployment requirement. It does not have the surveillance or OTP features, and this report does not say that it does.

Coding questions. The synopsis says objective items are primary, with room for descriptive and coding items. Auto-grading is defined for objective items. The software implements MCQ auto-scoring and descriptive manual scoring. A coding item, if it were stored, would be text for a person to mark. There is no code-execution sandbox.

Alternative tools. The synopsis lists Java/Spring Boot and Python/Django, MySQL and PostgreSQL, Eclipse and VS Code, Tomcat and Nginx. One stack is used everywhere in this report: Python 3, Django, Django REST Framework, React, MySQL 8, with Gunicorn named for production WSGI. PostgreSQL is not the project database. Java, Spring Boot and Tomcat remain the unused alternative from the synopsis.

2.9 What the literature in the synopsis does and does not support

Section 5.4 of the synopsis says that web-based examination systems can shorten evaluation time and that security and scale remain open problems. It cites “IEEE Research Papers” as a class of documents, not as particular papers. That class name is not copied into the reference list. The design choices that *are* grounded in standard software-engineering and database practice, and in the official Django, DRF, React and MySQL manuals, are: a three-tier split, an IEEE-shaped requirements list, function-point and PERT estimates, balanced data-flow diagrams, a key-based ER model, third normal form, and tests with an actual result column.

No claim is made that a particular research paper was read unless it appears as a full reference. The books named in the reference list (Sommerville; Pressman and Maxim; Elmasri and Navathe; Chopra, as named by the synopsis) are the textbook frame for estimation, process and normalization. They are not a substitute for the source code.

2.10 Disposition of each synopsis section

Table 2.8 — Synopsis section disposition

Synopsis section	What it asks for	What this project did
1 Title	ONLINE EXAMINATION SYSTEM	Kept unchanged on the cover, header and browser title
2 Introduction and objectives	Web examinations, MCQ, descriptive, coding, instant objective results, analytics	MCQ and descriptive are built. Coding and “real-time proctoring” in the introduction are not built
3 Category	Web application and database	Django REST API, React client, MySQL 8
4 Tools	HTML/CSS/JS; Java or Python; MySQL or PostgreSQL; Draw.io; Eclipse or VS Code; Tomcat or Nginx	Python, Django, DRF, React, MySQL 8, Draw.io-style diagrams, VS Code class of editor, Gunicorn rather than Tomcat
4.2 Hardware floor	i3, 4 GB RAM, 20 GB, network	Recorded as a floor in Table 3.2, not as a measured load test
5.1 Problem	Manual exams, delayed results, weak malpractice control	Chapters 1 and 2. Malpractice control is limited to the controls that exist
5.2 Functions	Registration, roles, scheduling, bank, timer, automatic evaluation, reports	Table 3.3
5.4 Literature	Efficiency claim and an unnamed IEEE class	Section 2.9. No unnamed paper is treated as read
5.5 Eight-week Gantt	Analysis, design, implementation, testing, documentation	Phase order only. Section 2.5 replaces the durations
6 Scope	Anytime access, objective first, auto-grade, proctoring, scale, LMS	Access, objective grading and reports are in scope. Proctoring, claimed scale-out and LMS integration are future scope
7 Analysis roles	Admin, Student, Faculty	All three are roles. Faculty is restored where the synopsis diagrams dropped it
7.1 DFD	Level 0, 1 and 2	Figures 4.2 to 4.5, including Faculty and the stores
7.2 ER	User–Exam, Exam–Question, Result–Report	Replaced by the bank, attempt and query-report model in Figure 4.6
7.3 Use case	Admin and Student only	Figure 4.7 adds Faculty
7.4–7.6 Behaviour diagrams	States, activities, classes	Figures 4.8 to 4.11 use the implemented states, not “Exam Active” as a separate undefined state
8 Tables	Users, Exams, Questions with option_a –d, Answers, Results, Reports	Thirteen domain tables in Section 4.5. No Reports table. Options are rows
9 Six modules	User, Admin, Exam, Evaluation, Report, Security	Section 4.2 and Section 4.11
9.3 Data structures	Arrays, hash maps, queues, tables	Answers are rows. Eligibility is a time

Synopsis section	What it asks for	What this project did
		comparison, not a queue
9.4 Method	Agile	Section 2.7
9.5 Reports	Student, faculty difficulty, admin pass/fail	Chapter 8
10 Network	Clients, application server, database, optional proctoring server	Clients, Django/Gunicorn, MySQL. No proctoring server
11 Security	Hashing, OTP, CAPTCHA, AES, lockdown, webcam, face recognition, audit, IP whitelist	Hashing, JWT, roles, randomization, timer, audit, DML-only database login. The rest are not implemented
12 Future scope	AI proctoring, mobile offline, Moodle, cloud	Chapter 9, written as future
13–14 Outcome and conclusion	Paperless exams and less manual marking	Claimed only for the functions the tests and screenshots show
15 Bibliography	IGNOU, unnamed IEEE papers, Chopra, NDEA, two websites	References that were actually used. The unnamed IEEE line is omitted

2.11 Feasibility rationale by implementation component

Technical feasibility

Custom identity and roles. Django supports a custom user model before the first migration. The project uses email as USERNAME_FIELD and a three-value role, so there is no need to build an authentication protocol. SimpleJWT integrates with DRF and accepts that custom primary key. Risk remains in token storage and expiry policy, which Chapter 7 discloses.

Question-bank rules. A relational schema naturally represents subject/topic/question/options. Serializer validation can count options and keys in one transaction. MySQL CHECKs cover simple domains and positive values, while service/serializer logic covers “exactly four and one correct”, which is cross-row. The design is feasible on the synopsis hardware floor because a bank lookup and one five-question paper are small; no national-scale claim follows from that observation.

Randomization and refresh. Randomizing only in React would reshuffle on reload and expose a client-controlled paper. Creating frozen rows at start costs one write per question and option, but makes later reads deterministic. SystemRandom.shuffle avoids a predictable application PRNG seed. The extra attempt_option_orders table is a technically feasible refinement beyond the synopsis.

Timer. The server already stores timezone-aware start and end values. Deadline calculation is constant-time and requires no job queue. Auto-submit is checked on interaction; therefore a completely disconnected browser may remain IN_PROGRESS until a later request or administrative maintenance. The design enforces that it cannot continue receiving valid saves past the deadline, but a production scheduler could close dormant rows proactively.

Evaluation. Objective scoring is deterministic. Descriptive marking is a Faculty workflow with a range check. Separating calculation from publication avoids exposing a partial result. There is no natural-language grading or code execution, removing the operational/security complexity of untrusted execution.

Reports. Aggregates over indexed attempts/results are technically simple. The report data in this build is small; no benchmark exists for a large cohort. Query-only reports avoid synchronization jobs. Production may later add read replicas/materialized aggregates after measurement.

Frontend. React Router and native form controls cover the defined screens. Vite produces static assets. Responsive CSS supports desktop/mobile-width layouts, but “usable” is demonstrated by screenshots rather than a formal accessibility or multi-browser laboratory.

Economic feasibility

Python, Django, DRF, React, Vite, MySQL Community, Gunicorn and the development tools are open-source. The academic cost estimate therefore focuses on human effort, not license purchase. The UFP calculation yields a projected development range. It excludes a guide fee, student device, already available internet/electricity and any institutional procurement.

A real deployment has recurring costs not present on the local host: domain and certificate operations (even if the certificate itself is free), compute, managed database or server administration, backups, monitoring, email/SMS if later added, and support during examinations. Proctoring would add legal review, storage and human monitoring costs and is not included. Economic feasibility should be re-evaluated with the institution's cohort and service-level target.

The system can reduce repeated question typing and objective marking, but this report has not measured monetary savings. "Saves cost" from the synopsis remains an expected benefit, not an observed rupee result.

Operational feasibility

Student actions follow familiar form/dashboard patterns. Faculty work is divided into bank, exam, evaluation and report screens rather than one Admin-only surface. Admin provisions staff and sees the cross-exam summary/audit. The role division matches the implemented responsibility and corrects the synopsis diagrams that omit Faculty.

Operational adoption requires account ownership, a question-review policy, who may publish, what happens after an outage, and how a disputed mark is corrected. The current software intentionally locks a published result, so an approved correction feature would be needed for production rather than direct SQL.

The absence of per-exam enrollment means any active Student can start any available published exam. That is acceptable only for a small demonstration or institution-wide test. A class/cohort eligibility table is a future operational requirement before use for restricted courses.

Schedule feasibility

The 74.9-working-day critical path includes documentation and the observed integration/test/debug stages. UI and backend overlap. The result is longer than the eight-week synopsis summary but consistent with a single student producing software, diagrams, tests, screenshots and the MCSP-232 report. It is not a timesheet claim: PERT values are estimates.

Schedule risk concentrates in attempt state logic, mixed evaluation, environment setup and final evidence. The sprint sequence reduces that risk by completing authentication/bank/exam before conduction, and conduction before reports. Report figures are generated from the real schema rather than drawn first.

Legal, privacy and ethical feasibility

The current build stores name, email, role, answer text, marks and audit metadata. A production owner must define purpose, consent/notice, retention, access, correction and deletion obligations. Because the build does not collect face, video, microphone, screen or biometric data, it avoids those higher-risk categories. Future proctoring cannot be enabled by simply changing a setting; it needs a separate privacy and proportionality assessment.

Question/answer keys and unpublished results are confidential academic records. HTTPS, protected backups and staff account governance are necessary. The demonstration screenshots use seeded/project data and are labelled, avoiding a claim that they are official learner records.

2.12 Risk register

Table 2.9 — Project risk register

Risk	Likelihood / impact	Existing response	Remaining action
Synopsis/report claims unbuilt proctoring	Medium / high evaluation risk	Explicit exclusions and future-scope wording	Keep viva/report consistent
Duplicate or unstable attempt	Medium / high	unique key, transaction, frozen rows, tests	production concurrency test
Time-zone or client-clock error	Medium / high	timezone-aware server deadline	synchronize production host and monitor
Unpublished mark leakage	Low / high	result gate, omitted key, tests	security review on new endpoints
Faculty account misuse	Medium / high	Admin provisioning, role checks, audit	institutional account/exit process, MFA future
Database/schema damage	Low / high	separate migrator/app grants and tests	backups, restore test, restricted credentials
Network interruption	Medium / medium	immediate + 15-second save; server deadline	user guidance and incident procedure
Question pool too small	Medium / medium	draw validates whole request/rolls back	bank readiness checklist
Report statistics misunderstood	Medium / medium	published-only definitions and Chapter 8 reconciliation	label snapshots/export date
Frontend regression	Medium / medium	Vite build and browser flow	component/E2E automation
Dependency vulnerability	Medium / high	pinned packages	update policy and scanning
Missing approved administrative pages	Medium / high submission risk	placeholders and handover checklist	attach signed proforma/bio-data/certificates

2.13 Stakeholder workflow and acceptance responsibilities

Student responsibility

The Student supplies a valid name/email/password, keeps credentials private, chooses only their own attempt, reads instructions, watches the timer, confirms save feedback, and submits. The software enforces one sitting and the deadline; it cannot guarantee the Student has reliable connectivity or an undistracted environment. Result availability is controlled by Faculty publication.

Student acceptance is demonstrated when registration cannot self-elevate, an available exam starts once, refresh preserves paper order, answers persist, timeout/manual submit ends the attempt, unpublished marks remain hidden, and a published result is readable with the expected breakdown.

Faculty responsibility

Faculty prepares academically correct bank items, classifies topic/difficulty, sets marks/key, reviews the scheduled window and paper total, publishes the exam, marks descriptive responses within range, reviews the completed total, publishes the result and interprets reports. The application validates shape/state; it cannot judge whether a question or manual award is academically fair.

Faculty acceptance is demonstrated when invalid MCQ/descriptive shapes fail, inactive/foreign questions cannot enter a paper, draw either completes or rolls back, publish guards run, in-progress answers are not visible, manual marks create the result only when complete, and report calculations follow published rows.

Admin responsibility

Admin owns staff account creation/deactivation, institution summary and audit review. The role is powerful: it can perform Faculty operations as well. The project does not provide a second approval for account creation or publication. Production governance must identify who holds Admin and how access is reviewed.

Admin acceptance is demonstrated when a Student/Faculty cannot open user management or institution audit/summary, account state is reflected in authentication, runtime database DDL is denied, and the audit list contains the committed lifecycle actions.

Guide/evaluator responsibility

The evaluator should distinguish documentation evidence from a production assurance. The report provides source paths, tests, diagrams and screenshots. It does not provide a signed proposal, guide bio-data, production TLS evidence, load test or real institutional result. Missing administrative pages remain highlighted for attachment.

Acceptance sequence

Verify cover identity/title and attach signed administrative sheets.

Verify the schema/migrations and runtime/migrator grant split.

Execute the automated suite against an isolated test database.

Build React and run the browser journey with demonstration data.

Reconcile Student total, Faculty aggregate and Admin summary using the Chapter 8 definitions.

Review named limitations and future scope for consistent viva answers.

For a real deployment, complete the security/backup/eligibility checklist before accepting operational use.

This sequence avoids treating a screenshot alone, a health response alone, or a copied test table as acceptance.

Chapter 3 Software Requirements Specification

Prepared in the shape of IEEE Std 830. Requirement identifiers are the trace keys for design, tables, APIs, and tests. Mandatory behaviour uses “shall”. “Observed” means obtained from an executed command or browser run. “Projected” means an academic estimate.

3.1 Purpose

This SRS specifies ONLINE EXAMINATION SYSTEM for MCSP-232. It binds the student, faculty, and admin functions that the Django/React implementation must satisfy. It is not a generic SDLC essay.

3.2 Scope

The software is a web application named **ONLINE EXAMINATION SYSTEM**. It provides registration, JWT login, a question bank, exam lifecycle, timed attempts, evaluation, published results, reports, and an audit log, as listed in §3.6. It does not include proctoring hardware, OTP, CAPTCHA, IP whitelist, or a code judge.

3.3 Definitions, acronyms and references

Table 3.1 — SRS definitions

Term	Meaning in this system
Bank	questions plus question_options; not owned by one exam
Paper	Rows in exam_questions for one exam
Frozen paper	attempt_questions (+ attempt_option_orders for MCQ) created at start
Window	[start_at, end_at) on exams
Deadline	min(started_at + duration, end_at)
Published result	results.published = 1; the only result a student may read

References used: IGNOU MCSP-232 Project Guidelines (July 2024 / January 2025); the ONLINE EXAMINATION SYSTEM synopsis; Django, DRF, SimpleJWT, React and MySQL 8 documentation; Sommerville, Pressman, and Elmasri/Navathe for method chapters only. No unsourced “IEEE research papers” entry is claimed.

3.4 Overall description

Product perspective. New system; no wrapper around an existing LMS.

Product functions. Authenticate three roles; maintain a bank; schedule and publish exams; conduct one timed frozen sitting; score MCQ; mark descriptive; embargo then publish results; query reports; append audit rows.

User classes. Table 1.3.

Operating environment. Server: Python 3, Django, Gunicorn for production WSGI, MySQL 8, timezone Asia/Kolkata. Client: React SPA. Browsers: current Chrome or Firefox (assumed). Development path is HTTP.

Constraints. Title fixed. Single stack. Student registration cannot create FACULTY or ADMIN. Application database login is oes_app (DML only).

Assumptions and dependencies. MySQL 8 with database/grants.sql. No third-party exam API. Demonstration seed passwords exist only for local running.

3.5 Hardware and software requirements

Table 3.2 — Hardware and software

Class	Requirement (observed or pinned)
Development host (observed)	Python 3.12.3; Django 5.2.17 executed (requirements.txt pins 5.1.4); DRF 3.18.1; SimpleJWT 5.5.1; MySQL 8.0.46; Node 22.14; Google Chrome 148

Class	Requirement (observed or pinned)
Pinned Python packages	Django 5.1.4, DRF 3.15.2, SimpleJWT 5.3.1, mysqlclient 2.2.6, argon2-cffi 23.1.0, gunicorn 23.0.0, pytest 8.3.4
Frontend packages	React 18.3.1, react-router-dom 6.28.1, Vite 6.0.6
Synopsis hardware floor	i3-class CPU, 4 GB RAM, 20 GB disk, browser, network — a floor, not a measured production rating
Production WSGI	gunicorn config.wsgi:application --bind 0.0.0.0:8000

3.6 Functional requirements catalogue

The catalogue is the SRS index. Ten core requirements are specified in full immediately after the table. The remaining items are still mandatory; their acceptance is the API and test named in the row.

Table 3.3 — Functional requirements catalogue

ID	Shall statement	Actor	Verification
FR-01	Student self-registers with name, unique email, password; role forced to STUDENT	Visitor	SEC-AUTH-01
FR-02	Login issues JWT access and refresh	All	SYS-AUTH-01
FR-03	Refresh access token	All	no dedicated pytest (limitation)
FR-04	Authenticated profile /me/	All	SYS-AUTH-01
FR-05	Admin creates FACULTY or ADMIN	Admin	SEC-AUTH-04
FR-06	Admin lists/patches users; cannot deactivate self	Admin	SEC-AUTH-04
FR-07	Inactive user cannot authenticate	All	SEC-AUTH-03
FR-08	Faculty/Admin CRUD subjects	Faculty, Admin	covered via question setup
FR-09	Faculty/Admin CRUD topics unique per subject	Faculty, Admin	INT-QB-04
FR-10	MCQ: four options, one correct, marks > 0	Faculty, Admin	INT-QB-01, INT-QB-02
FR-11	Descriptive item has no options	Faculty, Admin	INT-QB-03
FR-12	Activate / deactivate question	Faculty, Admin	INT-QB-05
FR-13	Question already on an attempt cannot change stem/marks/type	Faculty, Admin	serializer update (no dedicated pytest)

ID	Shall statement	Actor	Verification
FR-14	Create exam as DRAFT with window and duration constraints	Faculty, Admin	SEC-EX-01, INT-EX-04
FR-15	Edit/delete exam only while DRAFT	Faculty, Admin	INT-EX-02
FR-16	Add/remove paper questions only in DRAFT; same subject; active	Faculty, Admin	INT-EX-01
FR-17	Rule-based draw; fail if pool too small	Faculty, Admin	INT-EX-03
FR-18	Publish requires non-empty active paper and pass marks ≤ total	Faculty, Admin	INT-EX-02
FR-19	Close; unpublish to draft only if no attempts	Faculty, Admin	INT-COND-06
FR-20	Student lists exams with derived state	Student	browser Fig. 4.21
FR-21	Only an active student starts a published exam inside the window	Student	INT-COND-01
FR-22	Start freezes shuffled questions and MCQ option order	Student	UNIT-COND-01
FR-23	At most one attempt per student per exam	Student	INT-COND-01
FR-24	Remaining seconds from server deadline	Student	INT-COND-05
FR-25	Save answers against the frozen paper	Student	INT-COND-02
FR-26	Student cannot save another student’s paper	Student	owner check in views
FR-27	Manual submit; second submit 409	Student	INT-COND-02
FR-28	Auto-submit when now ≥ deadline	System	INT-COND-04
FR-29	MCQ scoring and objective floor at zero	System	INT-COND-03
FR-30	Faculty awards descriptive marks in range	Faculty, Admin	SYS-EVAL-01
FR	Result row only when no descriptive is pending; unpublished	System	INT-EVAL-01

ID	Shall statement	Actor	Verification
-31			
FR-32	Publish result after EVALUATED	Faculty, Admin	SYS-EVAL-01
FR-33	Student reads result only when published	Student	SYS-EVAL-01
FR-34	Faculty cannot open an in-progress attempt	Faculty, Admin	views 403
FR-35	Student history and exam report	Student	SYS-RPT-01
FR-36	Faculty exam report and topic analysis	Faculty, Admin	SYS-RPT-01
FR-37	Admin institution summary	Admin	SYS-RPT-01
FR-38	Audit list (latest 500)	Admin	SYS-AUD-01
FR-39	Health probe	—	observed HTTP 200
FR-40	Save, return and clear a per-question mark-for-review flag without affecting marks	Student	INT-COND-07
FR-41	Search/filter the question bank by keyword, topic, difficulty and type	Faculty, Admin	INT-QB-06
FR-42	Export faculty exam and admin institution summaries as CSV	Faculty, Admin	SYS-RPT-01

3.7 Detailed specifications (core FRs)

FR-01 Student registration

Shall. A visitor shall create an active Student account using name, unique email and a valid password. A privileged role in the body shall be ignored.

Input. JSON name, email, password to POST /api/auth/register/.

Process. RegisterSerializer lowercases email, runs Django password validators, and create_user(..., role=STUDENT).

Output. HTTP 201 with user payload without the password. Stored hash prefix argon2 on the evidence host.

Failure. Duplicate email or weak password → 400. No user row is committed for those cases.

Test / evidence. test_student_self_registers_as_student_only; test_duplicate_email_and_weak_password_rejected.

FR-10 MCQ creation invariants

Shall. Faculty/Admin shall create an MCQ with subject, topic, text, difficulty, positive marks, exactly four options and exactly one correct option.

Input. Question JSON including options array.

Process. QuestionSerializer validates type, option count and key count. Options are rows, not columns.

Output. HTTP 201 with four options. Student POST is 403.

Failure. 3 or 5 options, 0 keys, or all keys → 400 (parametrized).

Test / evidence. test_faculty_creates_mcq_with_one_correct_option; test_mcq_option_rules.

FR-18 Exam publishing

Shall. A draft with a non-empty active same-subject paper and pass marks not exceeding paper total shall be publishable if the window has not already ended.

Input. POST /api/exams/{id}/publish/ by Faculty/Admin.

Process. publish_exam checks status DRAFT, paper, activity, subject, pass marks, window.

Output. Status PUBLISHED; audit PUBLISH. Subsequent patch/delete of the exam is rejected.

Failure. Empty paper, pass marks greater than total, or ended window → 400.

Test / evidence. test_publish_rules.

FR-22 / FR-23 One attempt and frozen paper

Shall. Starting shall create at most one attempt per student per exam and shall shuffle then persist the master paper and MCQ option order for that attempt.

Input. POST /api/exams/{id}/start/ as Student, exam PUBLISHED, now inside the window.

Process. Transactional insert of exam_attempts; unique constraint one_attempt_per_student; freeze_paper with SystemRandom.

Output. IN_PROGRESS attempt payload without is_correct. Reload returns the same order.

Failure. Second start → 409. Faculty start → 403. Early/late/CLOSED → 400.

Test / evidence. test_start_freezes_shuffled_paper_and_option_order; test_one_attempt_and_role_window_rules.

FR-24 / FR-28 Server timer and auto-submit

Shall. The deadline shall be the earlier of duration end and window end. An expired in-progress attempt shall be submitted by the server.

Input. Attempt row plus current time.

Process. deadline, remaining_seconds, ensure_open calling submit_attempt.

Output. Non-negative remaining seconds; status SUBMITTED/EVALUATED after expiry.

Failure. Save after expiry → 409; blank MCQ scores 0.

Test / evidence. test_timer_uses_the_earlier_of_duration_and_window; test_unanswered_mcq_scores_zero_and_expiry_autosubmits.

FR-29 Objective scoring and negative floor

Shall. A correct MCQ shall receive full marks; a blank 0; a wrong choice 0 minus the exam negative mark. The objective aggregate shall not fall below zero.

Input. Submitted attempt answers.

Process. score_mcq then objective_total with max(sum, 0).

Output. Persisted awarded_marks and objective_marks ≥ 0.

Failure. All-wrong heavy negative still stores 0.00 objective.

Test / evidence. test_negative_marking_floors_at_zero.

FR-30 / FR-32 / FR-33 Descriptive marking and result embargo

Shall. Faculty/Admin shall award descriptive marks in range. The result row shall stay unpublished until a separate publish call. Students shall not read the paper key until then.

Input. POST marks; POST publish; student GET result.

Process. award_descriptive, finalize_result, publish_result.

Output. Attempt EVALUATED; student 200 after publish with revealed correctness.

Failure. Student marks POST → 403. Publish after publish / remark after publish → 400. Unpublished student GET → 403.

Test / evidence. test_descriptive_is_manual_then_published; test_zero_descriptive_creates_unpublished_result.

3.8 Specifications for the catalogue items not opened in Section 3.7

Section 3.7 opens the rules that are easy to implement wrongly. The rest of Table 3.3 is still mandatory. Each item below names the endpoint, the input, the process in this codebase, the success response, the failure response, and the evidence. An item whose evidence is a screen or a code path, rather than a pytest name, is not marked Pass in Chapter 6.

FR-02 Login

Endpoint. POST /api/auth/login/

Input. email and password.

Process. The serializer lower-cases the email, calls Django authentication, and asks SimpleJWT for the token pair.

Success. HTTP 200 with access, refresh and the user object. A LOGIN audit row stores detail success.

Failure. HTTP 401. The failed row, when written, has a null user and detail failed:.

Evidence. test_login_and_me.

FR-04 Profile

Endpoint. GET /api/auth/me/

Input. Bearer access token only.

Process. The user id is taken from the token. A client-supplied id is ignored.

Success. HTTP 200 with id, name, email, role, active and created_at. The password hash is not in the payload.

Failure. HTTP 401 when the token is missing or rejected.

Evidence. test_login_and_me.

FR-05 Provision staff

Endpoint. POST /api/users/

Input. name, email, password, and role FACULTY or ADMIN.

Process. IsAdmin runs first. The serializer will not create a STUDENT on this URL.

Success. HTTP 201 and a staff row whose role is the one requested.

Failure. A student caller receives 403. A duplicate email receives 400.

Evidence. test_admin_creates_faculty_and_student_cannot.

FR-06 Maintain accounts

Endpoint. GET and PATCH /api/users/{id}/

Input. optional role filter; name, role and active.

Process. The view refuses to set active false on the caller’s own id.

Success. The list or the updated user.

Failure. Self-deactivation returns 400.

Evidence. Figure 4.24 and the admin-create test.

FR-07 Inactive account

Endpoint. POST /api/auth/login/

Input. credentials of a row whose active flag is false.

Process. The is_active property reads the active column, so Django authentication fails.

Success. HTTP 401 and no token.

Failure. The row stays inactive.

Evidence. test_inactive_user_cannot_authenticate.

FR-08 Subjects

Endpoint. /api/subjects/

Input. name.

Process. The name is stripped. A case-insensitive match counts as a duplicate. Delete uses PROTECT.

Success. Subject JSON, or 204 on a delete that nothing references.

Failure. 400 on a duplicate. 409 when a topic, question or exam still points at the subject.

Evidence. used by the bank screens; no standalone pytest.

FR-09 Topics

Endpoint. /api/topics/?subject=

Input. subject id and name.

Process. Unique (subject_id, name). The question serializer also rejects a topic from another subject.

Success. Topic JSON.

Failure. 400 on a clash or a mismatched question.

Evidence. test_topic_must_match_subject.

FR-11 Descriptive item

Endpoint. POST /api/questions/

Input. type DESCRIPTIVE, text, difficulty, marks, and no options.

Process. Any option list is a validation error. A student caller is rejected by IsFacultyOrAdmin.

Success. HTTP 201 with an empty options array.

Failure. 400 if options are present. 403 for a student.

Evidence. test_descriptive_rejects_options_and_student_cannot_write.

FR-12 Active flag

Endpoint. POST /api/questions/{id}/activate/ and deactivate/

Input. question id.

Process. Only the active flag changes. Later draw and add-to-paper require the flag to be true.

Success. HTTP 200 and the flag.

Failure. An inactive question remains stored and is skipped by selection.

Evidence. test_deactivate_question; activate has no separate case.

FR-13 Examined question

Endpoint. PATCH /api/questions/{id}/

Input. stem, marks, type or options.

Process. If the question already has attempt links, those content fields are rejected.

Success. HTTP 400 and the stored stem is unchanged.

Failure. A sitting cannot be rewritten after it has started.

Evidence. serializer rule; Table 6.4 records the missing pytest.

FR-14 Create draft

Endpoint. POST /api/exams/

Input. subject, title, window, duration, pass mark, negative mark.

Process. Status is set to DRAFT in the view, not from the client. End must follow start. Duration must fit in the window and be at least one minute.

Success. HTTP 201 with status DRAFT.

Failure. 400 when duration is longer than the window. 403 for a student.

Evidence. test_duration_cannot_exceed_window and the faculty-create test.

FR-15 Draft-only change

Endpoint. PATCH and DELETE /api/exams/{id}/

Input. a new window or a delete.

Process. Both require DRAFT. Delete also stops if any attempt exists.

Success. Updated JSON, or 204.

Failure. 400 after the exam has been published.

Evidence. test_publish_rules.

FR-16 Fixed selection

Endpoint. POST /api/exams/{id}/questions/

Input. question_id.

Process. The exam must be DRAFT. The question must be active and have the same subject. The pair must be new.

Success. HTTP 201 and a longer paper.

Failure. 400 for the wrong subject, an inactive question, or a duplicate.

Evidence. test_question_must_match_subject_and_be_active.

FR-17 Draw

Endpoint. POST /api/exams/{id}/draw/

Input. counts for easy, medium, hard and descriptive.

Process. The service samples without replacement from the active bank of that subject. A short pool raises before the caller can observe a half-built paper.

Success. HTTP 200 and the new question count.

Failure. 400, and the test asserts the previous count is unchanged.

Evidence. test_rule_based_draw.

FR-19 Close

Endpoint. POST /api/exams/{id}/close/ and unpublish/

Input. exam id.

Process. Close requires PUBLISHED. Unpublish requires PUBLISHED and no attempts.

Success. CLOSED, or DRAFT again.

Failure. A closed exam rejects start with 400.

Evidence. test_closed_exam_rejects_new_start.

FR-20 Catalogue

Endpoint. GET /api/student/exams/

Input. student token.

Process. Drafts are omitted. The state is UPCOMING, AVAILABLE, MISSED, IN_PROGRESS, SUBMITTED or EVALUATED from the clock and that student's attempt only.

Success. One JSON object per visible exam, including attempt_id and result_published.

Failure. Another student's attempt id is not copied onto this list.

Evidence. Figure 4.16; no pytest assertion.

FR-21 Who may start

Endpoint. POST /api/exams/{id}/start/

Input. student token.

Process. The service requires role STUDENT, status PUBLISHED, and now inside the half-open window, before it inserts.

Success. HTTP 201 and a paper that has no is_correct field.

Failure. 400 outside the window. 403 if a faculty token is used.

Evidence. test_one_attempt_and_role_window_rules.

FR-25 Save

Endpoint. PUT /api/attempts/{id}/answers/

Input. question_id with either a selected option or descriptive text.

Process. The attempt must still be IN_PROGRESS. An MCQ option is accepted only when attempt_option_orders contains that pair.

Success. HTTP 200 and the stored selection.

Failure. 400 for a foreign option. 409 once the attempt has left IN_PROGRESS.

Evidence. test_save_submit_and_double_submit.

FR-26 Owner

Endpoint. the same PUT and the attempt GET

Input. an attempt id.

Process. A student query adds user=request.user. Faculty are additionally blocked while the attempt is IN_PROGRESS.

Success. The owner’s paper.

Failure. HTTP 404 for a student asking for someone else’s id.

Evidence. owner filter in the attempt views.

FR-27 One submit

Endpoint. POST /api/attempts/{id}/submit/

Input. student token.

Process. The attempt row is locked. submitted_at is set, MCQs are scored, and finalize_result runs only if no descriptive mark is missing.

Success. Status SUBMITTED or EVALUATED.

Failure. The second call returns 409 and does not score again.

Evidence. test_save_submit_and_double_submit.

FR-31 When a result row appears

Endpoint. submit_attempt and award_descriptive

Input. the frozen paper.

Process. descriptive_pending stays true while any descriptive awarded_marks is null. An objective-only paper therefore creates the row at submit.

Success. A results row with published false.

Failure. The student still cannot read it.

Evidence. test_zero_descriptive_creates_unpublished_result.

FR-34 No live watching

Endpoint. GET /api/evaluation/attempts/{id}/

Input. faculty token during IN_PROGRESS.

Process. The view returns 403 before the answer serializer runs.

Success. No answer text is returned.

Failure. The student owner can still open their own attempt URL.

Evidence. EvaluationDetailView; browser evaluation page only lists submitted work.

FR-35 Student history

Endpoint. GET /api/results/me/ and /api/reports/student/history/

Input. student token.

Process. student_history copies totals only when published is true.

Success. Unpublished rows show the attempt status and a null result.

Failure. The detail URL for an unpublished row is 403, not an empty 200.

Evidence. test_reports_are_queries_and_role_gated.

FR-36 Faculty figures

Endpoint. GET /api/reports/faculty/exams/{id}/ and .../analysis/

Input. faculty or admin token.

Process. People, attempts and published results are counted separately. Analysis groups scored answers by topic, difficulty and type.

Success. The JSON fields used in Section 6.6.

Failure. A student receives 403.

Evidence. test_reports_are_queries_and_role_gated.

FR-37 Admin rollup

Endpoint. GET /api/reports/admin/summary/

Input. admin token.

Process. The faculty exam function is called once per exam. Published results are then counted again for the institution line.

Success. Exam rows plus published_results, pass_count, fail_count and average_total.

Failure. Faculty receives 403 on this URL even though faculty can open one exam report.

Evidence. test_reports_are_queries_and_role_gated.

FR-38 Audit list

Endpoint. GET /api/audit-logs/

Input. optional action, user and entity filters.

Process. The queryset is ordered by time descending and sliced at 500.

Success. Audit JSON for the administrator.

Failure. A student receives 403. Rows older than the slice are not claimed to have been displayed.

Evidence. test_audit_covers_login_start_save_submit_evaluate_publish.

FR-39 Health

Endpoint. GET /api/health/

Input. no token.

Process. This is a plain Django view so it can be called before login.

Success. HTTP 200 and the name ONLINE EXAMINATION SYSTEM.

Failure. A 200 here is not evidence that scoring or roles work.

Evidence. test_anonymous_api_requires_token.

3.9 Non-functional requirements

Table 3.4 — Non-functional requirements (implemented only)

ID	Requirement	Implemented evidence
NFR-01	Hash passwords with Argon2; accept PBKDF2	PASSWORD_HASHERS; SEC-AUTH-01 saw argon2 prefix
NFR-02	Password validators (length, common, numeric)	AUTH_PASSWORD_VALIDATORS
NFR-03	JWT access 8 h, refresh 1 day, Bearer	SIMPLE_JWT
NFR-04	Role permissions on protected views	permissions.py
NFR-05	Uniqueness of email, attempt, result	Uniques in schema.sql
NFR-06	ORM-parameterized SQL	Django ORM; no concatenated SQL in services
NFR-07	Application DB user cannot DDL	grants.sql; errno 1142 tests
NFR-08	Auditability of significant actions	audit_logs six actions
NFR-09	Timezone Asia/Kolkata	settings.TIME_ZONE
NFR-10	Do not claim unmeasured SLAs	No 3 s / 500 user / 95% figures in this report

Performance SLAs, concurrent-user numbers, encryption-at-rest, and UAT pass percentages were **not measured** and are not stated. CORS is an allow-list of the Vite origin for development; it is not a production security claim.

3.10 Validation rules

Table 3.5 — Serializer / screen validation

Input	Rule
Register name	Required, stripped
Email	Valid; stored lower-case; unique
Password	Django validators; write-only
Question text	≥ 5 characters
MCQ options	Exactly 4; exactly 1 is_correct
Exam title	≥ 3 characters
Duration	1–300 minutes and ≤ window length
Draw counts	Non-negative; sum ≥ 1
Awarded marks	0 ≤ value ≤ question marks

3.11 Detailed non-functional specifications

Table 3.4 is the index. The specifications below state the quality rule, the mechanism in this project, and the evidence boundary.

NFR-01 Password confidentiality

Quality rule. Passwords shall never be stored as plaintext or returned by an API.

Mechanism. Django set_password writes an Argon2 encoded value to users.password_hash. PBKDF2 and PBKDF2-SHA1 remain in the verifier list so an older encoded value can be checked and upgraded. Registration validates the password before saving. User serializers omit the field.

Evidence and boundary. The registration test checks the argon2 prefix and check_password; duplicate and weak values are rejected. There is no claim that the database column is encrypted independently of the hash.

NFR-02 Authentication and token lifetime

Quality rule. Protected JSON routes shall require an authenticated, active user.

Mechanism. SimpleJWT authenticates the Bearer header. Access lifetime is eight hours and refresh lifetime one day in settings. IsAnyRole and its subclasses additionally inspect the active flag. The React helper retries once after a 401 by calling refresh.

Evidence and boundary. Login, profile, anonymous and inactive-user tests cover the principal path. Refresh expiry and token tampering do not have dedicated tests and remain named gaps.

NFR-03 Authorization

Quality rule. A user shall perform only operations allowed for Student, Faculty or Admin.

Mechanism. HasRole is the base permission. IsStudent, IsFacultyOrAdmin and IsAdmin declare their role sets. Views do not accept a role from a request body. Object queries add the student owner where required.

Evidence and boundary. Security assertions cover user provisioning, exam creation, attempt start, manual marking, faculty/admin reports and the audit list. The access matrix in Chapter 7 is the human-readable version.

NFR-04 Transactional integrity

Quality rule. A failed multi-row business operation shall not leave a partial paper, attempt or evaluation.

Mechanism. Paper draw, start/freeze, answer persistence, submit and descriptive marking use transaction.atomic or a locked row where the state transition spans records. Database uniqueness and checks remain active if a serializer is bypassed.

Evidence and boundary. The draw test verifies the count is unchanged after an insufficient pool. Duplicate attempt and submit tests verify the first successful state remains.

NFR-05 Stable examination presentation

Quality rule. Refresh or navigation shall not change a student’s selected paper or MCQ option order.

Mechanism. Randomization occurs once, at start. attempt_questions.display_order and attempt_option_orders.display_order are then read on every response. The master question can be reused by another exam without changing this attempt.

Evidence and boundary. The deterministic reverse-RNG unit test reloads the rows and compares the stored order. Figure 4.17 shows a numbered palette based on those rows.

NFR-06 Server-authoritative time

Quality rule. A browser clock shall not extend an examination.

Mechanism. The server computes min(started_at + duration, exam.end_at). Every save/detail path calls ensure_open; once the deadline is reached, the service submits. The React countdown is presentation of remaining_seconds, not the authority.

Evidence and boundary. Expiry and shortened-window tests cover both branches. Network loss can delay the browser display but cannot change the stored start or end.

NFR-07 Auditability

Quality rule. The application shall record security-significant and examination-significant events.

Mechanism. Audit rows contain actor (nullable for a failed login), action, entity, entity id, timestamp and short detail. LOGIN, START, SAVE, SUBMIT, EVALUATE and PUBLISH are emitted from the code paths that commit those actions.

Evidence and boundary. The audit system test checks the six actions and Admin-only retrieval. The 500-row UI slice is disclosed; this is not represented as an immutable external audit archive.

NFR-08 Database least privilege

Quality rule. The running application account shall not own schema-changing privileges.

Mechanism. oes_migrator applies migrations. oes_app receives SELECT, INSERT, UPDATE and DELETE on oes.*. Application settings can select the role through environment variables; root is not the documented runtime login.

Evidence and boundary. Two direct MySQL tests issue CREATE TABLE as oes_app and observe an OperationalError, including error number 1142.

NFR-09 Maintainability and traceability

Quality rule. A requirement shall have an identifiable model, API/service and verification location.

Mechanism. The four Django apps separate accounts, catalogue, examination and audit. Business rules are in services rather than React components. Migrations version schema changes. FR ids are used in the SRS and Appendix A.

Evidence and boundary. Tables 5.5 and A.1 map requirement groups to files and tests. Full source is supplied separately, avoiding a report that is mostly copied code.

NFR-10 Honest performance and deployment scope

Quality rule. The report shall not state an unmeasured response time, concurrency level, availability or production security control.

Mechanism. Queries use indexes and related-object loading, but no load test was run. Gunicorn is the named production WSGI server; the demonstration is HTTP. HTTPS, secret rotation, backups, rate limiting and monitoring are deployment work.

Evidence and boundary. There is therefore no fabricated three-second SLA, 500-user claim, 99.9% availability or UAT percentage. These are explicit limitations rather than failed tests.

Chapter 4 System Design

4.1 Architecture

Three tiers, one stack. Browsers load the React SPA. The SPA calls JSON routes with Authorization: Bearer. Django middleware (CORS, Security, Common, CSRF, Auth, X-Frame-Options) precedes APIView → serializer → service → ORM → MySQL 8. Gunicorn is the production WSGI entry. Reports never write a report table.

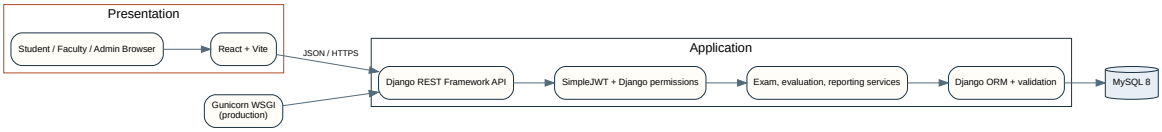


Figure 4.1 — Three-tier architecture: React, DRF/Django services, MySQL 8, Gunicorn.

4.2 Modularization

Table 4.1 — Modules, tables and process logic

Module	App	Tables	Process logic
1 Authentication	accounts	users	Register forces STUDENT; login issues JWT; inactive rejected; admin CRUD users
2 Question bank	catalog	subjects, topics, questions, options	CRUD; activate; MCQ shape; topic must match subject
3 Exam management	examination	exams, exam_questions	Draft-only edits; fixed attach or rule draw; publish/close/unpublish
4 Conduction	examination	attempts, frozen paper, answers	Eligibility → freeze → timer → save → submit/auto-submit
5 Evaluation	examination	answers, results	score_mcq; descriptive queue; finalize; publish
6 Reports	examination	queries only	student history, faculty analysis, admin summary
7 Audit / security	auditlog + grants	audit_logs	Six actions; DML-only oes_app
8 UI	React frontend/src	—	Role-gated routes, 15 s autosave, server timer display

4.3 Data flow diagrams

Context (Level 0). External entities are Student, Faculty and Admin. The single process is ONLINE EXAMINATION SYSTEM. Incoming flows: credentials, registration, question and exam definitions, answers, descriptive marks, publish decisions. Outgoing flows: JWT and role, frozen paper without keys, remaining seconds, save acknowledgements, unpublished/published results, faculty and admin aggregates, audit extracts. Data stores are internal and appear from Level 1.

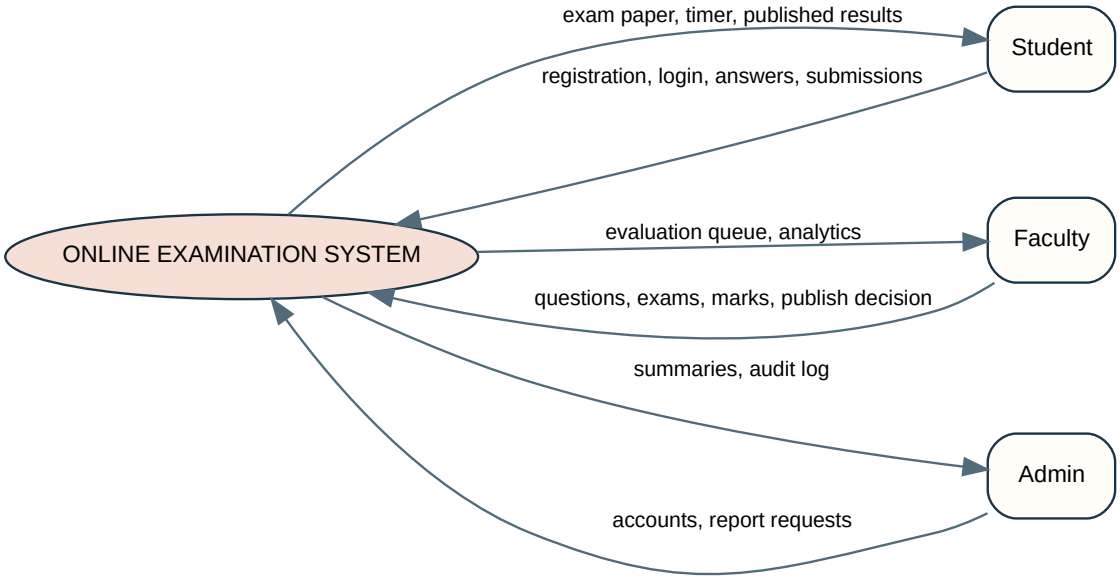


Figure 4.2 — Context-level DFD.

Level 1. The context process decomposes into seven numbered processes that keep the same actors. Stores are the implemented tables, not invented files.

Table 4.2 — Level 1 processes

P	Process	Inputs	Outputs	Store
1.0	Authenticate	email, password	JWT, user	users; audit_logs
2.0	Maintain question bank	question fields	bank rows	subjects, topics, questions, options
3.0	Manage exams	exam fields, draw counts	paper	exams, exam_questions
4.0	Conduct attempt	start, answers, submit	frozen paper, remaining seconds	attempt tables
5.0	Evaluate	submit, descriptive marks	awarded_marks, result	attempt_answers, results
6.0	Report	exam id / none	aggregates	queries only
7.0	Audit	action events	log rows	audit_logs

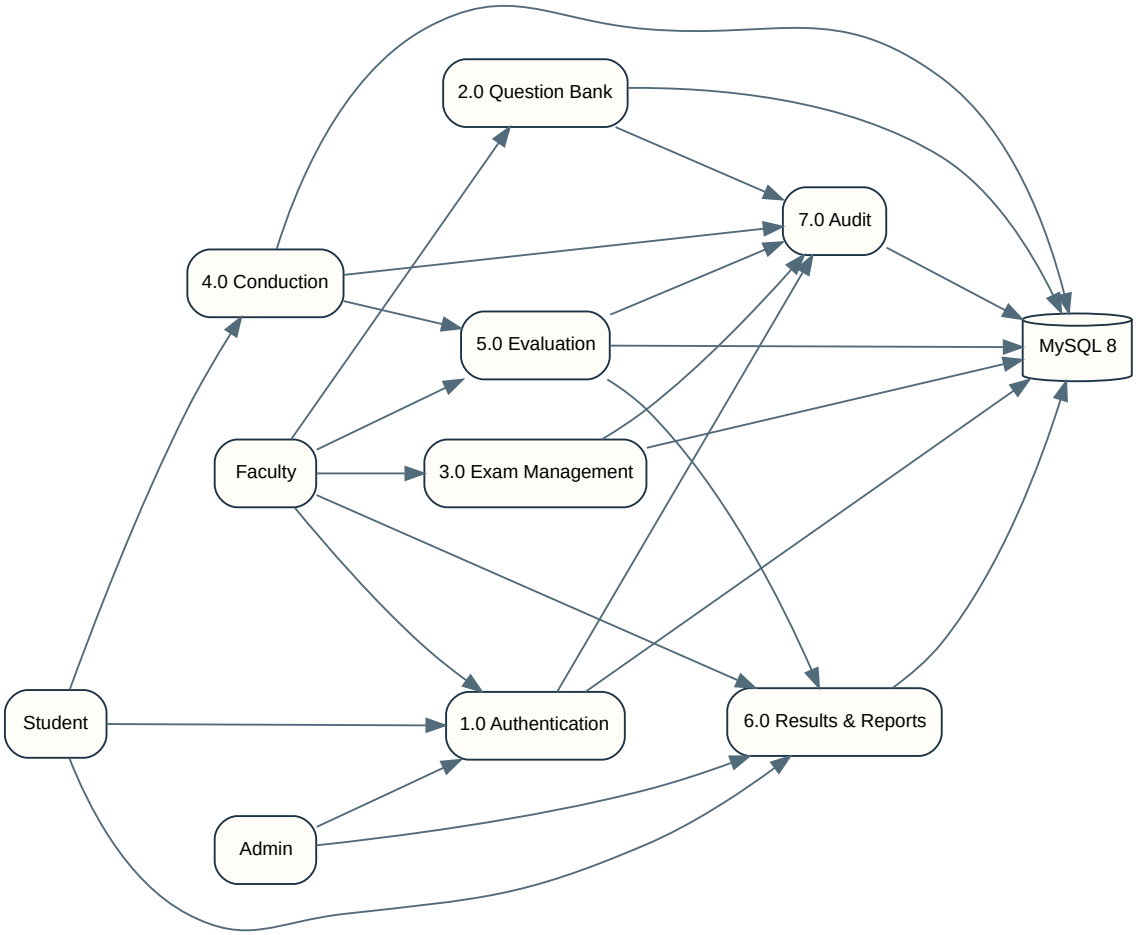


Figure 4.3 — Level 1 DFD.

Level 2 — conduction (process 4.0). 4.1 Eligibility checks role STUDENT, active, exam PUBLISHED, window open, and no prior attempt. 4.2 Freeze paper shuffles exam_questions into attempt_questions.display_order and shuffles four MCQ options into attempt_option_orders. 4.3 Timer computes deadline and remaining_seconds. 4.4 Save validates each option against that frozen order. 4.5 Submit is manual or automatic via ensure_open. Stores: exams, exam_questions, exam_attempts, attempt_questions, attempt_option_orders, attempt_answers.

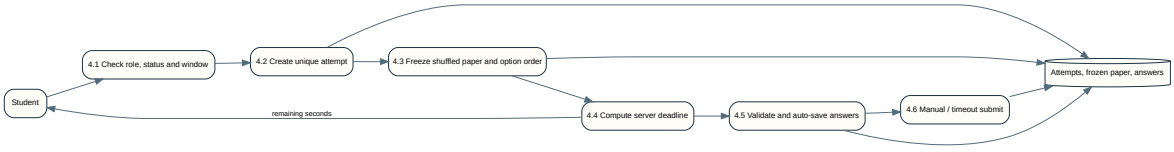


Figure 4.4 — Level 2 DFD: conduction.

Level 2 — evaluation (process 5.0). 5.1 Fetch a submitted attempt. 5.2 Auto-score MCQ. 5.3 If any descriptive awarded_marks is null, wait in the faculty queue. 5.4 Manual score in range. 5.5 finalize_result writes PASS/FAIL unpublished. 5.6 Publish flips the flag; only then does the student GET succeed. No result is shown to the student between 5.5 and 5.6.

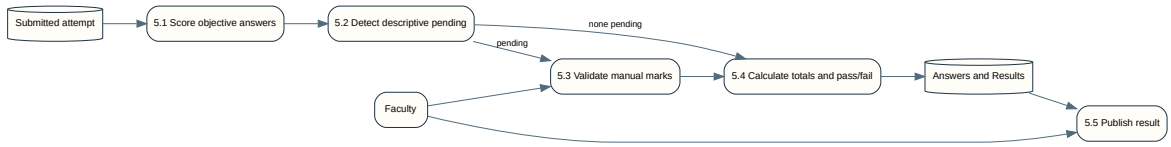


Figure 4.5 — Level 2 DFD: evaluation.

4.4 ER model

Thirteen domain tables match database/schema.sql. Questions are not owned by an exam. Reports are not an entity. attempt_option_orders is a justified refinement of the synopsis: randomized options must survive a refresh, so the permutation is stored.

Table 4.3 — Principal relationships

Relationship	Cardinality	On delete
Subject–Topic / Subject–Question / Topic–Question	1:N	PROTECT
Question–QuestionOption	1:N	CASCADE options
Exam–ExamQuestion–Question	N:M	exam CASCADE; question PROTECT
Exam–ExamAttempt–User	N:M unique pair	PROTECT
Attempt–AttemptQuestion–Question	N:M unique pair	attempt CASCADE; question PROTECT
AttemptQuestion–AttemptOptionOrder–Option	N:M	AQ CASCADE; option PROTECT
Attempt–Result	1:1	CASCADE
User–AuditLog	1:N	SET_NULL

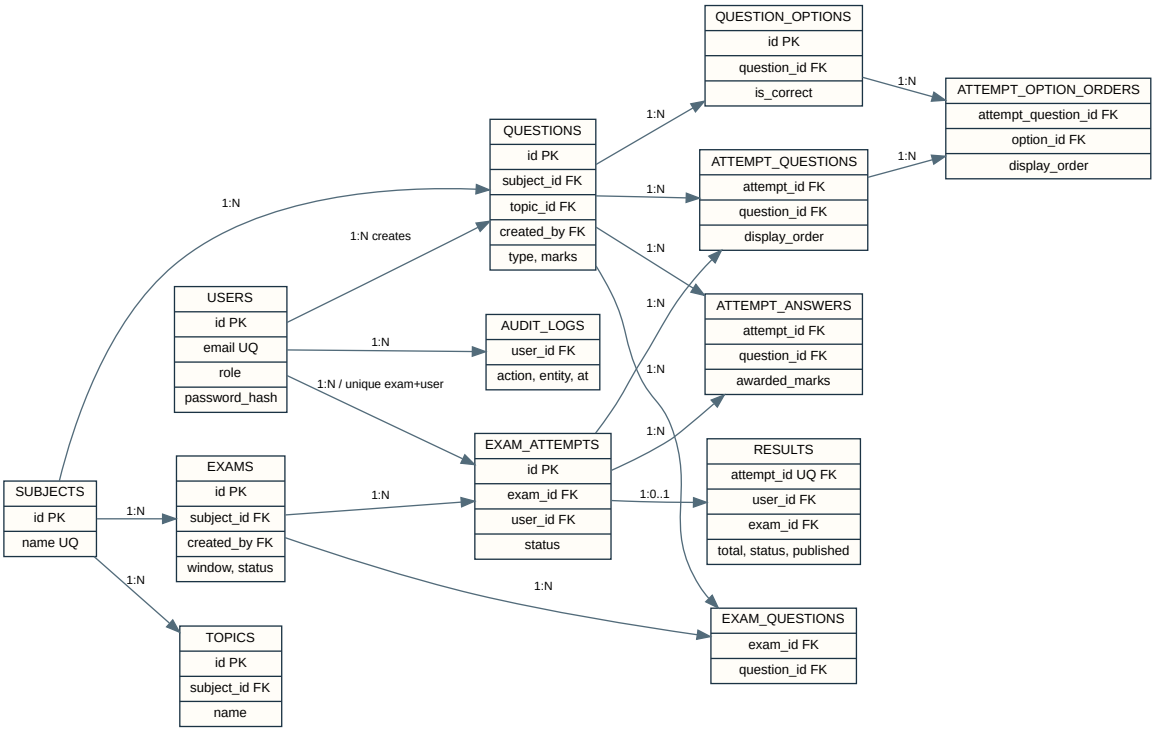


Figure 4.6 — ER diagram of the implemented schema.

4.5 Data dictionary

Types are MySQL as dumped in schema.sql. One table follows.

users

Table 4.4 — users

Column	Type	Null	Key	Notes
id	bigint AI	N	PK	
password_hash	varchar(128)	N		Django hash; Argon2 prefix argon2
last_login	datetime(6)	Y		
name	varchar(150)	N		
email	varchar(254)	N	UNIQUE	login identifier
role	varchar(20)	N	INDEX	CHECK STUDENT/FACULTY/ADMIN
active	tinyint(1)	N		

Column	Type	Null	Key	Notes
created_at	datetime(6)	N		

subjects

Table 4.5 — subjects

Column	Type	Null	Key
id	bigint AI	N	PK
name	varchar(150)	N	UNIQUE

topics

Table 4.6 — topics

Column	Type	Null	Key
id	bigint AI	N	PK
name	varchar(150)	N	UNIQUE with subject_id
subject_id	bigint	N	FK → subjects

questions

Table 4.7 — questions

Column	Type	Null	Key / checks
id	bigint AI	N	PK
type	varchar(20)	N	MCQ / DESCRIPTIVE
text	longtext	N	
difficulty	varchar(10)	N	EASY / MEDIUM / HARD
marks	decimal(6,2)	N	> 0
explanation	longtext	Y	shown after publish
active	tinyint(1)	N	INDEX
created_by_id	bigint	N	FK users
subject_id	bigint	N	FK; bank index
topic_id	bigint	N	FK topics

question_options

Table 4.8 — question_options

Column	Type	Null	Key
id	bigint AI	N	PK
option_text	longtext	N	
is_correct	tinyint(1)	N	
question_id	bigint	N	FK questions CASCADE

Exactly four rows and one correct option are enforced in Django validation and in freeze_paper, not by a CHECK on this table.

exams

Table 4.9 — exams

Column	Type	Null	Checks
id	bigint AI	N	PK
title / instructions	varchar(200) / longtext	N	
start_at, end_at	datetime(6)	N	end_at > start_at
duration_minutes	int unsigned	N	> 0
pass_marks / negative_mark	decimal(6,2)	N	≥ 0
status	varchar(20)	N	DRAFT / PUBLISHED / CLOSED
created_by_id / subject_id	bigint	N	FK users / subjects

exam_questions

Table 4.10 — exam_questions (master paper)

Column	Type	Null	Key
id	bigint AI	N	PK
exam_id	bigint	N	UNIQUE with question_id
question_id	bigint	N	FK questions PROTECT

exam_attempts

Table 4.11 — exam_attempts

Column	Type	Null	Key / checks
id	bigint AI	N	PK
started_at / submitted_at	datetime(6)	N / Y	
status	varchar(20)	N	IN_PROGRESS / SUBMITTED / EVALUATED
exam_id, user_id	bigint	N	UNIQUE one_attempt_per_student

attempt_questions

Table 4.12 — attempt_questions (frozen paper)

Column	Type	Null	Key
id	bigint AI	N	PK
display_order	int unsigned	N	UNIQUE with attempt_id
question_id	bigint	N	UNIQUE with attempt_id
attempt_id	bigint	N	FK CASCADE

attempt_option_orders

Table 4.13 — attempt_option_orders

Column	Type	Null	Key
id	bigint AI	N	PK
display_order	int unsigned	N	UNIQUE with attempt_question_id
option_id	bigint	N	UNIQUE with attempt_question_id
attempt_question_id	bigint	N	FK CASCADE

attempt_answers

Table 4.14 — attempt_answers

Column	Type	Null	Key
id	bigint AI	N	PK
descriptive_text	longtext	Y	
awarded_marks	decimal(6,2)	Y	null until scored
evaluated_at / evaluated_by_id	datetime / bigint	Y	FK users
selected_option_id	bigint	Y	FK options
attempt_id, question_id	bigint	N	UNIQUE pair
marked_for_review	tinyint(1)	N	DEFAULT 0

results

Table 4.15 — results

Column	Type	Null	Checks
id	bigint AI	N	PK
objective_marks, descriptive_marks, total_marks	decimal(6,2)	N	≥ 0
max_marks	decimal(6,2)	N	
status	varchar(10)	N	PASS / FAIL
published	tinyint(1)	N	default false
attempt_id	bigint	N	UNIQUE
user_id + exam_id	bigint	N	UNIQUE one_result_per_student_exam

audit_logs

Table 4.16 — audit_logs

Column	Type	Null	Checks
id	bigint AI	N	PK
action	varchar(32)	N	LOGIN START SAVE SUBMIT EVALUATE PUBLISH
entity / entity_id	varchar(32) / bigint	N / Y	
at	datetime(6)	N	INDEX with user
detail	longtext	N	
user_id	bigint	Y	FK SET_NULL

4.6 Normalization and data integrity

1NF. MCQ options are rows in question_options, not option_a…option_d. Frozen option order is rows, not a CSV.

2NF. Junction keys are complete pairs such as (exam_id, question_id) and (attempt_id, question_id). Non-key attributes (display_order, awarded_marks) depend on the full pair.

3NF. Subject name is not copied onto every question or exam row. Averages and participation are computed in reports.py. results stores the published evaluation outcome, which is the business record rather than a transient report.

Table 4.17 — Integrity types applied to OES

Integrity class	OES example
Entity	Primary keys on all 13 domain tables; unique email; unique exam+user attempt; unique user+exam result
Referential	Foreign keys with PROTECT on subjects/questions used by history, CASCADE on attempt-owned frozen rows, SET_NULL on audit user
Domain	CHECKs on role, question type/difficulty, exam status, attempt status, result status, positive marks, window order
User-defined	Serializer: four MCQ options and one key; pass marks ≤ paper total; duration ≤ window; option must belong to frozen order

MySQL snippets from the implemented schema (not from the Job Portal sample):

```
CONSTRAINT `one_attempt_per_student` UNIQUE (`exam_id`,`user_id`),
CONSTRAINT `exam_window_order` CHECK ((`end_at` > `start_at`)),
CONSTRAINT `result_objective_non_negative` CHECK ((`objective_marks` >= 0)),
CONSTRAINT `user_role_valid` CHECK ((`role` in ('STUDENT','FACULTY','ADMIN')));
```

4.7 UML views

Use case. Student: register, login, start/continue, save, submit, view published result. Faculty: bank, exams, draw, publish/close, mark, publish result, exam report. Admin: those plus users, summary, audit. Start includes eligibility; submit includes auto-score.

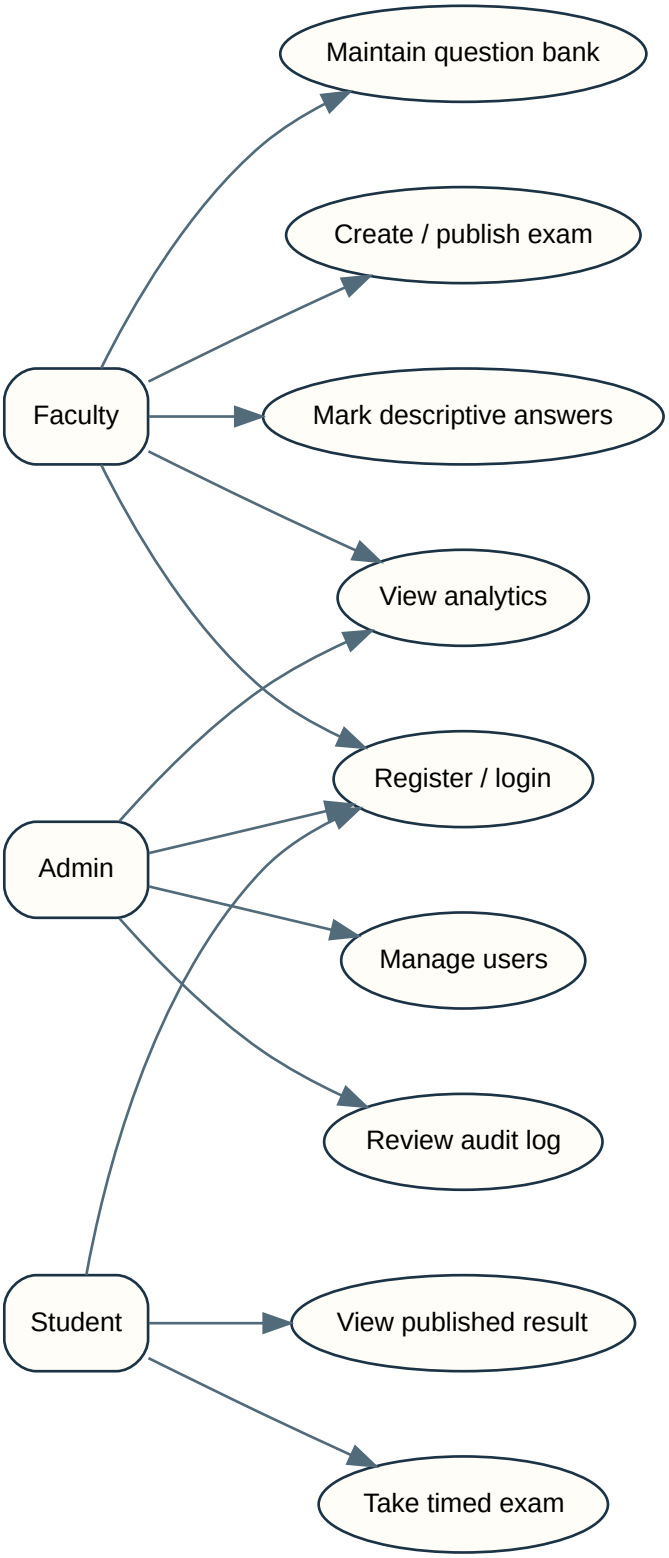


Figure 4.7 — Use-case diagram.

Class. Django models listed in §4.5 plus services and permission classes. There is no Report class.

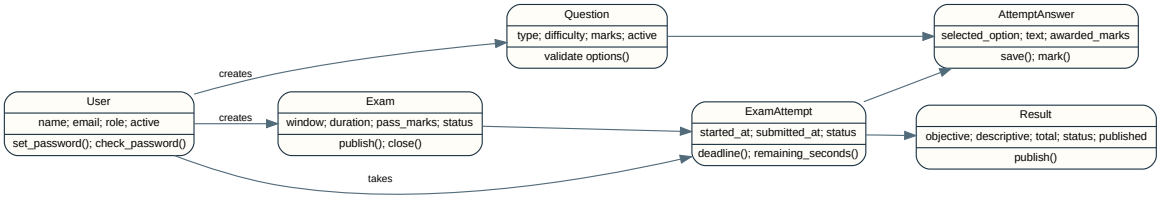


Figure 4.8 — Class diagram of principal models and services.

Activity (student). List exams → if AVAILABLE, start → freeze → loop {navigate, save} until submit or remaining_seconds = 0 → wait for publish → view result. **Activity (faculty).** Open queue → needs_marking or ready_to_publish → post marks → finalize → publish.

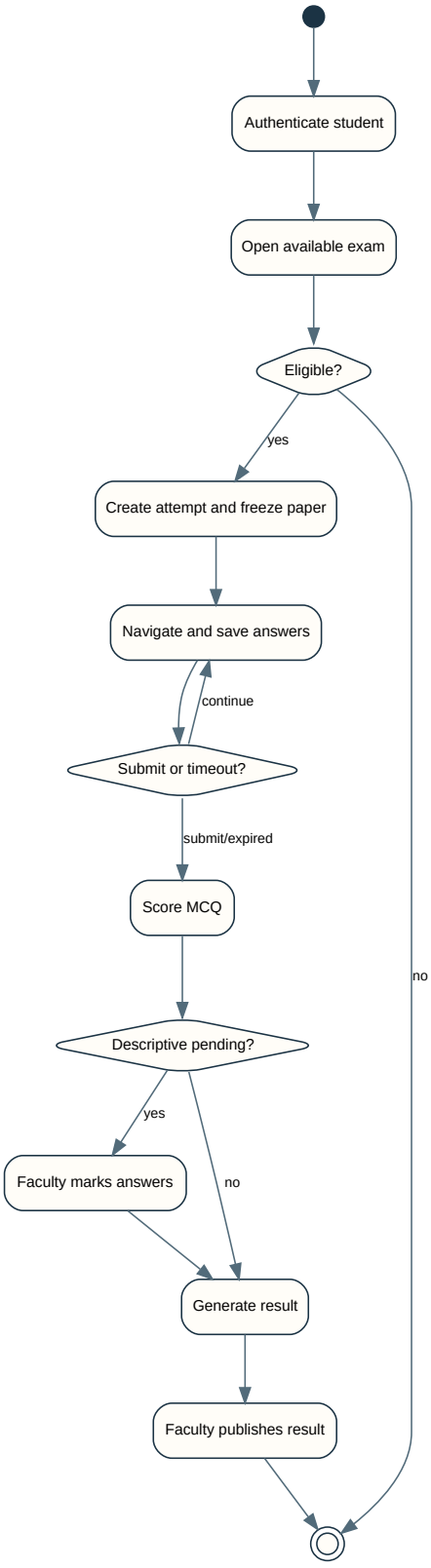


Figure 4.9 — Combined activity of attempt through publication.

Sequence. Browser → API → services → ORM. Remaining seconds are computed on the server on each save.

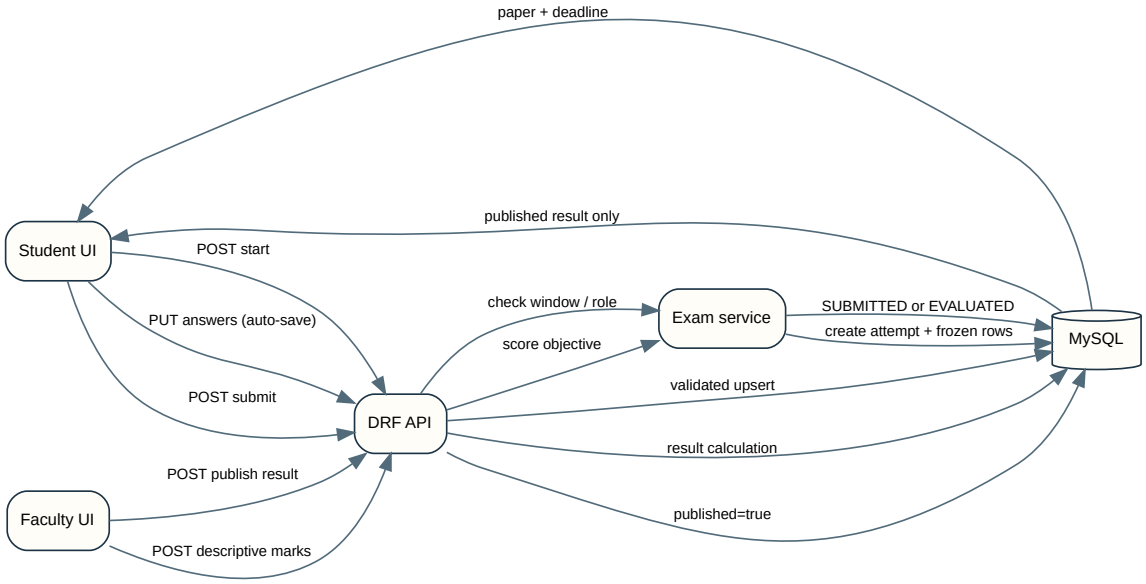


Figure 4.10 — Sequence among browser, API, services and MySQL.

State. Exam: DRAFT → PUBLISHED → CLOSED; unpublish to DRAFT only with zero attempts. Attempt: (no row) → IN_PROGRESS → SUBMITTED → EVALUATED. Published is a flag on results.



Figure 4.11 — Exam, attempt and result states.

4.8 Procedural design

Structured English matches examination/services.py. Two algorithms are drawn; the others are listed as pseudo-code.

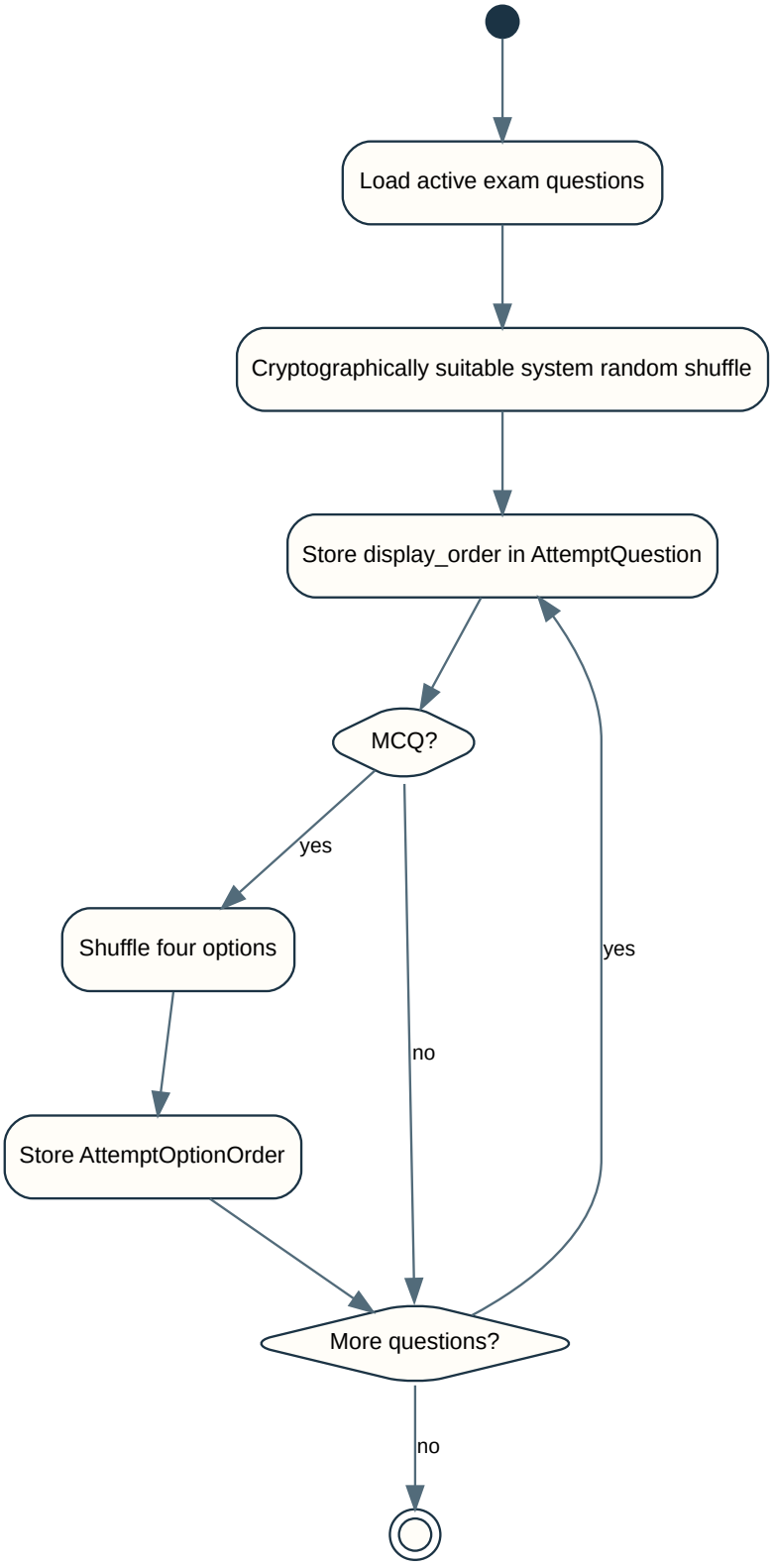


Figure 4.12 — Procedural design: freeze_paper (question shuffle and option shuffle).

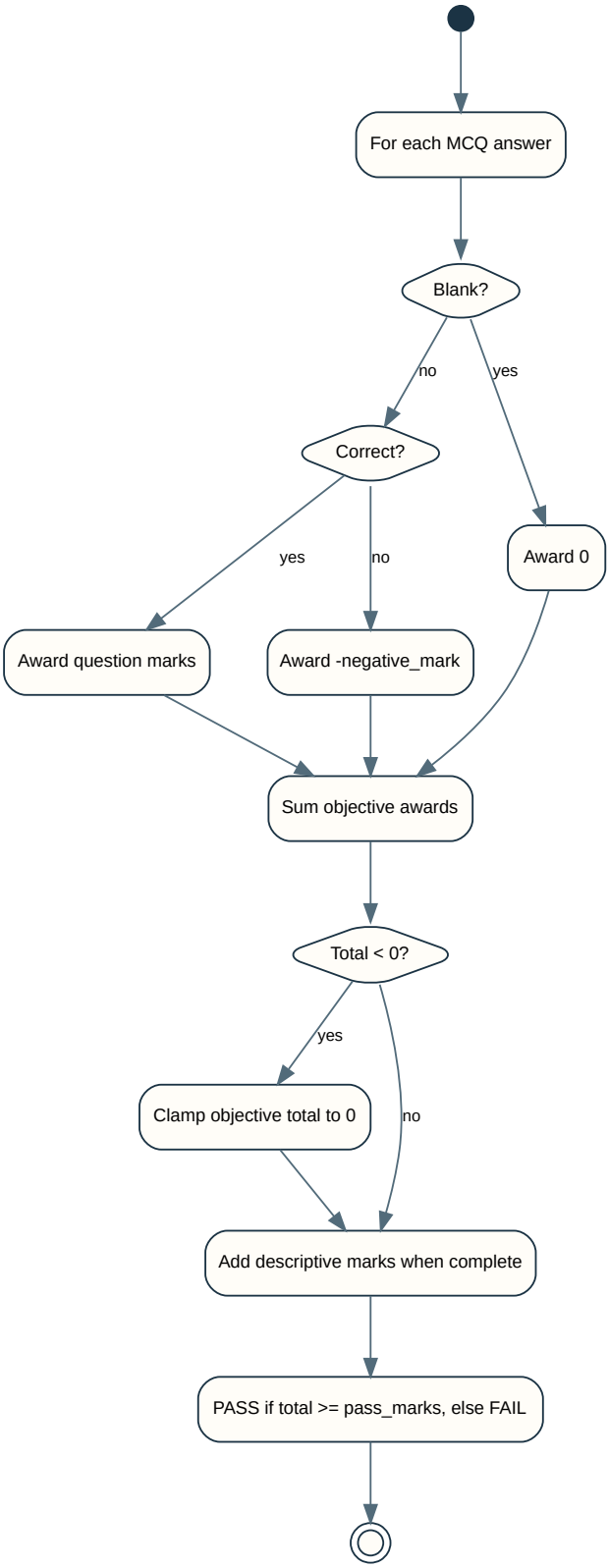


Figure 4.13 — Procedural design: score_mcq and objective floor.

Timer. deadline = min(started_at + duration, end_at); remaining = max(0, deadline − now) while IN_PROGRESS.

Auto-save. Client every 15 s and on option change. Server ensure_open; reject if not IN_PROGRESS; option must exist in frozen order.

Submit. Lock row; if already not IN_PROGRESS, manual path 409; set submitted_at; score MCQ; finalize; audit.

Result. If any descriptive on the frozen paper has awarded_marks null, no result row. Else PASS iff total ≥ pass_marks, published=false.

4.9 User interface design

Demonstration screens below were captured from the running React application on 5 October 2026 talking to the MySQL-backed API at http://127.0.0.1:8080. Values are seed/demo data (admin@oes.local, faculty@oes.local, student@oes.local and related attempts). They are **not** institutional production records. This disclaimer is not repeated under every figure.

Table 4.18 — UI inventory

Screen	Route	Principal controls
Sign in	/login	email, password
Registration	/register	name, email, password minLength 8; server forces STUDENT
Student exams	/student	state badge, Start / Continue / View result
Exam room	/student/attempts/:id	Server timer; answered/not-answered/marked filters; palette; mark/unmark; save; submit
Question bank	/faculty/questions	Keyword, topic, difficulty and type filters; result count; new question
Exam editor	/faculty/exams/:id	window, duration, paper, draw, Publish/Close
Evaluation	/faculty/evaluation	needs_marking, ready_to_publish, marks, Publish
Users / Audit	/admin/users and /admin/audit	create FACULTY/ADMIN; last 500 audit rows

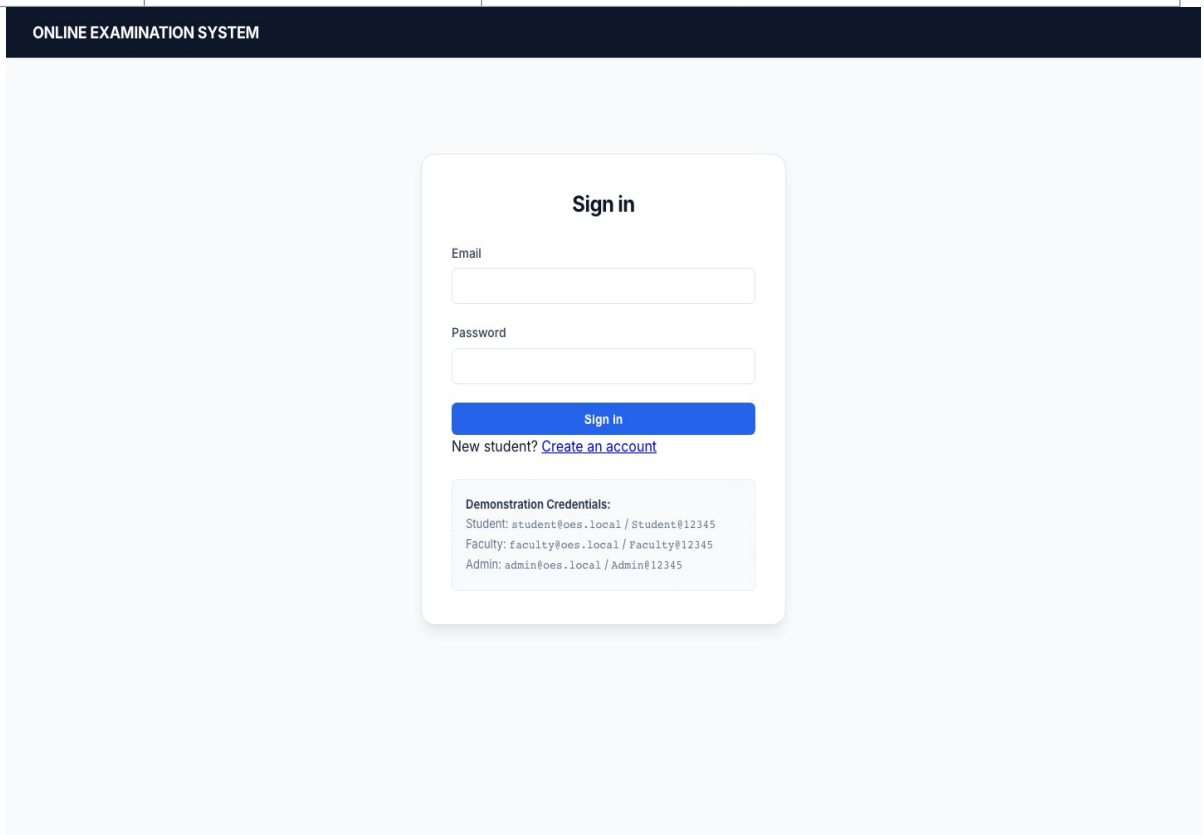


Figure 4.14 — Login. Email/password form and demonstration account reminder.

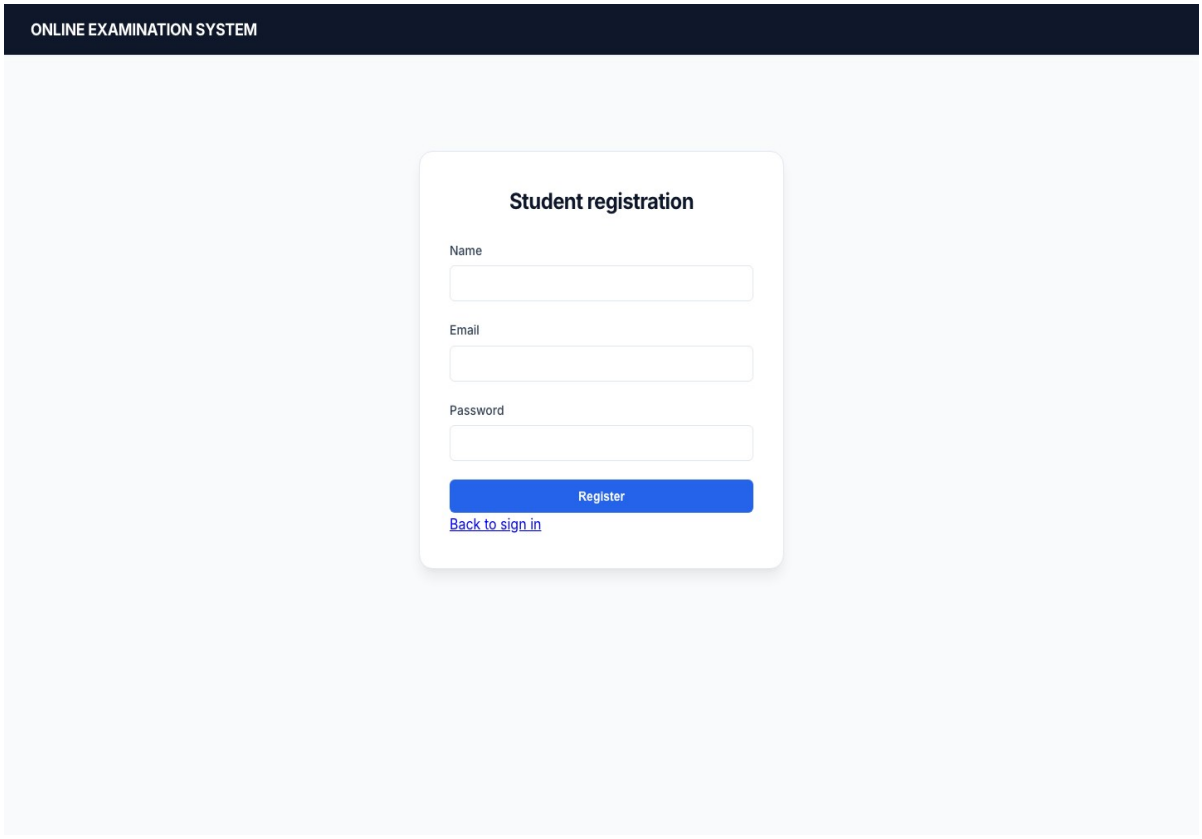


Figure 4.15 — Student registration. No role picker; server assigns STUDENT.

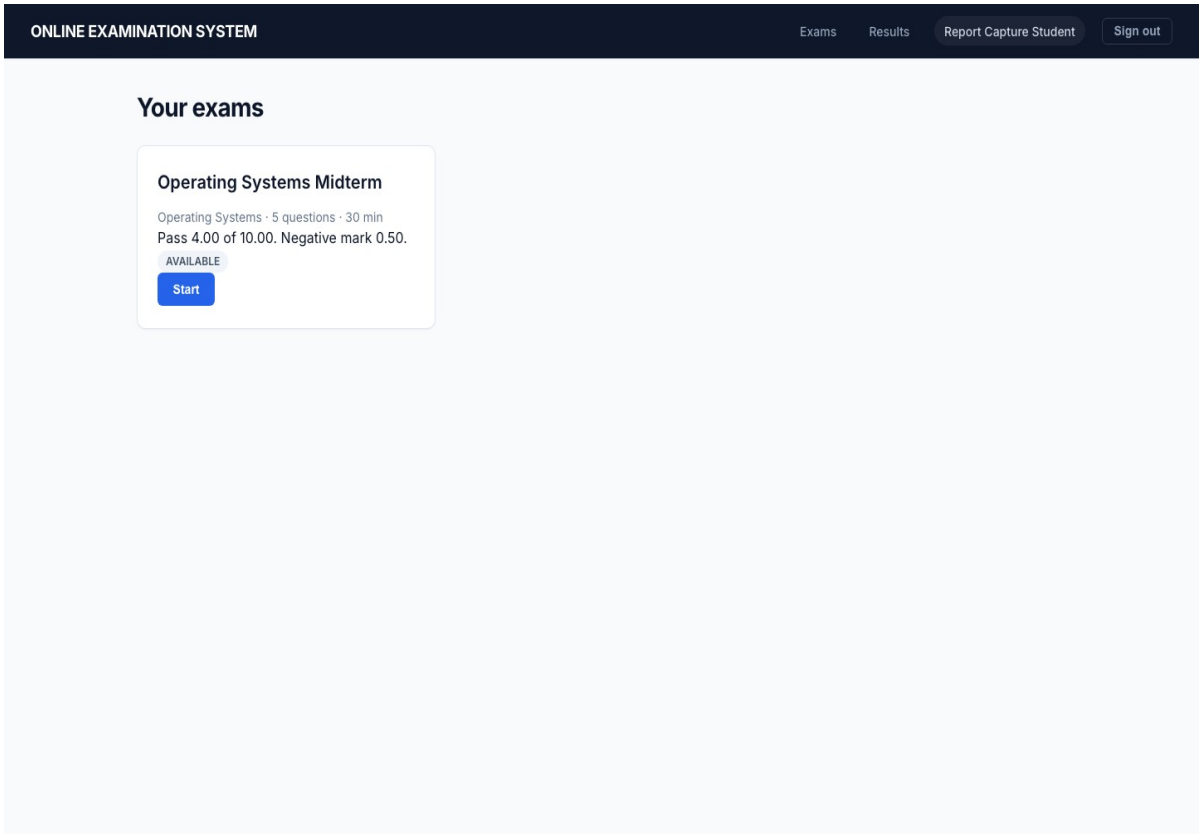


Figure 4.16 — Student dashboard with exam state and Start.

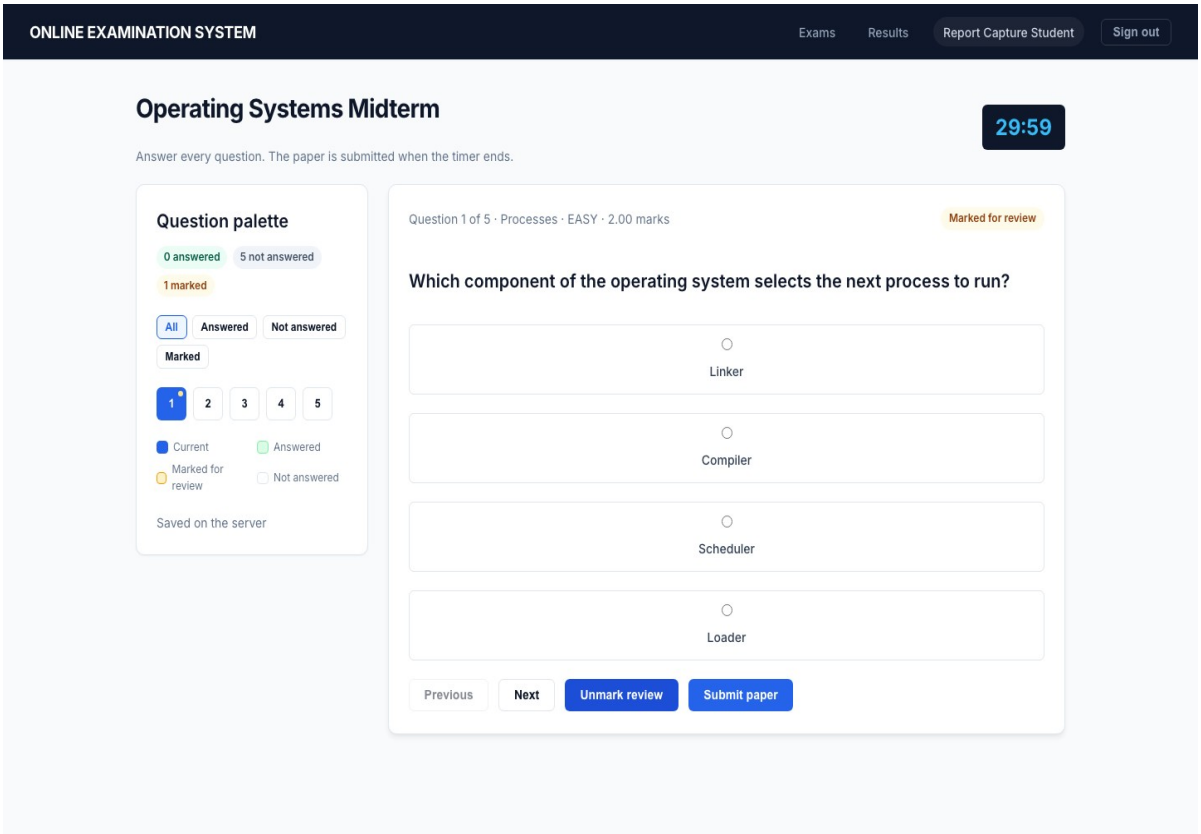


Figure 4.17 — Exam room: frozen paper, server timer, palette filters and mark-for-review state.

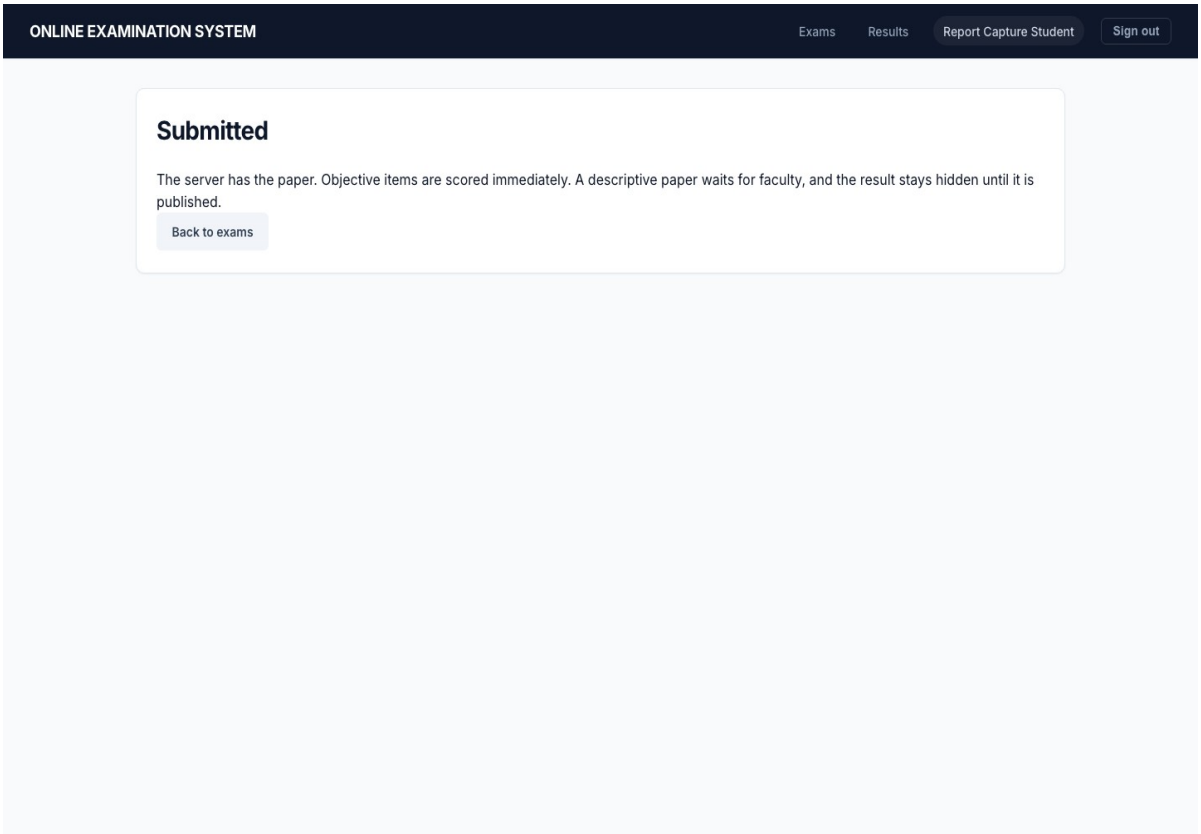


Figure 4.18 — Submission confirmation after the server accepts the paper.

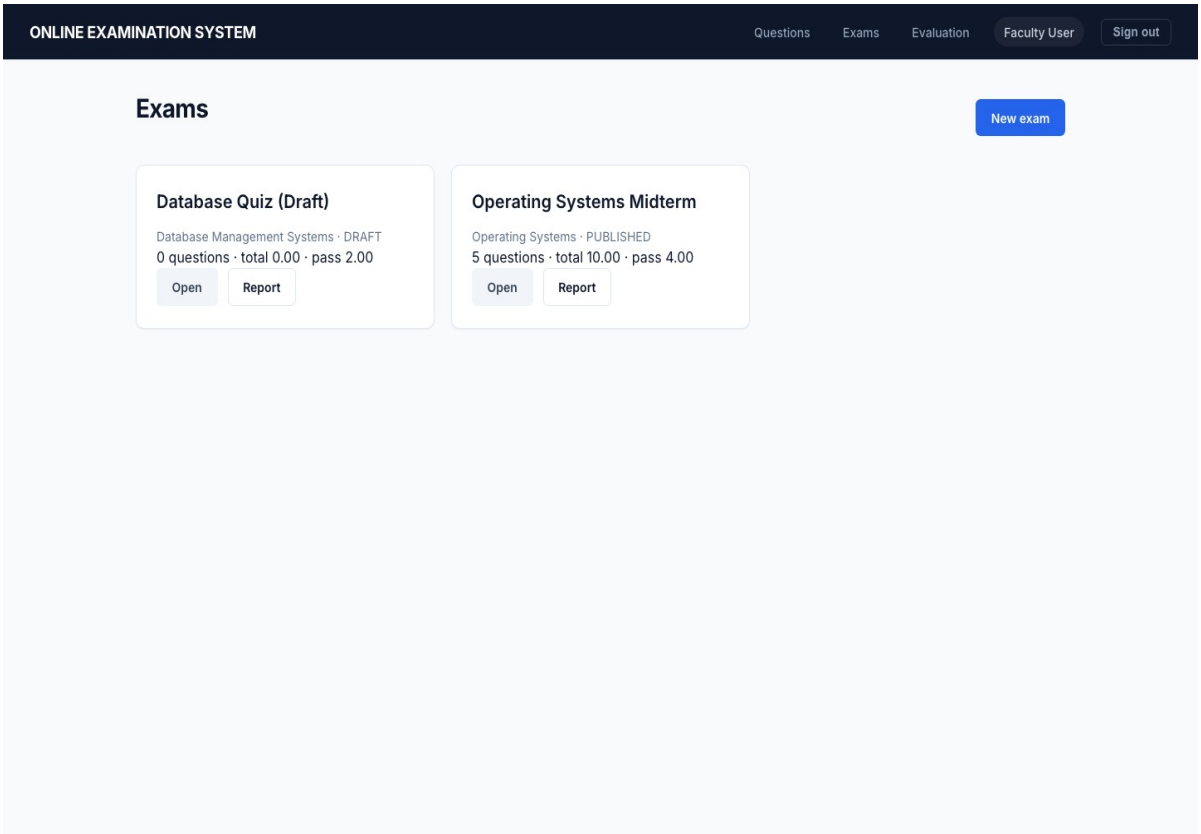


Figure 4.19 — Faculty exam list (draft and published).

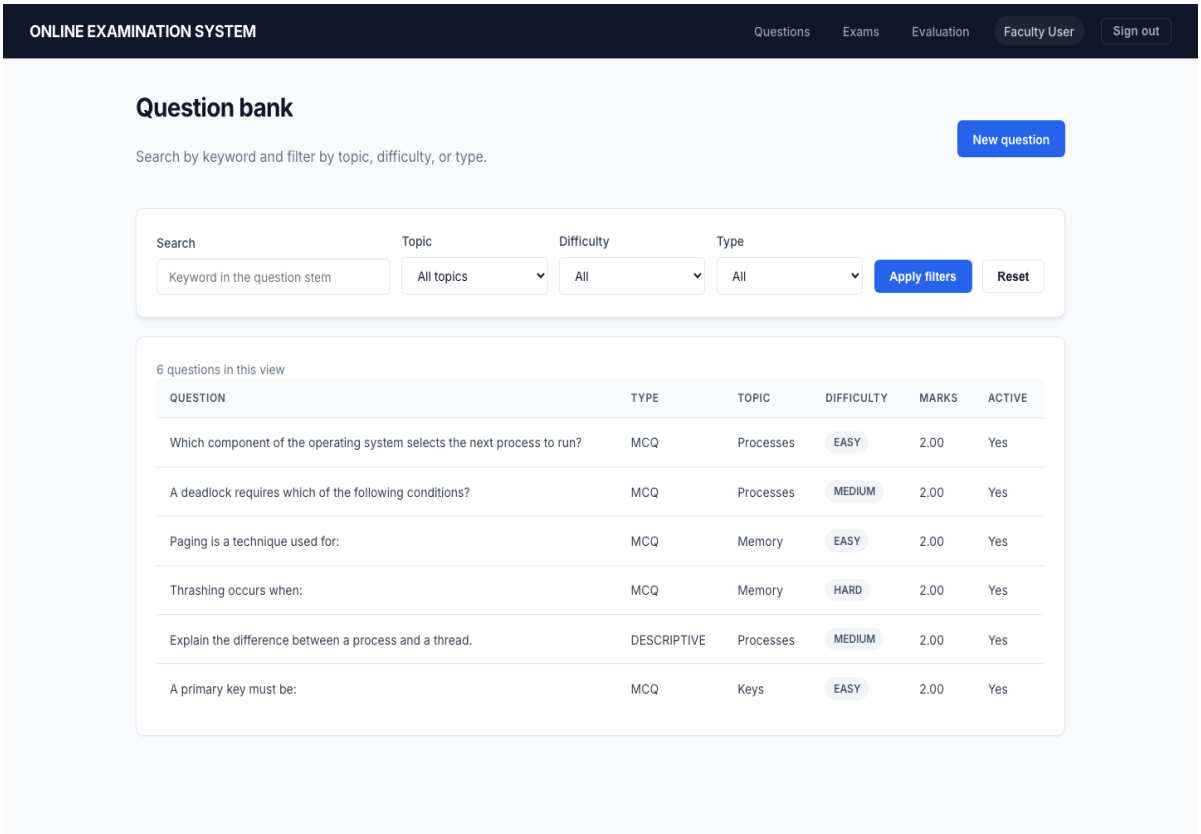


Figure 4.20 — Question bank with keyword, topic, difficulty and type filters.

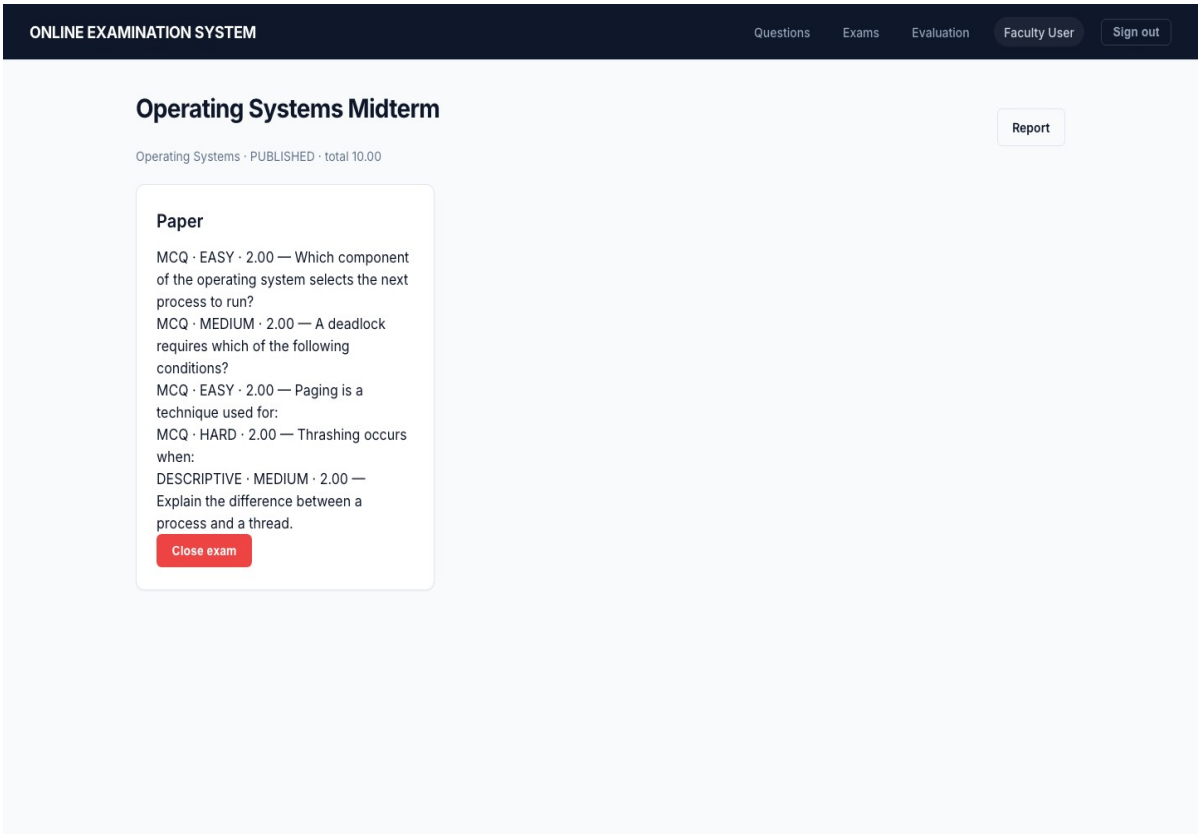


Figure 4.21 — Published paper and lifecycle controls.

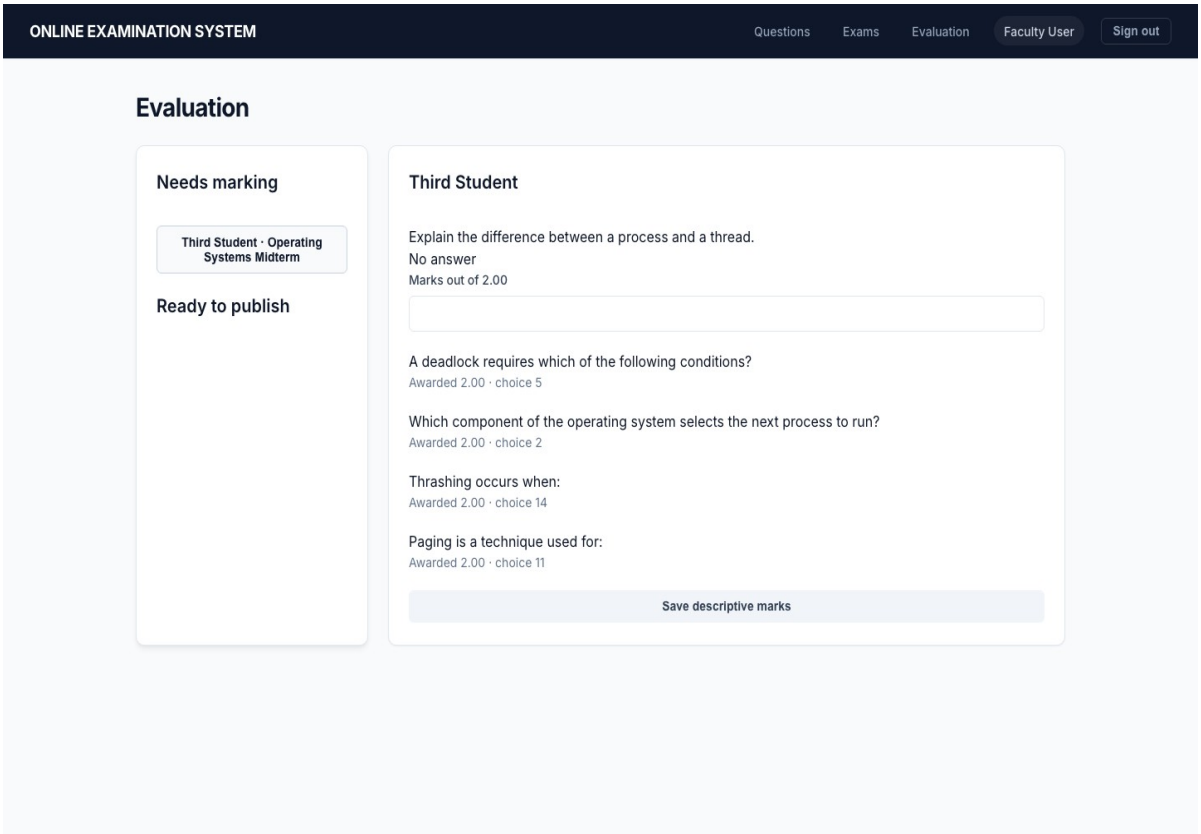


Figure 4.22 — Manual evaluation of a descriptive answer.

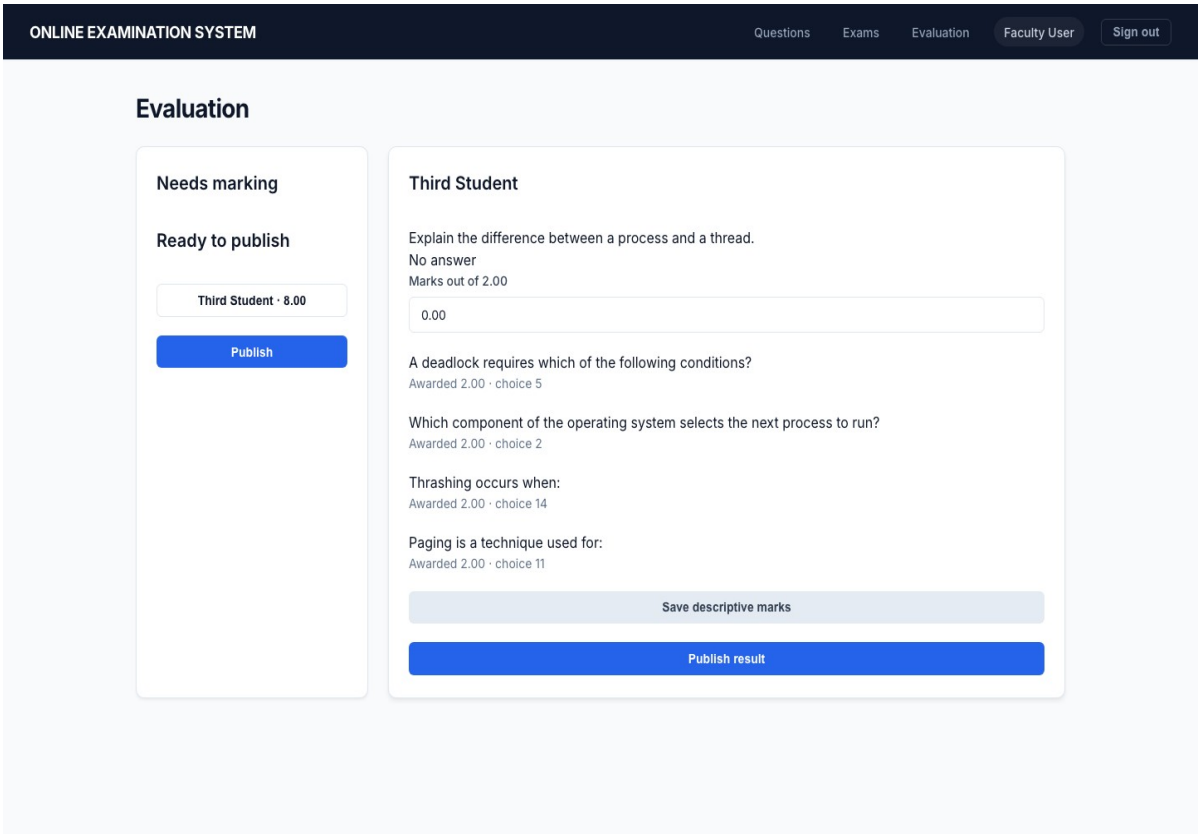


Figure 4.23 — Evaluated result ready to publish (publish is a distinct action).

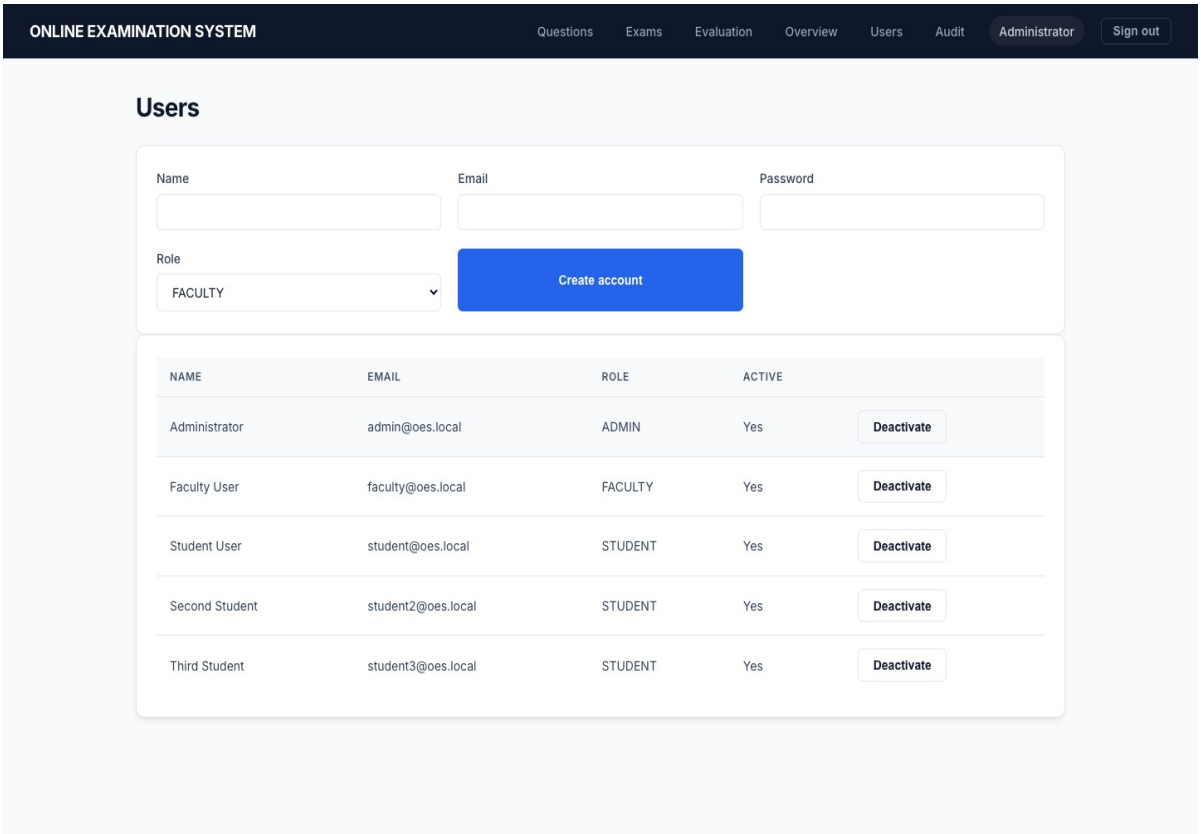


Figure 4.24 — Admin user management.

4.10 Design-time test cases

These cases were written from the SRS before packaging execution results. Chapter 6 records what was actually run.

Table 4.19 — Design-time test cases (not execution results)

ID	Module	Precondition	Steps	Expected
DT-S-01	Auth	Email unused	POST register with role ADMIN	Stored role STUDENT
DT-S-02	Auth	Active user	Login then GET /me/	200 with role
DT-F-01	Bank	Faculty token	MCQ with 3 options	400
DT-F-02	Exam	Draft empty paper	Publish	400
DT-ST-01	Conduction	Published window open	Student start twice	Second 409
DT-ST-02	Timer	Short remaining window	Read remaining_seconds	≤ window remainder
DT-E-01	Eval	Mixed paper submitted	Student GET result	403 until publish
DT-A-01	Admin	Student token	GET /api/audit-logs/	403

4.11 Process logic of each module

The synopsis asks for the process logic of every module. The text below is the logic actually coded. It is written as structured English so that a reader can follow it without opening the repository. Names in parentheses are functions or tables.

Authentication. The visitor submits name, email and password. The register serializer lower-cases the email, rejects a duplicate, runs Django’s password validators, and always stores role STUDENT. set_password writes an Argon2 hash into password_hash. Login looks up that email. A wrong password or an inactive row fails authentication and, for a failed login, an audit row is written with a null user and the detail failed:<email>. A success returns an access token (8 hours) and a refresh token (1 day) and writes LOGIN. Every later request must send Authorization: Bearer. IsAnyRole also requires active is true, so a token issued before deactivation stops working on the next call.

Question bank. Faculty or Admin chooses a subject and a topic that belongs to that subject. For an MCQ the payload must contain exactly four options and exactly one is_correct. A descriptive item must contain no options. Marks must be greater than zero. The row is stored with created_by set to the caller. Deactivate sets active false; a later draw or paper-add refuses inactive questions. A question that is already on an attempt cannot have its text, options or marks rewritten.

Exam management. Create stores a DRAFT with start, end, duration, pass mark and negative mark. End must be after start, and duration must not exceed the window. Questions are added one by one or by draw_questions. The draw takes non-negative counts for easy, medium, hard and descriptive, samples without replacement from active questions of the exam’s subject, and refuses the whole draw if any pool is short. Publish requires a non-empty paper, every question active and in the same subject, pass marks not above the paper total, and a window that has not already ended. After publish, the draft editor is closed. Close moves PUBLISHED to CLOSED. Unpublish is allowed only when the exam has no attempts.

Conduction. The student calls start. The service checks role STUDENT, status PUBLISHED, and start_at <= now < end_at. Inside one transaction it inserts exam_attempts and then freeze_paper. Freeze shuffles the master paper with random.SystemRandom, writes display_order, and for each MCQ checks that there are four options and one key before shuffling those options into attempt_option_orders. The same shuffle is not repeated on refresh; the GET reads the stored order. Save checks the attempt is still IN_PROGRESS and not past deadline. A selected option must be one of the frozen option rows for that question. Submit locks the attempt. A second submit returns 409. Expiry is not a special client mode: ensure_open calls the same submit path with auto=True when now >= deadline.

Evaluation. On submit, score_mcq sets each objective answer: blank scores 0, the key scores the question marks, any other frozen option scores minus negative_mark. objective_total sums those awards and returns 0 if the sum is negative. If any descriptive question on the frozen paper still has null awarded_marks, no result row is created and the attempt stays SUBMITTED. Faculty then posts marks. Each award must lie between 0 and the question marks. When the last descriptive item is scored, finalize_result sets EVALUATED, stores objective, descriptive, total and maximum, and sets PASS only when total is at least pass_marks. published stays false.

Results and reports. The student history query returns attempts, but totals only when published is true. The result-detail URL returns 403 while the row is unpublished, including the case of an objective-only paper that was finalized immediately. Faculty exam figures count active students, started attempts, and submitted attempts separately from published pass, fail and average. Topic analysis groups scored answers by topic, difficulty and type. Admin summary is the same faculty query repeated per exam, plus a published-result total. None of these write a report row.

Audit. record_audit inserts action, entity, entity id, time and a short detail. The admin list is filterable and capped at 500 rows. It is an operational extract, not a compliance archive and not a proctoring log.

4.12 Edge cases called out by the design

Table 4.20 — Edge cases and the observed check

Situation	Where it is decided	What the software does	How it was checked
Registration asks for role ADMIN	Register serializer	Stored role is STUDENT	pytest registration test; Figure 4.15 has no role picker
Duplicate email or password 12345678	Validators	HTTP 400, no second row	pytest auth tests
Inactive account, still holding an old token	Login and IsAnyRole	Login 401; later calls rejected	pytest inactive-user test
Start before start_at, after end_at, or after CLOSE	start_attempt	HTTP 400	pytest window and close tests
Second start by the same student	Unique (exam_id,user_id) and the transaction	HTTP 409	pytest one-attempt test
Refresh during the exam	Stored display_order	Same question and option order	pytest freeze test; Figure 4.17
Tab closed after a save	Last successful PUT	Answers remain; the timer is recomputed from started_at	save test plus the 15-second client loop
Save or GET after the deadline	ensure_open	Attempt is submitted on the server	pytest expiry test
Double submit	Status no longer IN_PROGRESS	HTTP 409	pytest submit test
Blank MCQ	score_mcq	Award 0, not the negative mark	pytest unanswered test
All MCQs wrong and negative marks large	objective_total	Objective total floored at 0	pytest floor test
Paper with no descriptive item	descriptive_pending	Result row exists immediately, still unpublished	pytest zero-descriptive test
Student opens the result early	MyResultDetailView	HTTP 403	pytest and the evaluation flow
Faculty marks the same descriptive item again after publish	award_descriptive	HTTP 400	pytest publish-lock assertion
Student calls the audit list	IsAdmin	HTTP 403	pytest audit test
oes_app issues CREATE TABLE	MySQL grants	Error 1142	pytest grant tests

The synopsis also names a scheduling queue. Eligibility is a comparison of the current time with start_at and end_at, so no queue structure was added. Temporary answers are rows in attempt_answers, not an in-memory array that would vanish on refresh. Question lookup during save is by the frozen attempt-question key.

4.13 Field validation on the screens

The React forms do not decide the business rules. They collect values and show the server's error detail. The rules below are the ones the server applies to those fields.

Table 4.21 — Screen fields and the server rule

Screen	Field	Rule in the server	What the user sees on failure
Registration	name	required, stripped, max 150	400 and no account
Registration	email	required, lower-cased, unique	400 "already exists"
Registration	password	minimum length 8, not a common password, not entirely numeric	400 from Django validators
Registration	role	not offered; any posted role is ignored	stored role STUDENT
Login	email, password	active user and matching hash	401 and a failed LOGIN audit
Question	subject, topic	topic.subject_id must equal subject	400
Question	type MCQ	exactly four options and one is_correct	400
Question	type DESCRIPTIVE	options must be empty	400
Question	marks	greater than 0	400, and the database CHECK agrees
Question edit	stem, marks, options	rejected when an attempt already links the question	400 and the old stem remains
Exam	start_at, end_at	end after start	400
Exam	duration_minutes	1 to 300, and not longer than the window	400 and no row
Exam	pass_marks, negative_mark	not negative; at publish, pass marks not above the paper total	400
Exam	title	stripped, at least 3 characters	400
Draw	counts	each count ≥ 0 and at least one question requested	400 if a pool is smaller than its count
Exam room	selected option	must be in attempt_option_orders for that question	400, selection not stored
Exam room	descriptive text	stored only on a DESCRIPTIVE question	400 if an option is sent for it
Evaluation	awarded_marks	0 through question.marks, and only before publish	400
Admin user	role	FACULTY or ADMIN on create	400 for any other role
Admin user	active on self	cannot be set false	400

Empty client-side checks (the password box has minLength 8, and required attributes on inputs) only avoid a round trip. The pytest suite calls the API directly, so a bypassed browser check still fails.

4.14 Entity-by-entity design notes

Section 4.5 is the field dictionary. These notes explain why each relation exists and which dependency or lifecycle rule it owns.

4.14.3 users — One authentication identity

id is the surrogate primary key; email is the candidate key. password_hash stores an encoded hash, role is STUDENT/FACULTY/ADMIN, active drives is_active, and created_at records creation. Questions, exams, attempts, results and audit rows refer to a user. Protected deletion preserves academic history.

Principal trace. See the users row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.9 subjects — Reusable academic subject

name is unique. Topics, questions and exams refer to a subject. A protected delete prevents a subject disappearing while a paper or bank item still needs its meaning.

Principal trace. See the subjects row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.15 topics — Subject-scoped classification

The unique pair (subject_id, name) permits the same word under two subjects but not twice under one. Question validation checks that its topic and subject agree.

Principal trace. See the topics row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.21 questions — Reusable bank item

type selects MCQ or DESCRIPTIVE; difficulty is EASY/MEDIUM/HARD; marks is positive; created_by is protected; active controls new selection. Text, type, marks and options become effectively frozen when an attempt links the item.

Principal trace. See the questions row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.27 question_options — Normalized MCQ alternatives

Each row has text and is_correct. Four rows and one key are enforced in the serializer and again at paper freeze. Descriptive questions have none. This replaces the synopsis fields option_a through option_d.

Principal trace. See the question_options row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.33 exams — Definition, window and marking scheme

The row holds subject, title, instructions, start/end, duration, pass and negative marks, lifecycle status and creator. Checks enforce ordered dates, positive duration, non-negative marks and a valid status.

Principal trace. See the exams row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.39 exam_questions — Master-paper junction

The pair (exam_id, question_id) is unique. An exam can select many reusable questions and a question can be selected by many exams. The exam side cascades while the question side is protected.

Principal trace. See the exam_questions row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.45 exam_attempts — One student's sitting

The unique pair (exam_id, user_id) is the main duplicate-start defence. started_at, submitted_at and status IN_PROGRESS/SUBMITTED/EVALUATED represent the sitting rather than the exam definition.

Principal trace. See the exam_attempts row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.51 attempt_questions — Frozen question order

Each question appears once per attempt and each display order appears once. Rows are created transactionally at start and cascade with the attempt. The question foreign key is protected.

Principal trace. See the attempt_questions row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.57 attempt_option_orders — Frozen MCQ option order

Each option and display position is unique within an attempt question. This table is the design addition required to make random option order stable through refresh.

Principal trace. See the attempt_option_orders row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.63 attempt_answers — Current response and award

The unique (attempt_id, question_id) row stores either a selected option or descriptive text plus marked_for_review. The review flag defaults false, persists independently of the answer and never changes scoring. Award, evaluator and evaluation time are null until scoring. The selected option is protected so a historic answer does not lose its label.

Principal trace. See the attempt_answers row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.69 results — Recorded evaluation outcome

attempt_id is one-to-one and (user_id, exam_id) is unique. Objective, descriptive, total and maximum marks are non-negative. PASS/FAIL and published separate calculation from student visibility.

Principal trace. See the results row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.14.75 audit_logs — Operational event record

The actor may be null for a failed login. Action, entity, optional entity id, time and detail support the admin extract. The user is SET NULL on deletion so the event is retained.

Principal trace. See the audit_logs row in Section 4.5, the ER diagram (Figure 4.6), migration 0001_initial.py in the owning app, and Appendix A.

4.15 Normalization derivation

The implemented schema was not obtained by copying the six-table synopsis layout. It was derived from the facts that must remain independent.

Unnormalized examination record. A single paper-like row would contain student name and email, exam title and window, subject name, question text, four option texts, selected option, marks, result status and report average. It would repeat student and exam data once per question; updating an email would require many rows; deleting the last answer could delete the only copy of a question; an average would become stale when another result was published.

First normal form. Every column contains one value. Options are rows in question_options, not option_a through option_d. A paper selection is a row in exam_questions; a frozen paper is a row per question in attempt_questions; an answer is a row per attempted question. The option permutation is also a row per option. These choices remove repeating groups while preserving the order explicitly.

Second normal form. Junction facts depend on the whole relationship key. In exam_questions, the fact is that one particular exam selects one particular question; neither id alone identifies it. In attempt_questions, display order belongs to the pair represented by the row, not globally to the question. In attempt_answers, the response belongs to the student sitting and question, enforced by unique (attempt_id, question_id). Topic name depends on topic id, not on part of a question relationship.

Third normal form. Subject name appears in subjects, not in every question and exam. Topic name appears in topics. User name/email/role appear in users, not in attempts/results. Exam title and pass mark remain in exams; a result refers to the exam. A question's difficulty and marks remain in questions; an answer stores only the response and award. Faculty/admin report averages, participation and pass percentage are calculated queries, so there is no transitive reports row whose value depends on all current results.

Intentional recorded outcome. results.total_marks is not treated as a report cache. It is the recorded outcome of completed evaluation, including a publish flag and generation time. The component objective and descriptive amounts explain the total. Once published, the marking service refuses a later change. The faculty aggregate remains a query over those recorded outcomes.

Candidate and alternate keys. users.email is a candidate key; subjects.name is a candidate key; (subject_id, topics.name) is a composite candidate key; (exam_id, question_id) selects one master-paper row; (exam_id, user_id) selects one attempt; (attempt_id, question_id) selects one answer; (user_id, exam_id) selects one result. Surrogate bigint ids make foreign keys uniform without removing these uniqueness rules.

Functional dependencies.

Table 4.22 — Functional dependencies used in normalization

Relation	Principal dependency
users	id → name, email, password_hash, role, active, created_at; email → id and the same attributes
topics	id → subject_id, name; subject_id+name → id
questions	id → subject_id, topic_id, type, text, difficulty, marks, explanation, created_by, active
exams	id → subject_id, title, instructions, window, duration, marking scheme, status, created_by
exam_attempts	id → exam_id, user_id, times, status; exam_id+user_id → id
attempt_answers	id → attempt_id, question_id, response and evaluation fields; attempt_id+question_id → id
results	id/attempt_id/user_id+exam_id → component marks, total, maximum, status, published, generated_at

The third-normal-form argument is tied to the live tables. It does not claim that framework tables such as django_migrations are part of the examination data model.

4.16 Referential actions and index rationale

PROTECT. A subject used by a topic/question/exam, a question used by a paper/attempt/answer, an exam used by an attempt/result, and a user whose authored or attempted record has academic meaning are protected. The UI can deactivate a user or question instead of erasing history.

CASCADE. Rows that have no meaning without their parent sitting or definition cascade: question options with a deleted unused question; exam-question selections with a deleted draft exam; frozen questions/options/answers with an attempt. Deleting a DRAFT remains application-controlled; a published exam with attempts is not offered as ordinary cleanup.

SET NULL. An audit row may keep the action if its actor row is eventually removed. Its user_id is nullable for the same reason failed anonymous login has no authenticated user.

Indexes. The bank index (subject, type, difficulty) supports filters and draw pools. questions.active supports eligible selection. Exam status/start/end supports student catalogue windows. Attempt user supports history. Answer attempt supports paper loading. Audit user/time and action support the Admin filters. Unique indexes also serve their equality lookups.

An index is not claimed merely because a field appears in a screenshot; the list comes from model Meta/migration declarations and the keys created by MySQL for foreign and unique constraints.

4.17 Algorithm specifications in pseudocode

The flowcharts in Figures 4.12 and 4.13 are supplemented here with input, output, invariants and failure behavior.

Build a frozen attempt paper

Input: a newly created attempt whose exam is published; an optional random generator.
Output: ordered attempt-question rows and, for every MCQ, ordered option rows.
Invariant: the set of question ids equals the exam's master paper; each position is unique.

```
links := all exam_questions for attempt.exam, with question
if links is empty: raise "Exam has no questions"
shuffle links with supplied RNG or SystemRandom
for each link at position qpos:
   slot := insert AttemptQuestion(attempt, question, qpos)
   if question.type is DESCRIPTIVE: continue
   options := all question_options for question
   if count(options) != 4 or count(is_correct) != 1:
      raise "MCQ paper items must have exactly four options and one answer"
   shuffle options
   insert one AttemptOptionOrder(slot, option, position) for each option
```

This runs inside the start transaction. A malformed MCQ rolls back the attempt as well as partial slots.

Start one attempt

```
require user.role == STUDENT
require exam.status == PUBLISHED
require exam.start_at <= now < exam.end_at
begin transaction
   insert ExamAttempt(exam, user, started_at=now, IN_PROGRESS)
   freeze_paper(attempt)
commit
audit START(attempt, exam)
return attempt_payload(reveal=false)
```

The database unique key translates a racing or repeated insert to conflict 409. The response is assembled from frozen rows, not from the master paper.

Save current answers

```
attempt := ensure_open(attempt)       # may auto-submit expired work
require attempt.status == IN_PROGRESS
require attempt.user_id == actor.id
paper := map question_id -> frozen slot
for each submitted item in one transaction:
   require question_id in paper
   answer := get-or-create(attempt, question)
   if MCQ:
      allow blank, otherwise require option in slot's frozen option order
      set selected_option; clear descriptive_text
   else:
      reject selected_option
      set stripped descriptive_text or null; clear selected_option
```

```
    save answer
audit SAVE(count)
```

The client sends the whole visible answer set. The unique answer key turns repeated save into update rather than duplicate rows.

Submit and evaluate objective items

```
lock attempt
require status == IN_PROGRESS
set submitted_at and status SUBMITTED
for each frozen MCQ:
    blank -> 0
    selected option is_correct -> question.marks
    otherwise -> 0 - exam.negative_mark
objective := max(sum(MCQ awarded_marks), 0)
if no descriptive item has null awarded_marks:
    finalize_result
audit SUBMIT(auto/manual)
```

finalize_result adds descriptive awards, sums the frozen paper maximum, compares total with pass marks, creates or updates one result, and sets the attempt EVALUATED. It does not publish.

Award descriptive marks and publish

```
require attempt not IN_PROGRESS
require result not already published
for each mark item:
    require question is DESCRIPTIVE and on frozen paper
    require 0 <= award <= question.marks
    set award, evaluated_by, evaluated_at
audit EVALUATE
if no descriptive is pending: finalize_result

publish:
    require result exists
    require attempt EVALUATED and no descriptive pending
    set result.published = true
    audit PUBLISH
```

Calculation and publication are distinct because Faculty may need to review a complete total before disclosure.

Derive dashboard state

```
if an attempt exists:
    ensure_open(attempt)
    state := attempt.status
else if exam is not PUBLISHED:
    omit it
else if now < start_at:
    state := UPCOMING
else if now >= end_at:
    state := MISSED
else:
    state := AVAILABLE
attach attempt_id, result_published, paper total and question count
```

No scheduling queue is needed for this calculation. The server clock and indexed exam window are the data structure.

4.18 Module interfaces: inputs, processing, outputs and stores

4.18.1 Authentication and user administration

Inputs. Public registration receives name, email and password. Login receives email and password. Refresh receives the refresh token. Admin user creation receives name, email, password and a role limited to FACULTY or ADMIN; update receives name, role or active.

Processing. Registration normalizes email, applies Django password validators, and explicitly passes STUDENT to the manager. Login delegates hash verification to Django and token creation to SimpleJWT. Failed and successful logins call the audit service. Admin writes run after IsAdmin; a caller cannot deactivate their own row.

Outputs. Registration and Admin creation return user identity without password. Login returns access, refresh and the same safe user representation. /me/ returns the token's user. Admin list can filter role.

Stores. users owns identity. audit_logs receives login events. The module does not create a separate Student or Faculty table because role is the discriminator.

Boundary. Accounts are internal to this application. There is no IGNOU directory, email verification, forgotten-password mailer or social login in the build.

4.18.2 Subject, topic and question bank

Inputs. Subject name; topic subject/name; question subject/topic/type/text/difficulty/marks/explanation/options; activate/deactivate action; filter query strings.

Processing. Subject names are case-insensitive unique at serializer level. Topic uniqueness is subject+name. Question validation confirms topic ownership, positive marks, and the type-specific option invariant. Update checks whether an attempt already links the question. Protected deletion turns referenced subject/question deletion into a conflict.

Outputs. JSON includes subject/topic names and creator name in addition to ids. The bank list can filter subject, topic, type, difficulty and active. Option correctness is visible only to Faculty/Admin bank users; Student has no bank route.

Stores. subjects, topics, questions, question_options. Exam selection and attempt freeze are other modules and do not change bank ownership.

Boundary. Four MCQ options are the implemented academic rule. There is no image/file attachment, formula editor, import spreadsheet, revision history or coding judge.

4.18.3 Exam definition and scheduling

Inputs. Subject, title, instructions, start/end, duration, pass/negative marks; one question id; draw counts; lifecycle action.

Processing. A create is forced to DRAFT. Serializer and database validate window and non-negative marking values. Fixed add and rule draw are DRAFT-only. Draw samples active same-subject items without replacement. Publish validates the whole paper and total. Close and unpublish enforce the lifecycle rules.

Outputs. Exam list/detail includes subject/creator names, total marks and question count. Detail includes selected question metadata and, for staff, options. Lifecycle actions return the updated exam.

Stores. exams and exam_questions; bank tables are read. Attempts are read only to protect delete/unpublish.

Boundary. An exam is visible to Student only through the derived student catalogue after publish. There is no class/section enrollment table, admission ticket or payment prerequisite.

4.18.4 Conducting one sitting

Inputs. Authenticated Student and exam id at start; attempt id at detail/save/submit; answer list during save with selected_option_id, descriptive_text and marked_for_review.

Processing. Start checks actor, status and window, creates one attempt, shuffles and freezes paper/options. Detail calls ensure_open and calculates remaining time. Save validates ownership, status, paper membership and option membership, then persists the answer and optional review flag. Submit locks the row, records time, scores objectives, and finalizes only if descriptive work is complete.

Outputs. The attempt payload includes exam metadata, status, deadline, server time, remaining seconds, ordered questions and marked_for_review for each item. During the sitting, option objects contain id/text only. Submit returns status/result-ready and publish state.

Stores. exam_attempts, attempt_questions, attempt_option_orders, attempt_answers; possibly results on completion. START, SAVE and SUBMIT audit rows.

Boundary. The module cannot know whether a browser tab is visible and does not collect camera/screen data. A periodic save limits lost typed work but cannot guarantee receipt during network failure.

4.18.5 Objective and descriptive evaluation

Inputs. Frozen answers at submit; attempt id and question/award pairs from Faculty/Admin; result id for publish.

Processing. Objective scoring treats blank/correct/wrong distinctly and floors only the aggregate. Evaluation queue separates pending descriptive work from calculated unpublished results. Manual marking validates item type, range and publish lock. Finalization calculates components, maximum and pass/fail. Publish flips visibility only after EVALUATED.

Outputs. Queue rows include student/exam and stage. Evaluation detail reveals answer/key to staff after submit. Marking returns result when complete. Publish returns id/status/total/published.

Stores. Awards/evaluator/time on attempt_answers, outcome on results, status on exam_attempts, EVALUATE/PUBLISH in audit.

Boundary. Descriptive text is not scored by AI. Grade bands are not implemented; the result is PASS or FAIL against the exam pass mark.

4.18.6 Student, Faculty and Admin reports

Inputs. Authenticated role, optional exam id. No report-edit form exists.

Processing. Student history maps own attempts to published outcomes. Faculty query separates attempts and published results, then aggregates. Topic analysis uses frozen paper/answers. Admin reuses per-exam summary and institution-published totals.

Outputs. Student history/detail, faculty participation/outcomes/analysis, admin exam rows and totals. Chapter 8 describes all columns and two observed snapshots.

Stores. Read-only queries over users, exams, attempts, answers and results. The absence of a reports table is deliberate.

Boundary. CSV/PDF export and scheduled email are not implemented. A browser print is not described as a signed institutional marksheet.

4.18.7 Security and audit

Inputs. Events called by other services; Admin filters action/user/entity; deployment environment settings.

Processing. Permission classes run before views. ORM parameterizes SQL. MySQL grants restrict runtime DDL. Audit insert tolerates null actor for failed login and orders newest first.

Outputs. Authentication/authorization response codes; up to 500 audit rows. /api/health/ is a public process probe.

Stores. audit_logs; Django token authentication is stateless and no server session is created for API clients.

Boundary. Audit is not an external SIEM, and CORS is not a substitute for authentication. Production hardening remains Section 7.4/7.7.

4.19 End-to-end data-flow narratives

Student registration to first dashboard. Registration writes one users row. Login reads it and returns tokens. React stores the tokens and asks /me/. Student catalogue reads published/closed exams, attempts for that user and results for that user, then labels each exam. No marks from another user or unpublished row enter the response.

Faculty creates and publishes a paper. Faculty selects existing subject/topic bank rows or creates them. Question options are separate rows. Exam create writes DRAFT. Fixed add/draw writes junction rows. Publish reads the complete paper, bank flags and marks before updating one status and writing audit. Student catalogue can then include it.

Student starts and saves. Start reads master junctions, creates an attempt and frozen rows. React receives their order. Selecting a radio triggers PUT; the service verifies that option is in the frozen order, then writes one answer. Repeating PUT updates that unique row. Deadline derives from persisted start and exam end.

Submission through publication. Submit updates attempt and MCQ awards. If a descriptive award is null, Faculty queue sees SUBMITTED and Student sees no result. Faculty writes the award, which creates the result EVALUATED/unpublished. Faculty reviews and publishes. Student detail now returns component marks and answer correctness.

Reports and audit. The same attempt/answer/result rows feed report aggregates. Audit rows are independent append-style events and do not calculate marks. Thus the result screen, faculty aggregates and audit timeline can be reconciled without storing a second report copy.

4.20 Diagram interpretation and consistency checks

Context DFD

The context process is the whole Online Examination System. Student supplies registration/login/answers/submission and receives exam paper/timer/published result. Faculty supplies bank/exam/marks/publish requests and receives queues/analytics. Admin supplies account/report/audit requests and receives summaries and events. MySQL is internal and therefore is not an external entity on the context boundary. No camera, Moodle, payment or email entity appears because no such interface exists.

Level 1 DFD balance

The Level 1 processes preserve the context flows. Authentication accepts all actor credentials and writes/reads users/audit. Question bank accepts Faculty/Admin maintenance and writes catalogue stores. Exam Management consumes bank selections and writes exam stores. Conduction accepts Student start/save/submit and writes frozen/answer stores. Evaluation consumes submitted answers and Faculty marks and writes result. Results/Reports returns Student/Faculty/Admin outputs. Audit returns Admin-only event data.

The Level 1 does not create a data flow from Student directly to Results storage or from Faculty directly to answers; flows cross a process where permission and validation apply. Reports read transactional stores and do not write a Reports store.

Level 2 conduction balance

The parent conduction input—start/save/submit from Student—decomposes into eligibility, attempt creation, paper freeze, timer, answer validation and submit. Output paper/timer/status balances the parent output. exam_attempts, attempt_questions, attempt_option_orders and attempt_answers are shown as separate stores because the database does. A scheduling queue would not balance to an implementation store and is omitted.

Level 2 evaluation balance

Evaluation reads the submitted attempt/answers, writes objective awards, exposes only descriptive work to Faculty, finalizes one result and accepts a separate publish action. Student does not enter this diagram. The output to Student is through the Results process after publication, preventing the DFD from implying an early result.

ER cardinality checks

User–Attempt is 1:N and Exam–Attempt is 1:N, with a uniqueness rule making the pair one. Exam–Question is M:N through exam_questions, correcting the synopsis's 1:N. Attempt–Question is M:N through frozen rows but unique per pair. Attempt–Result is 1:0..1. User–Result and Exam–Result are 1:N with user+exam unique. Question–Option is 1:N. AttemptQuestion–AttemptOptionOrder is 1:N. Report is not an entity.

Use-case consistency

Every use case has an API and permission. Student: register/login, catalogue/start/save/submit, history/published detail. Faculty: login, bank, exam, evaluation, publish, faculty report. Admin: login, users, same academic staff actions, admin summary/audit. "Monitor camera" and "export report" are absent rather than drawn as future-looking ovals.

Class diagram consistency

The class view uses models rather than separate Student/Admin subclasses because the code uses one User with role. Services such as start_attempt and publish_result own cross-model behavior. Serializer/view classes are omitted from the simplified figure but described in Chapter 5. Attributes shown correspond to fields; operations shown correspond to service functions.

Activity and sequence checks

The activity branches after objective scoring: a mixed paper waits for manual marking; an objective-only paper finalizes. Both wait for publication before Student result display. The sequence diagram distinguishes browser request, DRF permission/validation, service transaction and ORM/MySQL. It does not suggest the browser calculates trusted marks.

State-machine invariants

Exam transitions are DRAFT → PUBLISHED → CLOSED, with PUBLISHED → DRAFT only when no attempts. Attempt transitions are IN_PROGRESS → SUBMITTED → EVALUATED,

or IN_PROGRESS → EVALUATED when no descriptive is pending. Result visibility is false → true; there is no unpublish-result transition. A state change that is absent from this diagram is not supported merely because a database administrator could edit a row.

PERT/Gantt consistency

Both use A1–A15. Gantt overlap (UI/API/database and backend/frontend) comes from PERT branches. Integration waits for both implementations. Functional/security tests branch after integration and converge at debugging. Report/package follow the critical path. The synopsis eight-week summary and sample 32-day job-portal schedule are not mixed into these figures.

4.21 Design decisions and rejected alternatives

Table 4.23 — Design decisions and rejected alternatives

Decision	Selected	Rejected alternative	Reason
Backend	Django + DRF	Spring Boot/Tomcat from synopsis OR-list	one consistent Python stack, ORM/auth/admin/validation
Database	MySQL 8	PostgreSQL alternative	synopsis/guide list and available environment
Question ownership	reusable bank + junction	question FK directly to one exam	required tagging/draw/reuse
Option storage	rows	option_a–option_d	first normal form and stable option id
Attempt	explicit entity	direct User–Exam relation	timer/status/one sitting/frozen paper
Randomization	frozen server order	client shuffle on every load	refresh stability and confidentiality
Timer	server deadline	client countdown authority	cannot extend by editing browser state
Result	evaluated row + publish flag	immediate display	descriptive workflow and Faculty release
Reports	queries	stored Reports averages	avoid stale aggregate rows
Authentication	JWT	Django browser session for SPA API	stateless JSON client; documented storage limitation
Authorization	permission classes + owner filters	role sent by UI	server trust boundary
Scheduling	datetime comparison	queue	no queued operation is needed for eligibility
Coding question	not executed	local sandbox	high security scope; synopsis says objective primary
Proctoring	future scope	webcam/screen/face	absent code and privacy/operational burden

Chapter 5 Implementation and Coding

MCSP-232 asks for SQL, comments, and coding standards. It also says the 100–125 page documentation excludes coding. This chapter therefore contains environment, grants, seed description, and a small set of annotated excerpts. The complete source is the project package / CD, not a second dump of the repository (the Job Portal sample spent over a hundred pages on that dump).

5.1 Development environment and repository

backend/config — settings, URLs, WSGI

backend/accounts, catalog, examination, auditlog

frontend/src — React routes and screens

database/schema.sql, database/grants.sql

tests/ — pytest suite

Compose / Colima stack serving UI and API on http://127.0.0.1:8080

5.2 Database implementation, seed and grants

DDL is generated from migrations and saved as database/schema.sql. Seed command seed_demo creates subjects Operating Systems and Database Management Systems, the published Operating Systems Midterm (duration 30 minutes, pass marks 4.00, negative mark 0.50), a Database Quiz draft, and demonstration attempts. Seed passwords are demonstration secrets only.

```
GRANT ALL PRIVILEGES ON oes.* TO 'oes_migrator'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON oes.* TO 'oes_app'@'localhost';
-- DDL is reserved for oes_migrator. oes_app is the runtime login.
```

5.3 Annotated excerpts

Registration always stores STUDENT (accounts/serializers.py). Input: name, email, password. Output: User row. Calls: Django create_user.

```
def create(self, validated_data):
    return User.objects.create_user(
        email=validated_data["email"],
        password=validated_data["password"],
        name=validated_data["name"],
        role=User.ROLE_STUDENT,
        active=True,
    )
```

Freeze paper — input: attempt with a master paper; output: AttemptQuestion and AttemptOptionOrder rows; calls: SystemRandom.shuffle.

```
def freeze_paper(attempt, rng=None):
    rng = rng or random.SystemRandom()
    links = list(ExamQuestion.objects.filter(exam=attempt.exam))
    rng.shuffle(links)
    for index, link in enumerate(links, start=1):
        attempt_question = AttemptQuestion.objects.create(
            attempt=attempt, question=link.question, display_order=index
        )
        if link.question.type != Question.TYPE_MCQ:
            continue
        options = list(link.question.options.all())
        rng.shuffle(options)
        AttemptOptionOrder.objects.bulk_create([...])
```

Deadline and objective floor — input: attempt; output: datetime or Decimal ≥ 0.

```
def deadline(attempt):
    duration_end = attempt.started_at + timedelta(minutes=attempt.exam.duration_minutes)
    return min(duration_end, attempt.exam.end_at)

def objective_total(attempt):
    total = AttemptAnswer.objects.filter(
        attempt=attempt, question__type=Question.TYPE_MCQ
    ).aggregate(total=Sum("awarded_marks"))["total"] or Decimal("0.00")
    return max(total, Decimal("0.00"))
```

Current save and bank-filter excerpts — review is independent of scoring; search is case-insensitive and combines with structured filters.

```
if "marked_for_review" in item and item.get("marked_for_review") is not None:
    answer.marked_for_review = bool(item.get("marked_for_review"))
answer.save()

search = (request.query_params.get("q") or request.query_params.get("search") or "").strip()
if search:
    questions = questions.filter(text__icontains=search)
```

5.4 Standardization: efficiency, errors, parameters, validation

Efficiency. select_related/prefetch_related; report aggregates in MySQL; freeze once; React JSON rather than full reloads; indexes on window, attempts, answers, audit.

Error handling. ExamRuleError carries an HTTP status. 400 invalid rule, 403 role, 404 missing, 409 duplicate attempt/submit. Transactions wrap draw, start/freeze, and evaluation.

Parameter passing. DRF serializer fields and ORM lookups. No string-concatenated SQL.

Validation. Serializers for names, topic/subject, option rules, windows, marks. Permission classes run before view logic. Frontend displays server detail rather than asserting success locally.

Naming. Python snake_case modules; React function components; domain constants for statuses.

5.5 API catalogue

Table 5.1 — Implemented API catalogue

Area	Methods and paths	Permission
Health	GET /api/health/	none
Auth	POST register/login/refresh; GET /me/	AllowAny / Any role
Users	GET/POST /api/users/; GET/PATCH /api/users/{id}/	Admin
Catalog	subjects, topics, questions CRUD; activate/deactivate	Faculty or Admin (reads: questions faculty+)
Exams	CRUD; questions/draw/publish/close/unpublish/start	Faculty/Admin; start = Student
Attempts	GET list/detail; PUT answers; POST submit	owner Student
Evaluation	queue, detail, marks, result publish	Faculty or Admin
Reports	student history/exam; faculty exam/analysis; admin summary	role-gated
Audit	GET /api/audit-logs/	Admin

5.6 How parameters move through one attempt

A start request carries only the exam id in the URL. The user id is not trusted from the body; it is taken from the JWT. start_attempt(exam, user) receives the Exam row and the User row. It passes the new ExamAttempt to freeze_paper. That function reads exam_questions for that exam id and writes child rows whose foreign keys point at the attempt and the question. The response serializer then reads those child rows. The browser never receives is_correct on this response.

A save request carries a list of {question_id, selected_option_id, descriptive_text}. The view passes that list to save_answers(attempt, items, actor). For an MCQ the selected option id is

accepted only when an attempt_option_orders row exists for that attempt-question and that option. A foreign option id is rejected before any award is stored. The actor id is written into the audit detail path, not into the answer as a mark.

A mark request carries {question_id, awarded_marks} from Faculty or Admin. award_descriptive receives the attempt, the list, and the marker. It rejects a student caller earlier, at the permission class. It rejects an award below 0 or above question.marks, and it rejects a change after the result has been published. The marker's user id is stored on evaluated_by.

These are the parameter rules an evaluator can trace. They are not a general essay on "parameter passing".

5.7 Error behaviour actually returned

Table 5.4 — Error map used by the API

Condition	HTTP	Body	State change
Missing or bad token	401	DRF authentication detail	none
Wrong role	403	permission detail	none
Unknown exam, attempt or result	404	not-found detail	none
Validation (options, window, marks, topic)	400	field errors	none, or the transaction rolls back
Second start, second submit, save after expiry once submitted	409	conflict detail	the first successful write remains
Expired in-progress save	409 after commit	time-is-up detail	submit has already been saved
Result not published	403	not-published detail	none

The defect recorded in Section 6.5 mattered because a 409 raised inside the same database transaction would have undone the auto-submit. The expiry path now commits the submission and only then returns the error. That is why a student who saves at time zero still has a SUBMITTED or EVALUATED row.

5.8 Constraint excerpts (not the full DDL)

The migration files are the full DDL. The constraints an examiner is most likely to ask about are:

```
-- one sitting
UNIQUE KEY one_attempt_per_student (exam_id, user_id);
-- one published outcome
UNIQUE KEY one_result_per_student_exam (user_id, exam_id);
-- reusable bank, not option_a..option_d
-- question_options.question_id -> questions.id
CONSTRAINT exam_window_order CHECK (end_at > start_at);
CONSTRAINT exam_duration_positive CHECK (duration_minutes > 0);
CONSTRAINT result_total_non_negative CHECK (total_marks >= 0);

GRANT SELECT, INSERT, UPDATE, DELETE ON oes.* TO 'oes_app'@'localhost';
```

oes_migrator is the account that applies migrations. The application process is documented to use oes_app. The privilege tests connect as oes_app and attempt CREATE TABLE; MySQL returns error 1142. That is the evidence for least privilege. It is not a claim that a production host has already been hardened beyond this grant split.

Seed data, loaded by seed_demo, creates the demonstration accounts admin@oes.local, faculty@oes.local and student@oes.local (plus further student rows when the command is repeated) and one open Operating Systems paper plus a Database Quiz draft. Passwords set by that command are demonstration secrets, printed by the command so the system can be shown. They are not production credentials.

5.9 Where each requirement lives

Table 5.5 — Requirement groups and the files that implement them

Requirement group	Python	React	Table
FR-01 to FR-07 accounts	backend/accounts/ serializers.py, views.py, permissions.py	Login.jsx, Register.jsx, auth.jsx	users
FR-08 to FR-13 bank	backend/catalog/ serializers.py, views.py	Faculty.jsx question screens	subjects, topics, questions, question_options
FR-14 to FR-19 exams	backend/examination/ services.py draw_questions, publish_exam, close_exam	exam list and editor in Faculty.jsx	exams, exam_questions
FR-20 to FR-28 conduction	start_attempt, freeze_paper, save_answers, submit_attempt, ensure_open	Student.jsx exam room	exam_attempts, attempt_questions, attempt_option_orders, attempt_answers
FR-29 to FR-34 evaluation	score_mcq, objective_total, award_descriptive, finalize_result, publish_result	evaluation screen in Faculty.jsx	attempt_answers, results
FR-35 to FR-37 reports	backend/examination/ reports.py	student history, faculty report, admin summary	none (queries)
FR-38 audit	backend/auditlog/services.py	admin audit screen	audit_logs

A change to scoring therefore starts in score_mcq and objective_total, not in the React timer. A change to who may publish starts in IsFacultyOrAdmin on ResultPublishView. The table is the map used to keep the traceability appendix attached to files rather than to chapter titles only.

5.10 Endpoint contracts by resource

Table 5.4 is the compact catalogue. The following contracts state the permission and the business outcome of each route family. JSON field-level errors remain DRF responses rather than a second custom error format.

5.10.3 Authentication

Routes. POST /api/auth/register/, /login/, /refresh/; GET /me/.

Permission. AllowAny on registration/login/refresh; active authenticated role on /me/.

Outcome. Creates Student, issues or refreshes JWT, returns the current user. Login success/failure is audited.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.13 Users

Routes. GET/POST /api/users/; GET/PATCH /api/users/{id}/.

Permission. Admin.

Outcome. Lists or provisions users and updates name/role/active. Self-deactivation is refused.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.23 Subjects

Routes. GET/POST /api/subjects/; detail GET/PATCH/DELETE.

Permission. Any role may read; Faculty/Admin mutate.

Outcome. Maintains the unique subject catalogue; protected deletion returns a conflict.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.33 Topics

Routes. GET/POST /api/topics/; PATCH/DELETE detail.

Permission. Any role may list; Faculty/Admin mutate.

Outcome. Maintains subject-scoped names and supports ?subject= filtering.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.43 Question bank

Routes. GET/POST /api/questions/; detail GET/PATCH/DELETE; activate/deactivate.

Permission. Faculty/Admin.

Outcome. Filters by subject/topic/type/difficulty/active and applies MCQ/descriptive invariants.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.53 Exam definition

Routes. GET/POST /api/exams/; detail GET/PATCH/DELETE.

Permission. Faculty/Admin.

Outcome. Creates and maintains DRAFT exams and returns totals/counts.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.63 Paper and lifecycle

Routes. question add/remove; draw/, publish/, close/, unpublish/.

Permission. Faculty/Admin.

Outcome. Builds a fixed/rule paper and moves only through permitted exam states.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.73 Student catalogue and start

Routes. GET /api/student/exams/; POST /api/exams/{id}/start/.

Permission. Student.

Outcome. Derives dashboard state and creates the unique frozen attempt.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.83 Attempt

Routes. GET /api/attempts/, detail; PUT answers/; POST submit/.

Permission. Owner Student for writes; staff read only after submission.

Outcome. Auto-saves validated responses, reports server time, and submits once.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.93 Evaluation

Routes. GET /api/evaluation/queue/, detail; POST marks/.

Permission. Faculty/Admin.

Outcome. Separates needs-marking and ready-to-publish work and validates descriptive marks.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.103 Results

Routes. GET /api/results/me/, own exam detail; POST /api/results/{id}/publish/.

Permission. Student reads published own rows; Faculty/Admin publish.

Outcome. Keeps evaluation calculation separate from disclosure.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.113 Reports

Routes. student history/exam; faculty exam/analysis; admin summary.

Permission. Role-specific.

Outcome. Runs aggregate queries without writing a report table.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.10.123 Audit and health

Routes. GET /api/audit-logs/; GET /api/health/.

Permission. Admin for audit; public health.

Outcome. Provides a filtered 500-row event extract and a process-liveness probe.

Common response rule. Success uses 200/201/204 as appropriate; validation is 400, authentication 401, authorization 403, missing object 404, and duplicate/state conflict 409.

5.11 Runtime configuration and deployment boundary

Environment variables. OES_SECRET_KEY, OES_DEBUG, OES_ALLOWED_HOSTS, OES_DB_NAME, OES_DB_USER, OES_DB_PASSWORD, OES_DB_HOST and OES_DB_PORT replace local defaults. The repository’s example environment is documentation, not a production secret. The report does not print any private deployed credential.

Database preparation. MySQL creates schema oes with utf8mb4. The administrator creates oes_migrator and oes_app, applies database/grants.sql, runs migrations with the migrator role,

then runs the web process with the app role. Strict transactional mode is requested in Django database options. A test database requires migrator privileges and must not be created by the runtime user.

Application preparation. Python creates a virtual environment and installs backend/requirements.txt. manage.py migrate checks and applies the versioned schema. manage.py seed_demo is optional and belongs only to demonstration. manage.py check validates Django configuration. Gunicorn loads config.wsgi:application; the React production build is static content that a reverse proxy may serve.

Client/API path. Vite proxies /api to Django during development. Production needs one origin or an explicitly allowed origin, HTTPS termination, secure headers and a route from /api to Gunicorn. The development CORS list contains only localhost Vite origins. That is not a wildcard and not a production topology.

Operational requirements still outside the repository. The institution must choose the domain, TLS certificate, secret store, backup retention, log retention, monitoring and alert destination, recovery objective, rate limits and maintenance window. Database backups must be restored in a test environment before they can be called verified. The current evidence does not include such a restore.

Demonstration start sequence.

Start MySQL 8 and verify both accounts.

Run migrations as oes_migrator.

Load demo data as oes_app only if a demonstration is needed.

Start Django or Gunicorn with oes_app.

Build/start React and open the browser.

Check /api/health/, then authenticate rather than inferring readiness from the landing page.

Run pytest using the isolated test database; do not run destructive tests against the demo schema.

Production handover. Replace development secrets, turn DEBUG off, set real hosts/CORS/CSRF origins, terminate HTTPS, move browser token handling to a reviewed policy, schedule backup and audit retention, and rerun the security and system checks. Proctoring is not activated by deployment because there is no proctoring code.

5.12 Representative JSON contracts

The examples show field shape. Ids and timestamps are illustrative placeholders, not claimed execution results. Actual observed totals remain in Chapter 8.

Registration

POST /api/auth/register/

{"name":"Student Name","email":"student@example.edu","password":"StrongPass@123"}

Success returns id, name, normalized email, role STUDENT, active and created_at. It does not return password or hash. Sending "role":"ADMIN" does not elevate the account.

Login and authenticated request

POST /api/auth/login/

{"email":"student@example.edu","password":"StrongPass@123"}

{"refresh":"<jwt>","access":"<jwt>","user":{"id":10,"role":"STUDENT", "...":"..."}}

Later requests use Authorization: Bearer <access>. /me/ has no user id parameter because the token identifies the row.

MCQ create

POST /api/questions/


```
{
  "subject": 1,
  "topic": 2,
  "type": "MCQ",
  "text": "Which component selects the next process?",
  "difficulty": "EASY",
  "marks": "2.00",
  "options": [
    {"option_text":"Compiler","is_correct":false},
    {"option_text":"Scheduler","is_correct":true},
    {"option_text":"Linker","is_correct":false},
    {"option_text":"Loader","is_correct":false}
  ]
}
```


The server sets created_by. Three/five options, no key or several keys returns a 400 options error.

Exam create and fixed add

POST /api/exams/


```
{
  "subject":1,
  "title":"Operating Systems Midterm",
  "instructions":"Answer all questions.",
  "start_at":"2026-10-05T09:00:00+05:30",
  "end_at":"2026-10-05T12:00:00+05:30",
  "duration_minutes":30,
  "pass_marks":"4.00",
  "negative_mark":"0.50"
}
```


The response is DRAFT. POST /api/exams/{id}/questions/ {"question_id":5} adds one active same-subject question. Publish is a separate empty-body POST.

Rule draw

POST /api/exams/{id}/draw/

{"easy":2,"medium":1,"hard":1,"descriptive":1,"replace":true}

The four counts select from the active subject bank. replace=true replaces the draft paper only if all requested pools are large enough.

Start payload

POST /api/exams/{id}/start/


```
{}

{
  "id":42,
  "status":"IN_PROGRESS",
  "started_at":"...",
  "deadline":"...",
  "remaining_seconds":1800,
  "server_time":"...",
  "exam":{"id":1,"title":"...","negative_mark":"0.50","subject":"Operating Systems"},
  "questions":[
    {"question_id":5,"display_order":1,"type":"MCQ","text":"...","marks":"2.00",
     "options":[{"id":21,"option_text":"Loader"}],
     "selected_option_id":null,"descriptive_text":null}
```

```
 ]
}
```

The abbreviated options array illustrates shape. The real MCQ has four. There is no is_correct key while the result is hidden.

Auto-save

PUT /api/attempts/42/answers/

```
{"answers":[
 {"question_id":5,"selected_option_id":21,"descriptive_text":null},
 {"question_id":9,"selected_option_id":null,"descriptive_text":"A process owns resources..."}
]}
```

The response is the refreshed attempt and remaining seconds. Sending option 21 for a different frozen question returns 400.

Manual marks

POST /api/evaluation/attempts/42/marks/

```
{"marks":[{"question_id":9,"awarded_marks":"2.00"}]}
```

The response has attempt id/status and result details when the last descriptive answer completes evaluation. A Student receives 403; a mark above question marks receives 400.

Result publication and student detail

POST /api/results/{result_id}/publish/ changes only published. The own-result GET before this returns 403. After it, the response includes the attempt payload with option correctness and a result object containing objective, descriptive, total, maximum, status, pass mark and generated time.

Report and audit shape

Faculty report returns counts and decimal strings; analysis returns an array of topic/difficulty/type buckets. Admin summary returns per-exam rows and institution totals. Audit returns rows with user/user email/name, action, entity, entity id, timestamp and detail. None accepts a client-supplied total or result status.

5.13 Repository and configuration map

Table 5.6 — Repository ownership and change impact

Path	Responsibility	Change impact
backend/config/settings.py	apps, middleware, MySQL, hashers, JWT, CORS	security/deployment review and full regression
backend/config/urls.py	route composition and health	API catalogue and routing tests
backend/accounts/models.py	custom user/roles	migrations, auth and every FK
backend/accounts/serializers.py	register/login/user writes	auth/security tests
backend/accounts/permissions.py	role gates and active check	authorization tests across modules
backend/catalog/models.py	subjects/topics/questions/options	migration, bank/exam tests
backend/catalog/serializers.py	MCQ/descriptive/freeze edit rules	question validation tests
backend/examination/models.py	exam, frozen attempt, answer, result	migration and all lifecycle tests
backend/examination/services.py	draw, publish, timer, scoring, marking	unit/integration/system tests
backend/examination/reports.py	student/faculty/admin queries	report tests and Chapter 8 reconciliation
backend/auditlog/*	event rows and Admin API	audit system test
frontend/src/api.js	token header/refresh/error helper	browser flow; no component test
frontend/src/auth.jsx	session context	login/logout browser behavior
frontend/src/pages/Student.jsx	catalogue, exam room, result	browser attempt flow
frontend/src/pages/Faculty.jsx	bank, exam, evaluation, report	browser staff flow
frontend/src/pages/Admin.jsx	summary, users, audit	browser Admin flow
database/grants.sql	migrator/runtime privilege split	direct grant tests
tests/	pytest evidence	Chapter 6 result

Generated frontend/dist is build output, and virtual-environment files are dependencies. They are not quoted as authored source. Diagram DOT/SVG and report screenshots are submission evidence under media/oes-report.

Chapter 6 Testing

6.1 Testing techniques and strategies

Table 6.1 — Techniques applied to this project

Technique	How it is applied to OES	Tool
White-box / unit	Service functions with injected RNG (ReverseRng) so shuffle order is deterministic	pytest
Black-box / integration	DRF APIClient against URL routes, serializers, permissions and MySQL	pytest-django
Security testing	AuthN/AuthZ, password policy, JWT gate, MySQL privilege denial, audit access	pytest + mysqlclient
System testing	Multi-step lifecycles: submit → mark → publish → reports/audit	pytest
Browser / system	Three-role journey on Chrome against http://127.0.0.1:8080	Playwright / Chrome; **not** counted in the 33
Not claimed	Jest/React Testing Library, Postman collection, formal UAT 95% pass	absent

6.2 Test plan

Table 6.2 — Test plan

Level	Responsibility	Tools	Timeline (plan)	Exit criterion
Unit	Developer	pytest on services	Sprint 4–5	Deterministic freeze/score cases pass
Integration	Developer	pytest-django + MySQL	Sprint 4–7	Invalid partitions return 4xx; valid paths persist
Security	Developer	pytest + oes_app connection	Sprint 7	oes_app CREATE TABLE denied (1142)
System / browser	Developer	Chrome + running stack	Sprint 7	Three-role journey screenshots reviewed

Entry. Migrations applied, MySQL up, dependencies installed. Exit (observed). 35 passed automated cases; Vite production build succeeded; health endpoint 200; three-role browser screenshots captured; limitations disclosed. Regression rule. Any source change invalidates these statuses until the suite is rerun.

6.3 Executed unit and integration reports

Command: ./backend/.venv/bin/pytest -q --tb=short. Observed output on 5 October 2026: 35 passed in 6.57s (file media/oes-report/pytest-results.txt). Actual = that run. No unexecuted row is marked Pass. Sample-report people and jobs are not used.

Table 6.3 — Executed automated cases (35 collected, including four MCQ parameter rows)

Test ID	Case	Input / condition	Expected	Actual	Result
SEC-AUTH-01	self-register	role=ADMIN in body	stored STUDENT; argon2 hash	assertions matched 5 Oct 2026	PASS
SEC-AUTH-02	duplicate / weak password	existing email; 12345678	400 both	matched	PASS
SEC-AUTH-03	inactive login	active=False	401	matched	PASS
SEC-AUTH-04	admin vs student user write	POST /api/users/	201 faculty; 403 student	matched	PASS
SEC-AUTH-05	anonymous	GET /me/ and /questions/	401; health 200	matched	PASS
SYS-AUT	login and /me/	good/bad password, case email	401 then 200	matched	PA

Test ID	Case	Input / condition	Expected	Actual	Result
H-01					SS
INT-QB-01	valid MCQ	4 options, 1 key	201	matched	PASS
INT-QB-02	MCQ mutations ×4	3/5 options; 0/all keys	400	matched	PASS
INT-QB-03	descriptive + student write	options on descriptive; student GET bank	400 / 403	matched	PASS
INT-QB-04	topic/subject mismatch	wrong topic	400	matched	PASS
INT-QB-05	deactivate	POST deactivate	active=False	matched	PASS
SEC-EX-01	draft RBAC	faculty vs student create	201 / 403	matched	PASS
INT-EX-01	paper attach rules	wrong subject, duplicate, inactive	400 then 201	matched	PASS
INT-EX-02	publish rules	empty / pass>total / valid	400 / 400 / 200 PUBLISHED	matched	PASS
INT-EX-03	rule draw	counts vs short pool	200 then 400, DB unchanged	matched	PASS
INT-EX-04	duration vs window	duration too long	400, no row	matched	PASS
UNIT-COND-01	freeze with ReverseRng	injected RNG	reversed stable order	matched	PASS
UNIT-COND-02	freeze keeps full set	MCQ + descriptive	set equal; descriptive 0 option rows	matched	PASS
INT-COND-01	one attempt / window / role	second start; faculty start; early/late	409 / 403 / 400	matched	PASS
INT-COND-02	save / submit / double	valid then second submit	200 then EVALUATED unpublished; 409	matched	PASS
INT-COND-03	negative floor	all wrong, heavy penalty	objective 0.00 FAIL unpublished	matched	PASS
INT-COND-04	expiry autosubmit	backdated start	409 save; blank MCQ 0.00	matched	PASS
INT-CON	timer min()	shortened end_at	remaining ≤ 90	matched	PA

Test ID	Case	Input / condition	Expected	Actual	Result
D-05					SS
INT-COND-06	closed exam	faculty close then student start	400	matched	PASS
INT-EVAL-01	MCQ-only result	no descriptive	EVALUATED unpublished	matched	PASS
SYS-EVAL-01	mixed lifecycle	mark, remark, publish, student read	403 then revealed PASS; remark locked	matched	PASS
SYS-RPT-01	reports role + embargo	history/faculty/admin	marks hidden until publish; 403 cross-role	matched	PASS
SYS-AUD-01	audit coverage	login fail/success, start, save, submit, evaluate, publish	actions present; student GET 403	matched	PASS
SEC-DB-01/02	oes_app DDL	CREATE TABLE	OperationalError 1142	matched	PASS
INT-COND-07	mark for review	save true; reload; clear false	flag persists; answer/scoring unchanged	matched	PASS
INT-QB-06	question-bank filters	keyword/topic/difficulty/ type combinations	only matching questions returned	matched	PASS

6.4 System / browser verification

Automated command result: 35 passed in 6.57s.

Browser workflow (separate from pytest): register/login, bank search/filter, start, save, mark/unmark for review, submit, descriptive mark, publish, result, CSV/report and audit.

React production build completed before evidence capture.

There are **no** React component tests in the pytest suite.

Dedicated pytest coverage remains absent for token refresh, subject/topic CRUD, exam unpublish and several list/detail APIs. The final 35-case run adds persistence coverage for mark-for-review and combined question-bank filtering.

6.5 Debugging and code improvement

Expired save transaction. Raising inside the save transaction rolled back auto-submission. Submission now commits before the expiry response.

Frontend export mismatch. A production build found a missing named export. Routes were reconciled; the subsequent Vite build passed.

Screenshot timing. First captures showed headings before report data arrived. Capture now waits for data-bearing cards/tables.

Report target. The first faculty report opened a draft exam. The corrected capture selects Operating Systems Midterm explicitly.

6.6 What the browser run and the later API read actually showed

Pytest does not click the React screens. A separate browser run on 5 October 2026 did. It registered a student, started Operating Systems Midterm, saved answers, submitted, opened the faculty evaluation screen, published a result, and opened the faculty and admin reports. The screenshots in Chapters 4 and 8 are from that run. They are not a second pytest count.

Two figures in those screenshots need a sentence so they are not mistaken for a different formula:

The Operating Systems report capture shows **3 submitted of 4 students**, **1 pass and 1 fail**, average **3.25**, high **4.00**, low **2.50**.

The admin capture shows **2 published results**, **1 pass, 1 fail**, average **3.25**.

One published paper captured on the student result screen was **FAIL, 2.50 / 10.00**, with objective **0.50** and descriptive **2.00**, against pass mark **4.00**. The arithmetic (2.50 + 4.00) / 2 = 3.25 matches the two published totals on that capture.

faculty_exam_report counts pass, fail and average only over rows with published=True. Started and submitted counts include attempts that are not yet published. That is why “3 submitted” can sit beside “1 pass and 1 fail” without a missing row: the third submission had not been published. The same rule explains the admin line “published results 2”.

A later read of GET /api/reports/faculty/exams/1/ on the same date, after further demonstration use of the database, returned registered_students **3**, started **2**, submitted **2**, evaluated **1**, published **1**, pass_count **1**, fail_count **0**, average_total **7.50**. Table 8.1’s published PASS of **7.50 / 10.00** (objective 5.50, descriptive 2.00, pass mark 4.00) is that later published row for student@oes.local, not a silent rewrite of the earlier screenshot. Both observations are kept. Neither is a planned production statistic.

Topic rows on the later analysis read were, for two scored attempts: Memory EASY, HARD and Processes EASY each 2 responses and 2 correct, average award 2.00; Processes MEDIUM MCQ 2 responses, 1 correct, average award 0.75; Processes MEDIUM descriptive 1 response, average award 2.00, and the “correct” count 0 because that count is defined only for an MCQ award equal to the question marks. Those are query definitions, visible in examination/reports.py, not spreadsheet adjustments.

6.7 Checks that were not given a Pass row

The following exist in the code and were not given their own pytest function. They are listed so the 33 Pass results are not stretched to cover them.

Table 6.4 — Not marked Pass

Item	Present in	Automated result
POST /api/auth/refresh/	RefreshView	Not executed as its own case
Subject and topic create, rename and delete	catalog/views.py	Not executed as its own case
POST .../questions/{id}/activate/	catalog URLs	Deactivate is tested; activate is not a separate case
POST /api/exams/{id}/unpublish/	ExamUnpublishView	Not executed
GET /api/student/exams/ and GET /api/attempts/	examination URLs	Used by the browser run; not a pytest assertion

Item	Present in	Automated result
GET /api/evaluation/queue/	EvaluationQueueView	Used by the browser run; not a pytest assertion
React components	frontend/src	No component test; production build succeeded
HTTPS, token theft, expired-token refresh	deployment and SimpleJWT	Not executed
PBKDF2 verifying an old hash	PASSWORD_HASHERS	Hasher is configured; no legacy-hash assertion

6.8 Detailed executed-case report

The summary table preserves one row per collected test. The descriptions below make Input, Expected and Actual explicit without inventing a UI outcome. “Matched” means the assertion in the named pytest case completed; it does not extend to the untested routes in Section 6.7.

SEC-AUTH-01 — Student self-registration

Input / condition. POST registration including role ADMIN.

Expected. 201; normalized email; stored role STUDENT; Argon2 hash; no password in response.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SEC-AUTH-02 — Duplicate and weak password

Input / condition. Repeat the email; then use numeric common password.

Expected. Both 400 and no invalid account.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SYS-AUTH-01 — Login and profile

Input / condition. Bad password, case-insensitive good login, then /me/.

Expected. 401, then 200 token, then the same user profile.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SEC-AUTH-03 — Inactive account

Input / condition. Set active false and login.

Expected. 401.

Actual. The assertion matched.

Status. PASS in the recorded 35-case run.

SEC-AUTH-04 — Admin provisioning

Input / condition. Admin creates Faculty; Student attempts the same URL.

Expected. 201 for Admin; 403 for Student; Admin list contains row.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SEC-AUTH-05 — Anonymous protection

Input / condition. Call /me/ and question bank without token; call health.

Expected. 401, 401, 200.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-QB-01 — Valid MCQ

Input / condition. Four options, one key.

Expected. 201 and exactly one correct option.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-QB-02a — Three MCQ options

Input / condition. Remove one option.

Expected. 400.

Actual. The parameter case passed.

Status. PASS in the recorded 35-case run.

INT-QB-02b — Five MCQ options

Input / condition. Add one option.

Expected. 400.

Actual. The parameter case passed.

Status. PASS in the recorded 35-case run.

INT-QB-02c — No MCQ key

Input / condition. Set every is_correct false.

Expected. 400.

Actual. The parameter case passed.

Status. PASS in the recorded 35-case run.

INT-QB-02d — Several MCQ keys

Input / condition. Set every is_correct true.

Expected. 400.

Actual. The parameter case passed.

Status. PASS in the recorded 35-case run.

INT-QB-03 — Descriptive shape and role

Input / condition. Create descriptive with/without options; Student writes.

Expected. 201 empty options; 400 with options; 403 Student.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-QB-04 — Topic/subject consistency

Input / condition. Topic from another subject.

Expected. 400.

Actual. The assertion matched.

Status. PASS in the recorded 35-case run.

INT-QB-05 — Deactivate

Input / condition. POST deactivate.

Expected. 200 and active false.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SEC-EX-01 — Faculty creates draft

Input / condition. Faculty and Student POST the exam.

Expected. 201 DRAFT; 403 Student.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-EX-01 — Paper question rules

Input / condition. Foreign-subject, valid, duplicate and inactive question.

Expected. 400, 201, 400, 400.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-EX-02 — Publish guards

Input / condition. Empty paper; pass above total; valid publish; edit/delete after.

Expected. 400, 400, 200 PUBLISHED, then 400.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-EX-03 — Rule draw

Input / condition. Available counts, then request a short pool.

Expected. 200 with four; 400 and count unchanged.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-EX-04 — Duration/window

Input / condition. Duration longer than the schedule window.

Expected. 400 and no exam row.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

UNIT-COND-01 — Frozen order

Input / condition. Inject reverse RNG when starting.

Expected. Question and option order reversed and stable on reload.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

UNIT-COND-02 — Complete frozen set

Input / condition. Paper containing MCQ and descriptive.

Expected. All questions copied; 4 option orders only for each MCQ.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-COND-01 — Role, window and unique attempt

Input / condition. Faculty start; Student twice; future; ended.

Expected. 403; 201 then 409; 400; 400.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-COND-02 — Save/submit/embargo

Input / condition. Valid selection, foreign option, submit twice, early result read.

Expected. 200 persisted; 400; evaluated/unpublished; 409; 403.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-COND-03 — Negative floor

Input / condition. All wrong with a large negative mark.

Expected. Objective and total 0.00; FAIL; unpublished.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-COND-04 — Expiry and blank

Input / condition. Backdate start and save after deadline.

Expected. 409 response after auto-submit; blank award 0.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-COND-05 — Earlier deadline

Input / condition. Exam end less than duration end.

Expected. Remaining seconds no more than the shortened window.

Actual. The assertion matched.

Status. PASS in the recorded 35-case run.

INT-COND-06 — Closed exam

Input / condition. Close then Student start.

Expected. Close 200; start 400.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SYS-EVAL-01 — Mixed paper lifecycle

Input / condition. Submit MCQ+descriptive, Student marks attempt, Faculty marks/publishes, Student reads.

Expected. SUBMITTED; Student 403; EVALUATED result; published reveal; later mark rejected.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

INT-EVAL-01 — Objective-only lifecycle

Input / condition. Submit paper with no descriptive.

Expected. EVALUATED and result exists but published false.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SYS-RPT-01 — Report visibility and role

Input / condition. Read before/after publish as three roles.

Expected. History hides marks before publish; faculty/admin aggregates; wrong roles 403.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SYS-AUD-01 — Audit coverage

Input / condition. Perform failed/success login, start, save, submit, evaluate, publish.

Expected. LOGIN, START, SAVE, SUBMIT, EVALUATE, PUBLISH are present; Student list 403.

Actual. The assertions matched.

Status. PASS in the recorded 35-case run.

SEC-DB-01 — Runtime account DDL

Input / condition. CREATE TABLE while connected as oes_app.

Expected. MySQL OperationalError.

Actual. The assertion matched.

Status. PASS in the recorded 35-case run.

SEC-DB-02 — Runtime error number

Input / condition. CREATE TABLE privilege probe.

Expected. OperationalError 1142.

Actual. The assertion matched.

Status. PASS in the recorded 35-case run.

6.9 Test environment, fixtures and reproducibility

pytest.ini fixes DJANGO_SETTINGS_MODULE=config.settings, adds backend to the Python path and discovers under tests. pytest-django creates the isolated database. Direct grant tests connect with MySQLdb instead of the Django fixture so they can prove the actual runtime account lacks DDL.

The shared fixtures create one Admin, one Faculty and two Students with known test-only credentials, then create an Algorithms subject and Sorting topic. make_mcq writes four options and marks Merge sort correct. make_descriptive writes a five-mark text item. make_exam uses a window that is already open and ends later, a thirty-minute duration, pass mark 2.00, negative mark 0.50, and status DRAFT unless the test overrides it. attach creates the master paper rows.

Tests do not depend on the demonstration users or Operating Systems report values. Database rollback/isolation prevents one case’s attempt or result from becoming the next case’s precondition. Where a time rule matters, the test uses timezone.now() offsets or backdates the attempt explicitly.

Deterministic randomization. A ReverseRng test double implements shuffle by reversing the list. Passing it into start_attempt gives an exact expected order without weakening production, which still defaults to random.SystemRandom. The test then reloads rows from MySQL and checks that the order persists.

Boundary data. Validation tests use three/five options, zero/multiple correct flags, a topic from another subject, an inactive question, an empty paper, pass mark above total, duration above window, a future window, an ended window, a closed exam, a foreign selected option, a blank MCQ, heavy negative marks, and descriptive marks before and after publication.

System lifecycle fixture. The mixed-paper case starts with one MCQ and one descriptive item. Student submits, objective score is available, result is absent while descriptive is pending, Student is forbidden to mark, Faculty marks, result becomes EVALUATED/unpublished, Faculty publishes, Student can then see correctness, and another mark is refused.

Exit evidence. The report run printed 33 dots and 35 passed in 6.57s. The exact elapsed time is environment-specific; the assertions, not speed, determine pass. A later verification in this workspace also passed, but the report retains the one transcript and date used during evidence capture.

Reproduction limits. The browser flow is not a pytest module. There is no automated React component assertion, concurrency/load harness, security scanner, TLS test, backup-restore test, accessibility audit or user-acceptance survey. Table 6.4 keeps those absences separate from Pass.

6.10 Interpreting the executed result

Thirty-three collected cases do not mean thirty-three features. Four cases are parameter rows of the same MCQ option-rule function, and several lifecycle tests assert many behaviors. Conversely, an implemented URL may have no dedicated case. The inventory in Sections 6.7 and 6.8 is therefore more informative than the number alone.

Security coverage. Tests reject anonymous, inactive and wrong-role users, keep unpublished marks private, prevent post-publish marking, restrict audit, and prove the runtime database login cannot execute DDL. They do not test cryptographic token attacks, rate limiting or TLS.

Unit coverage. Two direct service tests inject deterministic randomization and check frozen rows. Most rules are intentionally exercised through DRF because serializers, permissions, services and database constraints must work together.

Integration coverage. Bank shape, topic consistency, exam lifecycle, draw rollback, window checks, answer validation, submission, expiry and objective scoring are all API/database assertions. They are the centre of the suite.

System coverage. The mixed-paper lifecycle, audit sequence and report visibility span modules. The separate browser journey demonstrates React routes but is not counted as pytest.

Repeatability. The recorded command was run against a test database with fixtures, not the demonstration result rows. A future source or dependency change requires a new run; the present Pass text must not be copied forward automatically.

6.11 Test strategy rationale by level

Service-level unit tests

The paper-freeze behavior needs an exact expected random order. Calling the service directly with a reversible test RNG is clearer than asserting that a random production order is “different”. The test checks database rows after the call, so it covers the algorithm and persistence. The complete-set test checks that a descriptive item is not lost and does not receive option-order rows.

These are the only cases classified primarily as unit tests. The report does not relabel every DRF API case as a unit test to increase a count.

Serializer and API integration tests

Question and exam rules cross request parsing, serializer validation, permissions, models, database constraints and services. Calling URL routes with DRF APIClient is the appropriate integration level. The tests assert status and stored rows, including rollback after a failed draw and unchanged state after a foreign option.

Authentication tests similarly exercise password validators/hashers, custom user manager, SimpleJWT and the user response. A mocked token library would not show the actual integration.

Database security tests

The DDL probe deliberately avoids Django's database test fixture for the operation under test. It connects with MySQLdb as oes_app, issues CREATE TABLE and expects the database server to deny it. This proves the configured account privilege rather than only checking the text of grants.sql.

Lifecycle/system tests

The mixed evaluation case is the core system scenario: two item types, Student ownership/embargo, Faculty marking, result finalization, publish/reveal and post-publish lock. The report test crosses publish state. The audit test performs the actions and asks Admin for the event list. These cases provide more value than a separate "system test" that only opens health.

Browser verification

The browser journey checks React routing/forms and visible state against the real API. It caught an export mismatch and asynchronous screenshot timing. Because it was run as an evidence script rather than committed under the automated suite, Chapter 6 names it manual/system evidence and does not include it in 33.

Regression selection

An account/permission change requires auth, every role gate, browser login/navigation and audit. A catalogue change requires question/exam/freeze tests because historic paper integrity depends on bank behavior. A timer/attempt change requires all conduction/evaluation/report/audit tests. A report-only query change requires report values and Chapter 8 snapshot reconciliation. A schema migration requires full suite plus backup/restore rehearsal in the target environment.

6.12 Requirement coverage summary

Table 6.10 — Requirement coverage and named gaps

Requirement area	Automated strength	Browser evidence	Named gap
Registration/login/RBAC	strong	login, registration, three navigations	refresh/expiry/tamper
Bank validation	strong create/shape/deactivate	bank screen	subject/topic CRUD; activate separate
Exam lifecycle	strong create/add/draw/publish/close	exam list/editor	unpublish
Start/freeze/timer/save/ submit	strong	exam room and submission	concurrent stress/tab-close automation
Objective scoring	strong	published breakdown	none for observed formulas
Descriptive evaluation/publish	strong system case	evaluation screens	user-acceptance workflow
Student/faculty/admin reports	principal visibility/role	all report screens	export and large-data performance
Audit	six actions/role	audit screen	retention beyond 500
DB grants	direct denial	not applicable	backup/restore and production account review
Frontend	production build/manual journey	screenshots	component/E2E suite/accessibility
Production security	partial application controls	none	TLS, rate limit, scan, monitoring

Coverage is evaluated by behavior and boundary, not percentage. No line/branch coverage tool was executed, so this report does not state a coverage percentage.

Chapter 7 System Security

Security is described only as it exists in source. Proctoring, OTP, CAPTCHA, IP whitelist, field-level AES, httpOnly cookies, and facial recognition are **not** present and are not claimed.

7.1 Authentication

Custom user; USERNAME_FIELD = email.

Hasher order Argon2, PBKDF2, PBKDF2-SHA1.

SimpleJWT; failed login writes audit with user=NULL and detail failed:{email}.

Access 8 hours; refresh 1 day; Bearer header.

React stores tokens in localStorage (oes_access, oes_refresh) and retries once on 401. This is a documented limitation, not an httpOnly-cookie design.

Inactive users fail authentication. Minimum password length 8; common and numeric-only rejected.

7.2 Authorization and access-rights matrix

Table 7.1 — Access-rights matrix from permissions.py and views

Operation	Student	Faculty	Admin	Anonymous
Register / login	yes	yes	yes	register/login only
GET /me/	own	own	own	401
User admin CRUD	403	403	yes	401
Question bank write	403	yes	yes	401
Exam write / publish / close	403	yes	yes	401
Start attempt / save / submit	own	403	403	401
Descriptive marks / publish result	403	yes	yes	401
Student history / published result	own	403	403	401
Faculty exam report / analysis	403	yes	yes	401
Admin summary / audit (500 cap)	403	403	yes	401

7.3 Database and examination controls

oes_migrator has DDL. oes_app has SELECT/INSERT/UPDATE/DELETE only. Tests observed MySQL 1142 when oes_app issued CREATE TABLE. The running application is not configured to log in as root.

Examination controls: unique sitting, frozen paper, server clock, auto-submit, option must belong to frozen order, result embargo, question lock after an attempt exists, publish refused if the window already ended. These are application rules, not a locked browser.

7.4 Production controls still required

HTTPS/HSTS, secret rotation (development SECRET_KEY default must be replaced), token storage other than localStorage, reverse-proxy hardening, backups, monitoring, rate limiting, privacy/retention policy, and incident response are **not** claimed for the local run. Admin audit listing is capped at 500 rows.

7.5 One request, followed through the layers

Take POST /api/exams/1/start/ from a student browser.

The React page sends the access token. It does not send a role or a user id in the body.

JWT authentication loads users by the token subject. If the signature is wrong or the row is inactive, the view is not called.

IsStudent allows the call only when role is STUDENT.

start_attempt loads the exam, rejects a draft, a closed exam, and a clock outside [start_at, end_at), and inserts the attempt under the unique key.

freeze_paper writes the shuffled question rows and option rows in the same transaction. A failure rolls the attempt back.

The serializer builds JSON from those rows and omits is_correct.

record_audit inserts START with the attempt id and the exam id.

MySQL, connected as oes_app, may insert and update. It cannot create a table to hide a second attempt.

That path is the security argument for this project: identity from the token, role from the row, time and uniqueness from the database, and the answer key withheld. It does not extend to a camera or a locked browser.

7.6 Threat, control and residual-risk review

Threat	Implemented control	Residual statement
Credential disclosure	Argon2 hash; password omitted from serializers; HTTPS still required	Database read does not reveal plaintext; a stolen token in localStorage remains a stated risk.
Self-elevation	Register forces STUDENT; staff provisioning is Admin-only	A role value in public registration is ignored and tested.
Broken object authorization	Student attempt queries include owner; result detail uses authenticated user	Other-student access is 404/403; a formal IDOR penetration test was not run.
Question-key leakage	is_correct omitted during IN_PROGRESS and revealed only on published result	Browser and lifecycle tests cover the principal disclosure boundary.
Duplicate or concurrent start	Database unique exam+user plus transactional create	Second start is 409. A high-concurrency stress test is not claimed.
Client clock manipulation	Server deadline and ensure_open	Changing the browser countdown cannot change started_at or end_at.
SQL injection / schema damage	Django ORM parameterization; DML-only oes_app	DDL denial is tested. A dedicated injection scanner was not run.
Result tampering	Awards validated; totals calculated on server; published result locks further marking	Out-of-range and post-publish behavior are service/view rules.
Audit deletion or overflow	Admin-only query and retained user-null events	The in-app list is capped and is not an external write-once archive.
Denial of service	No measured rate limit or scale test	This is an unresolved production control, not described as protected.

The table uses only controls visible in settings, permissions, services, database grants and tests. It does not turn the synopsis’s future proctoring language into a current control.

7.7 Security verification scenarios for deployment review

The following scenarios are not additional executed test cases. They are a checklist for the institution before exposure outside the demonstration host.

Attempt login repeatedly from one address and confirm the chosen reverse-proxy or application rate limit, lockout and alert policy. No such measured policy is in the current code.

Use an expired access token and a revoked/deactivated user during refresh. Confirm refresh policy, token rotation and incident response.

Change object ids in Student requests for attempts and results. Confirm owner filters on every new endpoint added after this report.

Inspect browser storage and logs for access/refresh tokens. Decide whether localStorage remains acceptable or move to a reviewed cookie/BFF pattern with CSRF protection.

Run the production domain through TLS and header scanners. Confirm redirect, HSTS after rollout confidence, frame policy, content-type policy and proxy forwarding.

Restore a database backup to an isolated host. Verify users, frozen papers, answers, results and audit rows, then document recovery time and data loss.

Fill the audit table beyond 500 rows. Confirm retention/export/search requirements rather than treating the UI slice as the archive.

Exercise two start requests and two submit requests concurrently. Database uniqueness and row locks are designed for this, but production concurrency should be observed.

Review Faculty/Admin provisioning, deactivation and departed-staff access with the real institution owner.

Review privacy notices and retention before adding any camera, microphone, face or screen data; those data are not currently collected.

Until these scenarios are executed in the target deployment, Chapter 7 calls them required controls, not completed controls.

7.8 Layered security configuration

Browser and transport layer

Development origins are limited to localhost Vite entries. Production must use an HTTPS origin and matching CORS/CSRF settings. Tokens in localStorage are exposed to successful script injection; the project therefore needs a content-security policy and a reviewed token-storage decision before production. X-Frame-Options middleware is enabled, but response-header verification belongs to the deployed reverse proxy as well.

Django middleware and API layer

SecurityMiddleware, CommonMiddleware, CSRF, Authentication and X-Frame middleware are installed. JWT is the DRF authentication class and IsAuthenticated the default permission. Views override only for public auth/health or use stronger role classes. Serializer validation is the input boundary. Exceptions are converted to structured response codes, not stack traces when DEBUG is off.

Service and transaction layer

Cross-row exam rules are services, so the same invariant is used by API paths. Atomic blocks prevent partial paper/attempt/answer operations. Row locks protect submit. Randomization uses SystemRandom. The server, not React, determines ownership, deadline, awards and publication.

ORM and database layer

ORM parameters avoid hand-concatenated SQL. Foreign keys, uniques and checks survive a bypassed serializer. Runtime grants deny DDL. MySQL must be reachable only from allowed application/admin networks and backups encrypted/controlled; those are target-environment duties.

Operational layer

Staff accounts need named ownership, deactivation, strong password/MFA policy, least-role assignment and review. Secrets must be outside source and rotated. Logs/audit need retention and

alerting. Backups need restore tests. Patch management needs a test/deploy/rollback process. None of those is proven by the local health response.

Examination-specific layer

The bank key is staff-only. During a sitting the key is omitted and option ids are limited to frozen rows. One attempt and server deadline protect the main exam rules. Result embargo prevents early status/key disclosure. These controls improve integrity; they do not detect an external helper, second device or screen capture. The report therefore avoids “cheating prevention” as an absolute claim.

7.9 Access review checklist

Table 7.3 — Access review checklist

Review item	Evidence in this build	Production decision
Public registration role	always STUDENT	retain, restrict domain, invite, or admin-only
Faculty/Admin creation	Admin API	identify account approver and periodic review
Inactive enforcement	login + permission base	incident/offboarding time target
Access-token life	8 hours	choose risk-appropriate lifetime
Refresh-token life	1 day	rotation/revocation/blacklist policy
Student object ownership	attempt/result filters	retest every added endpoint
Faculty scope	all academic exams in this installation	add subject/cohort assignment if required
Admin scope	users, summaries, audit plus faculty operations	consider separated super-admin role
Database runtime	DML only	secret storage, host/network rule, rotation
Audit visibility	Admin, newest 500	archive/export and privacy access policy
Result correction	locked after publish	authorized correction workflow/audit
Exam eligibility	all active Students in window	add enrollment/cohort before restricted exams

Chapter 8 Results and Reports

This chapter shows **sample layouts** using demonstration data read from the live API on 5 October 2026 (Operating Systems Midterm and Database Quiz draft). They illustrate columns, not official institutional results. Screenshots of the same screens follow the tables.

8.1 Student history layout

Table 8.1 — Student history (demo student@oes.local, 5 Oct 2026)

Exam	Subject	Attempt status	Result	Total	Objective	Descriptive	Pass mark
Operating Systems Midterm	Operating Systems	EVALUATED	PASS (published)	7.50 / 10.00	5.50	2.00	4.00

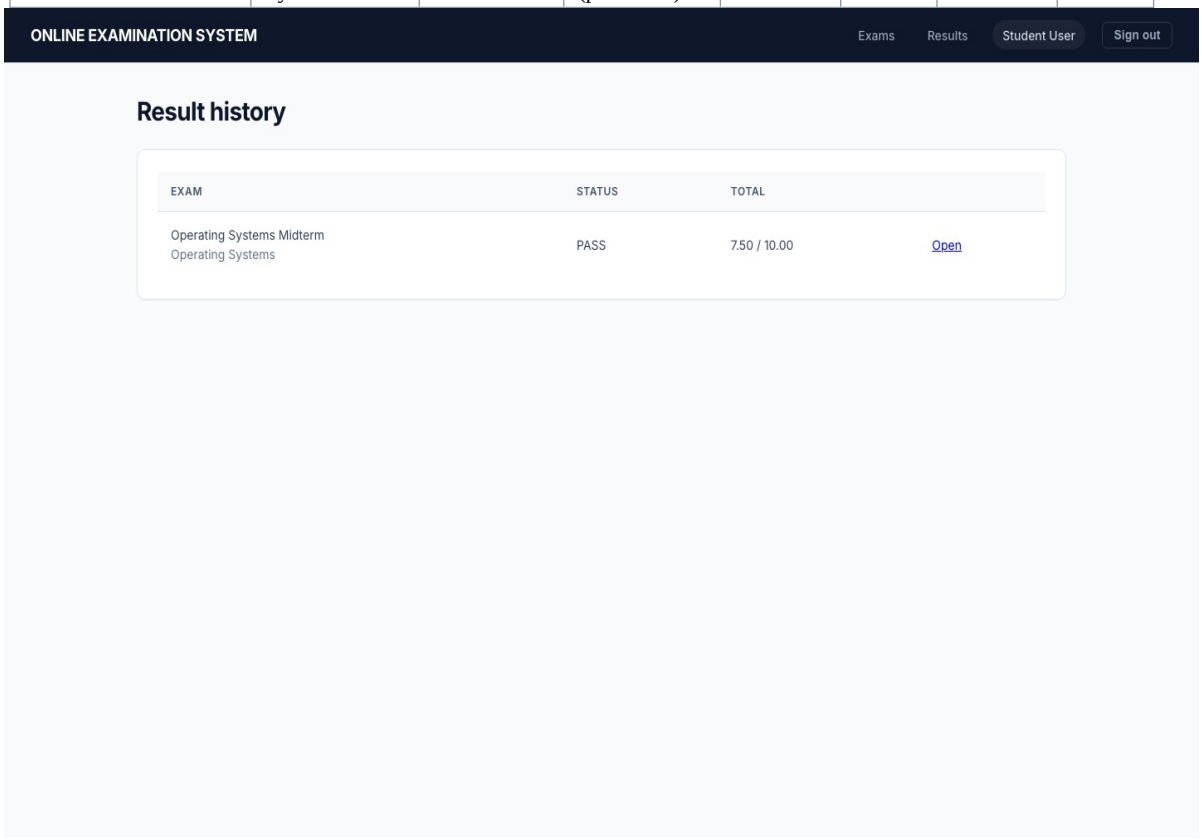


Figure 8.1 — Published student result list (demo).

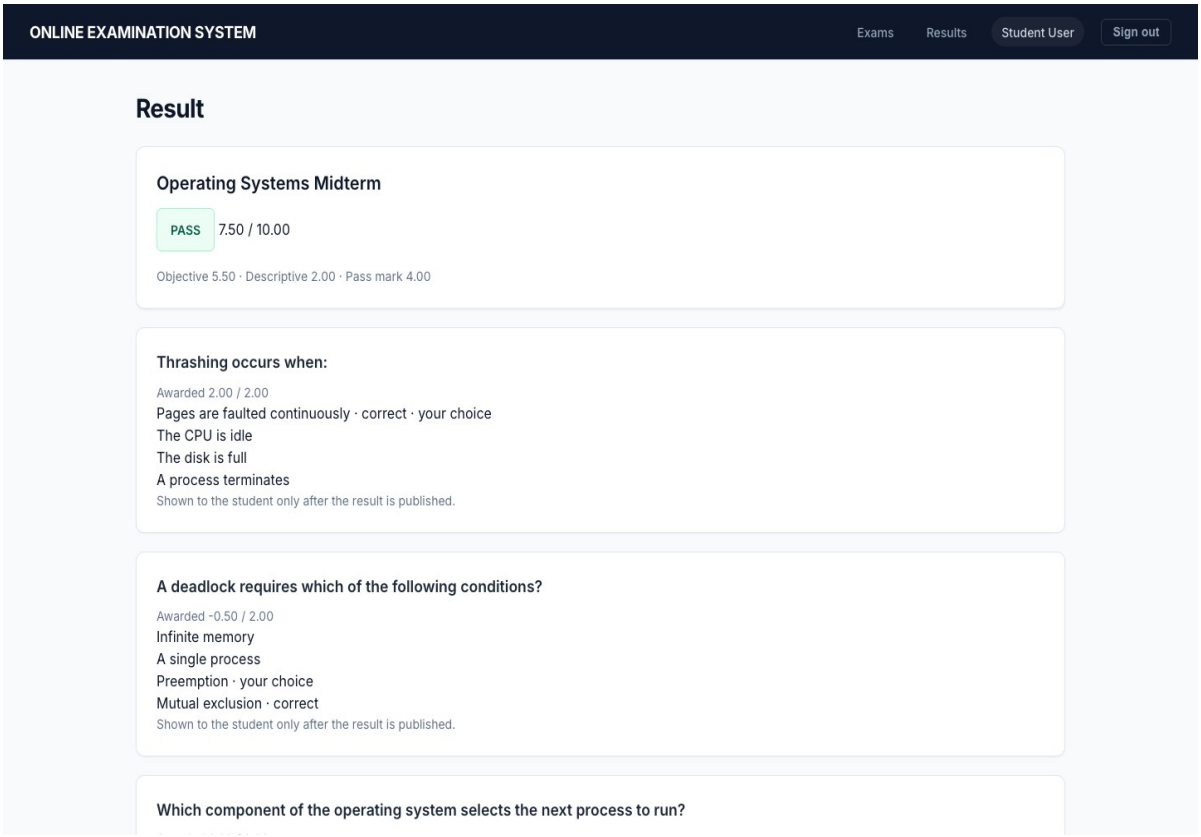


Figure 8.2 — Published result detail with answer review after publish.

8.2 Faculty exam analysis layout

Table 8.2 — Faculty exam report cards (live GET /api/reports/faculty/exams/1/)

Metric	Operating Systems Midterm (demo)
Registered students (active STUDENT count)	3
Started / submitted / in progress / evaluated / published	2 / 2 / 0 / 1 / 1
Pass / fail (published rows only)	1 / 0
Average / high / low	7.50 / 7.50 / 7.50
Pass marks	4.00
Participation rate (submitted / registered)	0.6667

Table 8.3 — Faculty topic/difficulty analysis (live GET …/analysis/)

Topic	Difficulty	Type	Questions	Responses	Correct	Average awarded	Average %
Memory	EASY	MCQ	1	2	2	2.00	100.00
Memory	HARD	MCQ	1	2	2	2.00	100.00
Processes	EASY	MCQ	1	2	2	2.00	100.00
Processes	MEDIUM	DESCRIPTIVE	1	1	0	2.00	100.00
Processes	MEDIUM	MCQ	1	2	1	0.75	37.50

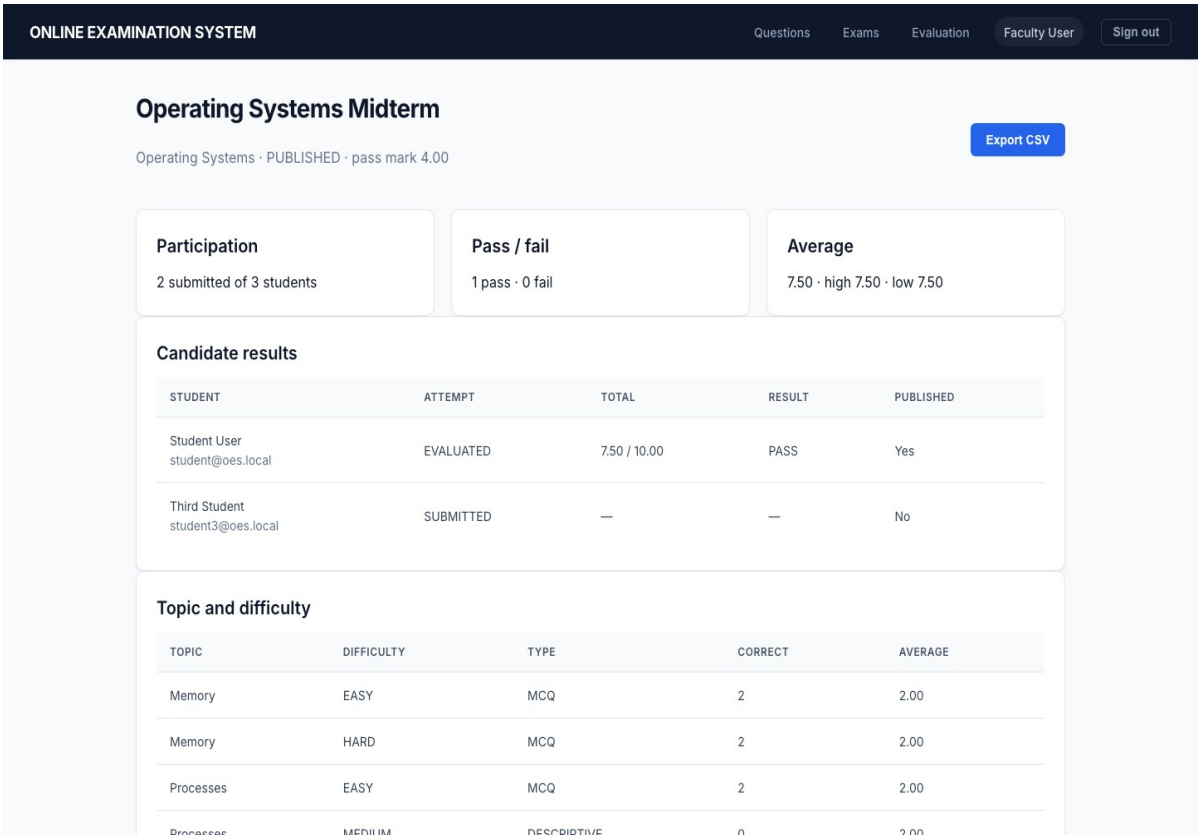


Figure 8.3 — Faculty report screen.

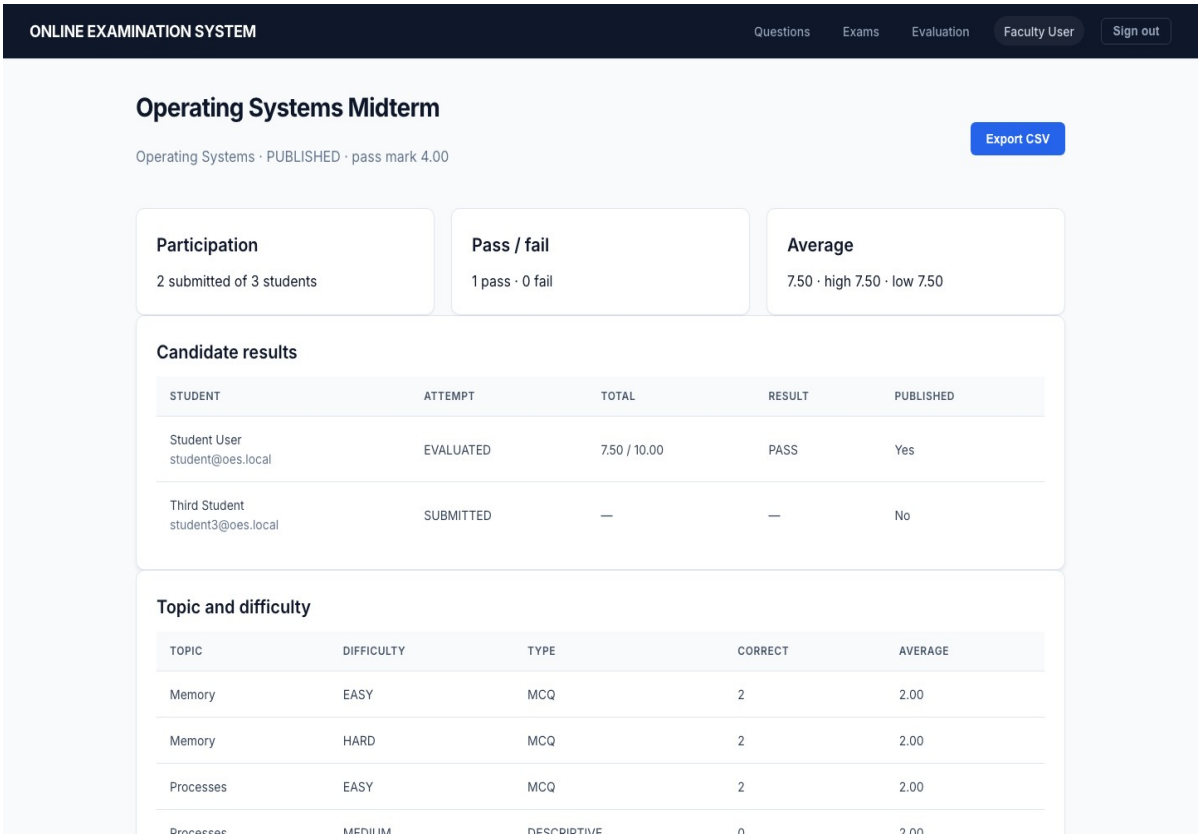


Figure 8.4 — Operating Systems Midterm analytics.

8.3 Admin summary layout

Table 8.4 — Admin summary rows (live GET /api/reports/admin/summary/). Institution totals: published_results=1, pass=1, fail=0, average=7.50

Exam	Status	Started	Submitted	Pass	Fail	Average
Operating Systems Midterm	PUBLISHED	2	2	1	0	7.50
Database Quiz (Draft)	DRAFT	0	0	0	0	—

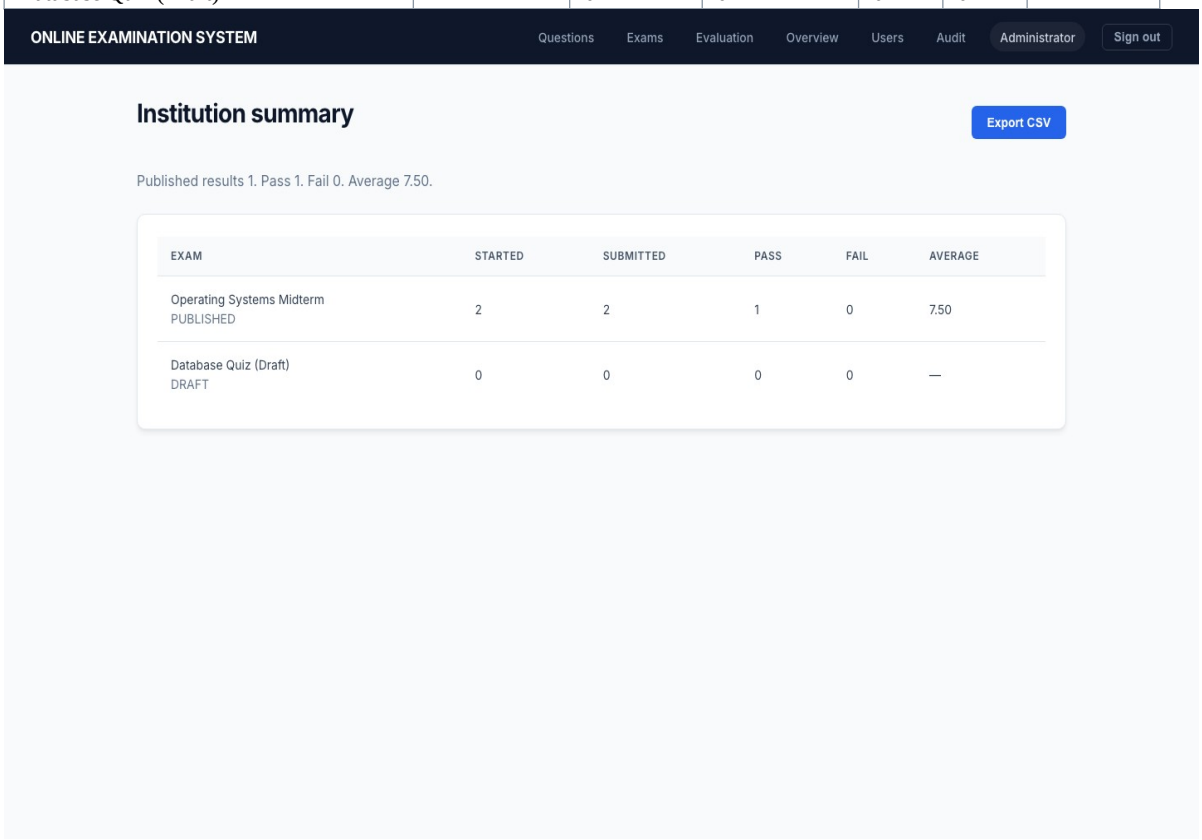


Figure 8.5 — Admin summary.

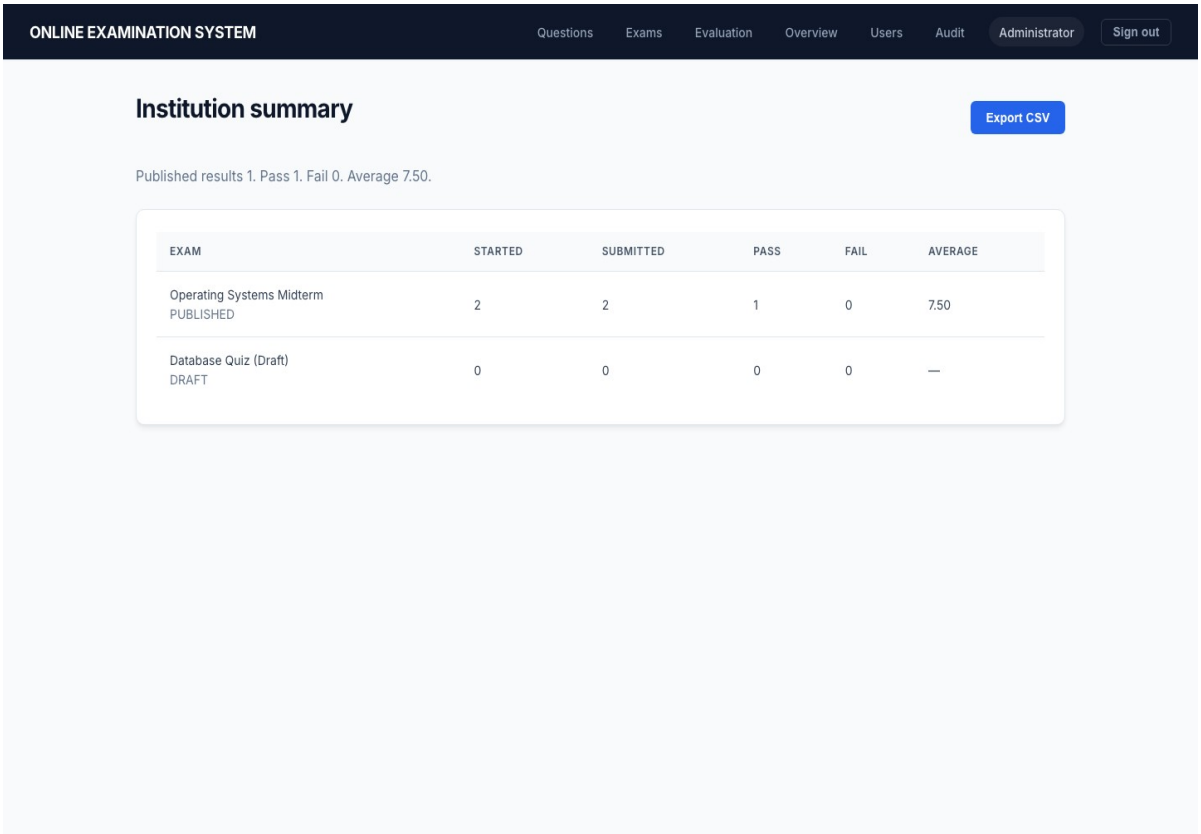


Figure 8.6 — Admin summary detail.

8.4 Audit extract layout

Table 8.5 — Audit layout (latest 500; filters action/user/entity)

Action	Entity	Detail (examples observed)	Who may list
LOGIN	user	success or failed:{email}	Admin
START	attempt	exam={id}	Admin
SAVE	attempt	count={n}	Admin
SUBMIT	attempt	manual or auto	Admin
EVALUATE	attempt	question ids	Admin
PUBLISH	result	exam={id}	Admin

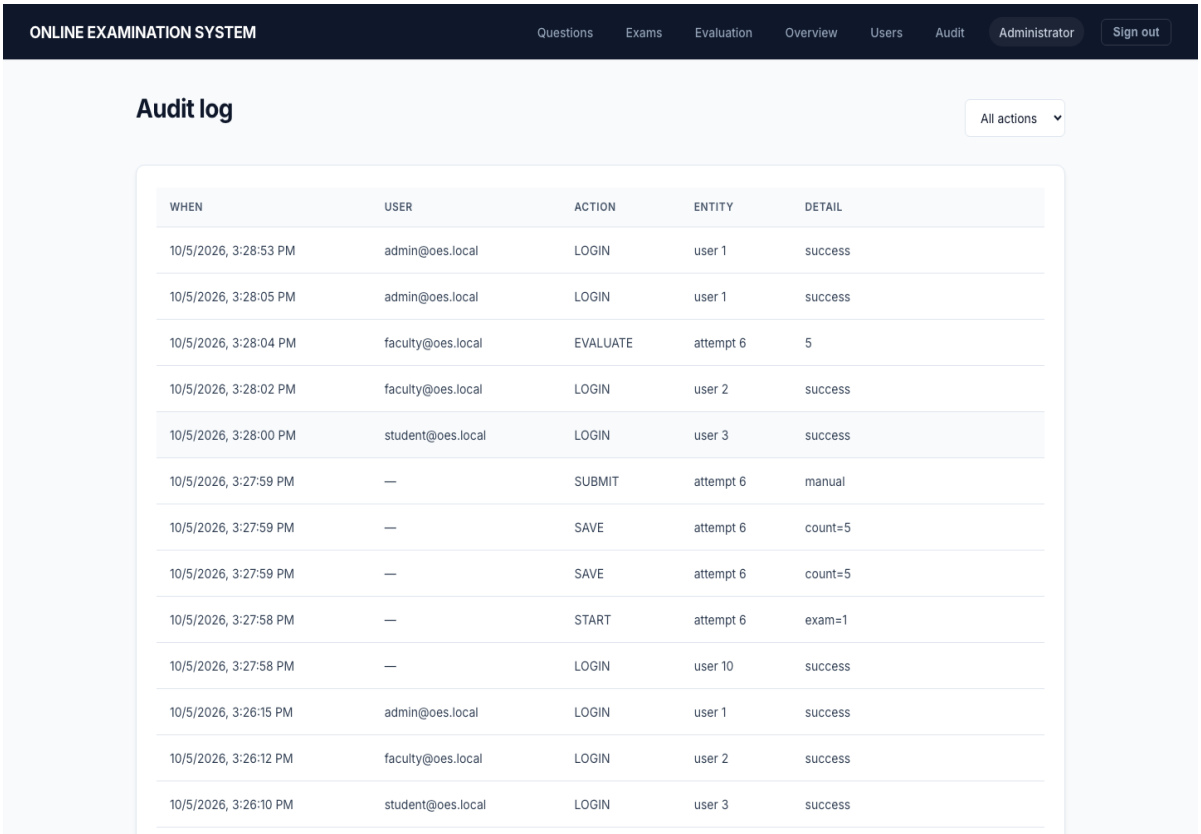


Figure 8.7 — Admin audit log (demo).

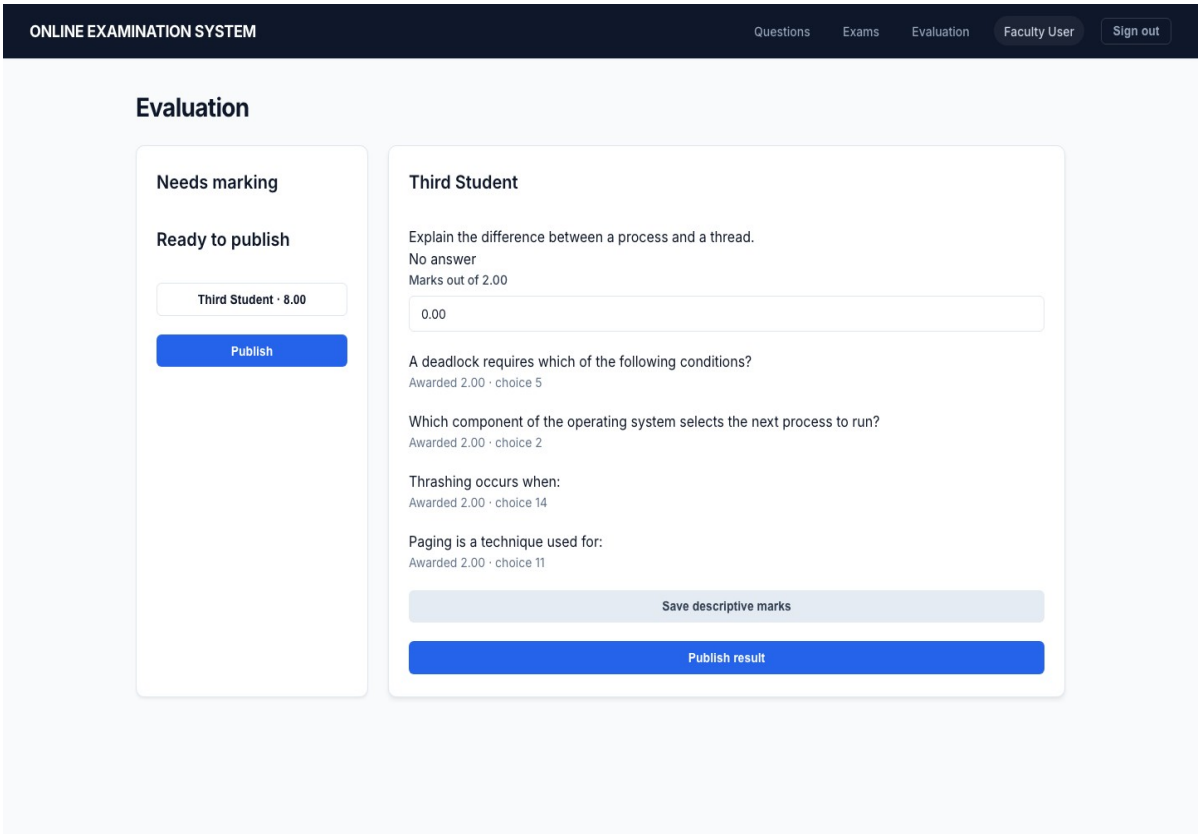


Figure 8.8 — Evaluation working lists (operational, not a stored report file).

8.5 Reading the two observed totals

Two different published totals appear in this report's evidence. They are not two versions of the same formula.

Capture during the browser run. Report Student's published paper, Figure 8.2, is FAIL, 2.50 / 10.00. The breakdown on that screen is objective 0.50 and descriptive 2.00, pass mark 4.00. The objective 0.50 is what remains after correct MCQ awards and negative awards of 0.50 are added and then left above zero. The descriptive award is 2.00 out of 2.00. Because 2.50 is below 4.00, status is FAIL. The faculty screenshot of the same period shows pass 1, fail 1, high 4.00, low 2.50, average 3.25. Those three summary numbers are the mean, maximum and minimum of 4.00 and 2.50. The admin screenshot's "published results 2, pass 1, fail 1, average 3.25" is the same pair of published rows.

Later API read the same day. After more demonstration use, GET /api/reports/faculty/exams/1/ returned one published result: pass_count 1, fail_count 0, average_total 7.50, highest 7.50, lowest 7.50. Table 8.1 is that row for student@oes.local: total 7.50 / 10.00, objective 5.50, descriptive 2.00, PASS against pass mark 4.00. 5.50 + 2.00 = 7.50, and 7.50 ≥ 4.00, so PASS follows the same comparison as the earlier FAIL.

What did not change. Pass, fail and average in faculty_exam_report use only published=True. Submitted attempts that are still awaiting a descriptive mark, or awaiting the publish button, stay in the submitted count and stay out of the average. That is why a screenshot can show more submissions than pass+fail. Topic "correct" counts an MCQ only when the award equals the question marks, so a descriptive item scored 2/2 adds to the average and adds 0 to the correct column.

No production cohort is implied by either snapshot. Both are demonstration rows on the project database.

8.6 Question-level arithmetic on the captured FAIL paper

Figure 8.2 shows Report Student's five answers. The awards visible on that screen, read in display order, are:

Table 8.6 — Awards read from Figure 8.2

Question on the screen	Award shown	How that award is produced
Paging is a technique used for	−0.50	Wrong MCQ. The selected text was not "Virtual memory". score_mcq subtracts negative_mark 0.50
Explain the difference between a process and a thread	2.00	Descriptive. Faculty stored 2.00, the full marks of that item
Thrashing occurs when	−0.50	Wrong MCQ against the same negative mark
Which component selects the next process	−0.50	Wrong MCQ. The screen shows the selected text was not "Scheduler"
A deadlock requires which condition	2.00	Correct MCQ. The award equals the question marks

The objective sum is −0.50 −0.50 −0.50 + 2.00 = 0.50. Because 0.50 is already above zero, the floor in objective_total does not change it. Descriptive marks are 2.00. Total is 0.50 + 2.00 = 2.50. Maximum on this paper is 10.00 (four MCQs at 2.00 and one descriptive at 2.00). Pass mark is 4.00. 2.50 < 4.00, so the status FAIL on the figure follows the comparison in finalize_result. The keys are visible on that figure only because the result had been published; the exam-room capture (Figure 4.17) does not show them.

The later PASS of 7.50 uses the same comparison: objective 5.50 plus descriptive 2.00 is 7.50, and 7.50 ≥ 4.00. The floor rule and the pass rule did not change between the two rows. Only the answers and the published set did.

8.7 Screen-by-screen acceptance notes

The screenshots are the visual evidence. This section states what each screen is responsible for so the report is useful even when printed in grayscale.

8.7.3 Login

Route and actor. /login — Visitor.

Principal controls/data. Email and password; seeded-account reminder in the demonstration.

Acceptance note. Successful login redirects by role. Wrong credentials remain on the form with the server error.

8.7.11 Registration

Route and actor. /register — Visitor.

Principal controls/data. Name, email and password; no role selector.

Acceptance note. The server assigns STUDENT. This screen is evidence that a visitor cannot choose Admin.

8.7.19 Student dashboard

Route and actor. /student — Student.

Principal controls/data. Exam title, subject, window, question count, duration, state.

Acceptance note. AVAILABLE offers Start, IN_PROGRESS offers Continue, and a published result offers View Result.

8.7.27 Exam room

Route and actor. /student/attempts/{id} — Student.

Principal controls/data. Server timer, numbered palette, radio/textarea, save note, previous/next, submit.

Acceptance note. Order comes from frozen rows. The key is absent. Save runs on change and every fifteen seconds.

8.7.35 Result history

Route and actor. /student/results — Student.

Principal controls/data. Attempt rows and publish state.

Acceptance note. Marks are blank until publish; the detail link appears only for a published result.

8.7.43 Result detail

Route and actor. /student/results/{exam} — Student.

Principal controls/data. Total, breakdown, status, per-question award and, after publish, answer key.

Acceptance note. Figure 8.2 is a demonstration FAIL; Table 8.1 records a later PASS row.

8.7.51 Question bank

Route and actor. /faculty/questions — Faculty/Admin.

Principal controls/data. Filters and create/edit/deactivate actions.

Acceptance note. Rows show subject, topic, type, difficulty, marks and active state.

8.7.59 Exam list/editor

Route and actor. /faculty/exams and /faculty/exams/{id} — Faculty/Admin.

Principal controls/data. Draft schedule, marking scheme, fixed add, draw counts, publish/close.

Acceptance note. Draft-only controls disappear or fail after publish.

8.7.67 Evaluation

Route and actor. /faculty/evaluation — Faculty/Admin.

Principal controls/data. Needs marking, ready to publish, descriptive mark field, publish button.

Acceptance note. In-progress attempts are not offered. Publish remains a separate action.

8.7.75 Faculty report

Route and actor. /faculty/reports/{exam} — Faculty/Admin.

Principal controls/data. Participation, pass/fail, average/high/low and topic rows.

Acceptance note. Only published rows contribute to outcome statistics.

8.7.83 Admin users

Route and actor. /admin/users — Admin.

Principal controls/data. Create Faculty/Admin, list role/active, toggle active.

Acceptance note. The logged-in Admin cannot deactivate self.

8.7.91 Admin summary/audit

Route and actor. /admin and /admin/audit — Admin.

Principal controls/data. Per-exam summary; action filter and event rows.

Acceptance note. The summary is a query; audit returns the newest 500 rows.

8.8 Report query definitions

Student history. Select attempts for the authenticated user, join exam and subject, order by started_at descending, and map results by attempt id. Always return attempt and schedule status. Copy status and marks only when the result exists and published is true. This allows the dashboard to say “submitted” without leaking an unpublished FAIL/PASS.

Faculty exam report. Let attempts be every attempt for the exam. Let published be result rows for the exam where published=true. Registered students is the current count of active STUDENT rows; started is the attempt count; submitted excludes IN_PROGRESS; evaluated counts EVALUATED. Pass, fail, average, highest and lowest use published, not attempts. Participation rate is submitted / registered students when the denominator is non-zero.

Topic/difficulty analysis. Build buckets from frozen attempt questions, keyed by topic, difficulty and type. Add scored answer rows. responses increments for each scored answer; awarded and possible marks are summed. Correct increments only for an MCQ whose award equals the question marks. Average award is awarded / responses. Average percent is awarded / possible × 100. A descriptive bucket therefore has meaningful averages but a correct count of zero by definition.

Admin summary. Order exams and apply the faculty report function to each. Independently aggregate all published results for institution published count, average, passes and fails. The row and institution totals are query snapshots; they are not written back.

Audit extract. Select audit rows with related user, optional exact filters for action, user and entity, newest first, and return at most 500. The UI is an operational view. Long-term retention/export is a production decision.

These definitions explain the numbers in Chapter 8 and provide acceptance criteria for any later SQL/report rewrite.

8.9 User operating guide

8.9.1 Student: register and sign in

Open Registration, enter name, an email not already used, and a password that meets the validators. There is no role selector. After a successful response, open Login and use that email/password. The Student navigation shows Exams and Results. If login fails, correct the credentials; an inactive account requires an administrator rather than repeated attempts.

8.9.2 Student: start or resume

The Exams page shows the state calculated by the server. UPCOMING has no Start; AVAILABLE has Start; IN_PROGRESS has Continue; MISSED cannot start; a completed exam may show its result only after publication. Start once. Do not open a second account/window expecting another sitting: the database allows one attempt for that student and exam.

The exam room displays title/instructions, remaining MM:SS, a numbered palette and one current question. Choose one option for an MCQ or type a descriptive response. The selection is saved immediately and the page also saves every fifteen seconds. “Saved on the server” means the last PUT succeeded; it does not guarantee unsaved keystrokes during an outage. Use Previous/Next or the palette. Submit when finished. At zero the server submit path applies even if the browser is delayed.

8.9.3 Student: result

After submission, objective marks may have been calculated, but the result remains private while descriptive work or publication is pending. Results history shows the attempt without marks. Once published, Open displays PASS/FAIL, component and total marks, per-question awards, the selected option and the key/explanation. This review screen is why key data must not appear during the exam.

8.9.4 Faculty: prepare the bank

Sign in with an account provisioned by Admin. Open Questions. Create/reuse subject and topic, select MCQ or descriptive, difficulty and positive marks. For MCQ, enter four non-empty option texts and select exactly one key. For descriptive, do not add options. Deactivate an item to remove it from new selection while preserving history. Do not attempt to rewrite an item that a student has already seen.

8.9.5 Faculty: schedule and publish

Open Exams and create a draft. Enter a title, same-subject schedule, duration no longer than the window, pass mark and optional non-negative wrong-answer deduction. Open the draft. Add active bank items individually or enter draw counts. Review total marks and paper. Publish only after checking instructions/window and pass mark. Published papers are no longer edited. Close stops a new start. Return-to-draft is only for a published exam with no attempts.

8.9.6 Faculty: evaluate and publish result

Open Evaluation. Needs Marking lists submitted mixed papers. Select an attempt, read each descriptive response, enter 0 through that question's marks, and save. When every descriptive item is scored, the result moves to Ready to Publish. Review total/status and press Publish. Publishing is not automatic because it is the disclosure decision. A published result cannot be re-marked through the service.

8.9.7 Faculty: read analytics

Open an exam report. Participation shows registered, started, submitted, in-progress and evaluated separately. Outcome figures use published results only. Topic/difficulty rows show responses and average awards; "correct" is meaningful for MCQ only. If submitted is greater than pass+fail, some submitted results are not published.

8.9.8 Admin: users

Open Users. Create Faculty or Admin accounts with name/email/password/role. Review active state. Deactivate an account that should no longer authenticate. The system refuses self-deactivation in this screen to avoid locking out the only current administrator. Student self-registration remains public in the implemented scope.

8.9.9 Admin: summary and audit

Overview shows each exam's lifecycle and outcome query plus institution totals. Audit shows newest events and supports action/user/entity filters. The list is capped at 500 and is not the backup. A failed login can have no user email because authentication never succeeded; detail still contains the attempted email.

8.9.10 Demonstration reset and data caution

seed_demo is for an academic demonstration. It prints passwords and may create additional Student rows if the command version does so. Never run it against real institutional data. Screenshots in this report are demonstration records. Replacing them with real names/results would require consent and a new evidence review.

8.10 Administrator maintenance guide

Before a session: confirm MySQL and storage capacity, current migrations, system time/timezone, domain/TLS, backup success, account list, exam window/paper total, and that the browser can reach health/login. Do not test by starting a real student's one allowed attempt.

During an exam: monitor server/database availability and application logs, not student cameras (none exist). Do not edit published questions. If an outage occurs, preserve logs and database state before manual intervention. The deadline remains based on stored times.

After an exam: verify attempts left IN_PROGRESS only where legitimately unfinished, complete descriptive marking, review then publish, export/retain reports according to institutional policy, and preserve a backup before upgrades.

Upgrade: snapshot/backup; test migrations against a copy; install pinned/approved dependencies; run manage.py check, migrations, pytest and frontend build; exercise the browser journey; then deploy. Rollback must consider database migrations, not only static files.

Account incident: deactivate the row, rotate relevant credentials, review audit and proxy logs, decide token invalidation strategy, and document the event. The current JWT design has no general server-side token blacklist configured.

Data correction: do not update marks with ad-hoc SQL. If an institution introduces an approved correction workflow, it needs authorization, before/after audit, recalculated result and a disclosure policy. The current publish lock intentionally rejects casual re-marking.

Chapter 9 Limitations, Future Scope and Conclusion

9.1 Current limitations

Single-institution academic build; public Student self-registration with no per-exam enrolment list.

HTTP in development; TLS is a reverse-proxy step, not a Django setting.

JWT in localStorage; no rate limiting module; Admin audit capped at 500 rows.

Development defaults for DEBUG and SECRET_KEY must be replaced.

No rich-text questions, attachments, email/SMS, or distributed task queue.

No formal accessibility audit; no automated React component tests.

Pytest strongly covers exam rules and grants (35 passed) but only two cases are service-level unit tests; refresh, subject/topic CRUD, unpublish, and several list APIs lack dedicated cases.

Study centre, guide designation, year of submission, signatures, and the original approved proforma remain administrative attachments.

9.2 Future scope

Items below are future work. They are not in the current modules. Monetization, AI job matching, and other Job Portal sample ideas are not imported.

OTP, CAPTCHA, two-factor login, and optional IP policy.

TLS termination, HSTS, and moving tokens out of localStorage.

Email/SMS notifications and LMS (Moodle) integration.

Native mobile clients and offline exam packing.

Richer analytics and accessibility certification.

A separately secured coding-sandbox service if the institution needs executable questions.

Camera / microphone / screen monitoring, browser lockdown, face recognition, and AI malpractice detection — only after legal, privacy, accessibility and security review.

9.3 Conclusion — objectives versus evidence

Table 9.1 — Objectives mapped to evidence

Objective	Status	Evidence
OBJ-01 Registration and login	Implemented	SEC-AUTH-01, SYS-AUTH-01, Fig. 4.14–4.15
OBJ-02 RBAC	Implemented	Table 7.1; SEC-AUTH-04 and role 403 cases
OBJ-03 Question bank	Implemented; no coding sandbox	INT-QB-*; Fig. 4.20
OBJ-04 Exam lifecycle	Implemented DRAFT/PUBLISHED/CLOSED	INT-EX-*; Fig. 4.19, 4.21
OBJ-05 Timed frozen attempt	Implemented including attempt_option_orders	UNIT-COND-*; Fig. 4.17
OBJ-06 Hybrid evaluation	Implemented	SYS-EVAL-01; Fig. 4.22
OBJ-07 Embargoed results	Implemented published flag	SYS-EVAL-01; Fig. 8.2
OBJ-08 Query reports	Implemented; no report table	SYS-RPT-01; Tables 8.1–8.4
OBJ-09 Audit	Implemented six actions	SYS-AUD-01; Fig. 8.7

ONLINE EXAMINATION SYSTEM, submitted by Sushmita Mishra (Enrolment No. 2350025885) under the guidance of Shiv Pratap Singh, is a Django/React/MySQL application that conducts a published exam with a server timer, a frozen shuffled paper, objective scoring with optional negative marking, faculty descriptive marks, and query-based reports, with role checks and an audit log. Claims stop at the 5 October 2026 evidence. Administrative signatures and production hardening remain the candidate's binding-time and deployment tasks.

9.4 Objective-by-objective validation

OBJ-01 — registration and authentication. Achieved for public Student registration and login of all active roles. Evidence is the registration/login/profile pytest group and Figures 4.14–4.15. It does not include email verification or forgotten-password mail.

OBJ-02 — role-based access. Achieved for the documented APIs using permission classes and ownership filters. Evidence includes Admin user provisioning denial, Student exam-create denial, Faculty start denial, Student mark/audit/report denial, and the access matrix. Production account governance remains external.

OBJ-03 — reusable question bank. Achieved for subjects, topics, MCQ/descriptive items, difficulty, marks, options/key and active state. Validation tests cover shape and topic consistency. Import, attachments and version history are not achieved and were not objectives of the implementation scope.

OBJ-04 — exam definition and lifecycle. Achieved for DRAFT, paper add/draw, schedule/marking scheme, publish, close and restricted unpublish. Tests cover the central guards. Per-course enrollment and a reschedule policy after attempts are not implemented.

OBJ-05 — one timed randomized sitting. Achieved through unique attempt, frozen question/option rows, server deadline, save and manual/timeout submit. Unit/integration tests and exam-room screenshots support this. The software does not assert browser lockdown or proctor identity.

OBJ-06 — objective evaluation and negative marking. Achieved and tested for correct, wrong, blank and aggregate floor. The FAIL arithmetic in Section 8.6 demonstrates the same algorithm using captured values.

OBJ-07 — descriptive manual evaluation. Achieved for a Faculty/Admin range-checked award, EVALUATED transition and publication lock. There is no automated language assessment.

OBJ-08 — result embargo and history. Achieved: result calculation can precede publication; Student detail is forbidden until publication; history hides totals. The mixed and objective-only cases cover both result paths.

OBJ-09 — Faculty analytics. Achieved as query-only exam and topic/difficulty/type outputs. Figures and API reads show the layouts. No downloadable signed report file or benchmark for a large cohort is claimed.

OBJ-10 — Admin summary and audit. Achieved for institution published totals, per-exam rows, account management and a newest-500 event extract. Audit is not a permanent external compliance archive.

OBJ-11 — relational integrity and least privilege. Achieved through thirteen normalized domain tables, foreign/unique/check constraints, protected history and separate MySQL roles. Direct DDL denial is tested.

OBJ-12 — maintainability and evidence. Achieved through four Django apps, service-layer rules, migrations, React route groups, requirement/file/test maps, diagrams, test transcript and demonstration screenshots. Frontend component automation and production observability remain improvement items.

Overall, the core academic objectives are implemented and evidenced. “Achieved” in this conclusion refers to this tested demonstration scope, not to future proctoring, national scale, formal accessibility certification or production service levels.

Bibliography

Indira Gandhi National Open University. *MCSP-232 Project Guidelines*, July 2024 / January 2025 (supplied PDF).

Project synopsis. *ONLINE EXAMINATION SYSTEM* (supplied PDF and front page naming Sushmita Mishra, enrolment 2350025885, guide Shiv Pratap Singh).

Sommerville, I. *Software Engineering*. Pearson. (method: Agile, requirements, testing)

Pressman, R. S. and Maxim, B. R. *Software Engineering: A Practitioner's Approach*. McGraw-Hill. (method: SRS, estimation)

Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems*. Pearson. (method: ER, normalization)

Django Software Foundation. Django documentation. https://docs.djangoproject.com/

Django REST Framework documentation. https://www.django-rest-framework.org/

SimpleJWT documentation. https://django-rest-framework-simplejwt.readthedocs.io/

Python Software Foundation. Python 3 documentation. https://docs.python.org/3/

React documentation. https://react.dev/

Oracle. *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/

Gunicorn documentation. https://docs.gunicorn.org/

IEEE Computer Society. IEEE Std 830, recommended SRS practice (structure only).

GeeksforGeeks, FreeCodeCamp, MongoDB, Express, Redux, and other Job Portal sample sites are omitted because they were not used.

Appendix A — Traceability

Table A.1 — Requirement traceability

Requirement	Module	Table(s)	API	Test	Report
FR-01–07	User	users	/api/auth/*, /api/users/*	test_auth.py	Ch. 3, 4, 7
FR-08–13	Bank	subjects, topics, questions, options	/api/questions/	test_questions.py	Ch. 3, 4
FR-14–19	Exam mgmt	exams, exam_questions	/api/exams/	test_exams.py	Ch. 3, 4
FR-20–28	Conduction	attempt*	/api/exams/{id}/start/, /api/attempts/	test_conduction.py	Ch. 3, 4, 7
FR-29–34	Evaluation	attempt_answers, results	/api/evaluation/, /api/results/	test_evaluation.py	Ch. 3, 4
FR-35–37	Reports	queries	/api/reports/	test_reports.py	Ch. 8
FR-38	Audit	audit_logs	/api/audit-logs/	test_audit.py	Ch. 7
NFR-07	DB grants	grants.sql	—	test_grants.py, test_privileges.py	Ch. 7
FR-40	Conduction	attempt_answers	PUT /api/attempts/{id}/answers/	INT-COND-07	Figures 4.17, 6.3
FR-41	Question bank	questions/topics	GET /api/questions/?q=&topic=&difficulty=&type=	INT-QB-06	Figure 4.20
FR-42	Reports	query output	GET report export endpoints	SYS-RPT-01	Figures 8.3–8.6

Appendix B — Observed pytest transcript

```
$ ./backend/.venv/bin/pytest -q --tb=short
.....................................                                    [100%]
35 passed in 6.57s
```

Dated 5 October 2026. The test database was created with the migrator role. Privilege tests connected as oes_app and confirmed CREATE TABLE was denied. The run includes the review-flag and question-bank-filter regression cases. This snapshot applies to the final source package.

Appendix C — CD handover

MCSP-232 requires a CD of the executable project to be bound with the report. Bind the Compose project (online-examination-system/) including backend, frontend, database scripts, tests, and this report. The student must retain an identical copy for viva. This page is the placeholder for that physical media; no fake CD serial is invented.

Still to attach before regional-centre posting: signed Certificate of Originality, original approved proforma, signed guide bio-data, year of submission on the cover, and study-centre details.

Appendix D — Glossary

Table D.1 — Glossary

Term	Definition
Attempt	One exam_attempts row; unique per student per exam
Argon2	First configured Django password hasher
Bank	Reusable questions not owned by an exam
Deadline	Earlier of duration end and exam end_at
DFD	Data flow diagram
DRF	Django REST Framework
Frozen paper	Attempt-specific question and option order
Gunicorn	Production WSGI server used in Compose
JWT	Access and refresh tokens from SimpleJWT
MCSP-232	IGNOU MCA project course
Negative mark	Penalty for a wrong MCQ; objective total floored at 0
oes_app	DML-only MySQL account for the running app
oes_migrator	DDL account for migrations and tests
Published	results.published; student visibility switch
RBAC	STUDENT / FACULTY / ADMIN permission classes
3NF	Third normal form of the 13 domain tables
UFP	Unadjusted function points (225 in this estimate)
VAF	IFPUG value adjustment factor; taken as 1.00 here

Appendix E — Viva preparation (project-specific)

Answers describe this implementation only. They are not copied from the Job Portal sample.

1. **What is the project title and why must it stay unchanged?**

   ONLINE EXAMINATION SYSTEM, identical to the approved synopsis. MCSP-232 rejects a renamed title.

2. **Which actors exist?**

   STUDENT, FACULTY and ADMIN. Faculty is a first-class API and UI actor.

3. **What stack is implemented versus mentioned in the synopsis?**

   Python 3, Django, DRF, React, MySQL 8. Java/Spring/Tomcat was an allowed alternative and is not implemented.

4. **How are passwords stored?**

   Django hashers: Argon2 first, then PBKDF2. Plaintext is never returned. Evidence host hashes showed the argon2 prefix.

5. **Where are JWT tokens kept in the browser, and why is that a limitation?**

   localStorage keys oes_access and oes_refresh. Any script on the origin can read them; httpOnly cookies are not used.

6. **How is self-registration prevented from creating an Admin?**

   RegisterSerializer.create always passes role=STUDENT even if a role field is posted.

7. **What is a frozen paper?**

   At start, exam_questions are shuffled into attempt_questions.display_order and MCQ options into attempt_option_orders. Reload does not reshuffle.

8. **Why does attempt_option_orders exist?**

   The synopsis requires randomized options that stay stable on refresh. The permutation must be stored; it is not a report table.

9. **How is the timer computed?**

   deadline = min(started_at + duration_minutes, exam.end_at). remaining_seconds is never negative. The client displays server time.

10. **What happens when time expires?**

    ensure_open calls submit_attempt. A later save returns 409. Blank MCQs score 0.

11. **How is one attempt enforced?**

    UNIQUE (exam_id, user_id) named one_attempt_per_student plus a transactional create. Duplicate start is HTTP 409.

12. **How are MCQs scored?**

Correct = question.marks; blank = 0; wrong = 0 − exam.negative_mark. objective_total floors at 0.

13. **When is a result row created?**

finalize_result returns None while any descriptive awarded_marks is null. Otherwise PASS/FAIL against pass_marks with published=false.

14. **When can a student see keys and explanations?**

Only after POST /api/results/{id}/publish/. Unpublished GET is 403.

15. **Are reports stored?**

No. examination/reports.py aggregates attempts, answers and published results. There is no reports table.

16. **What are the two MySQL accounts?**

oes_migrator (DDL) and oes_app (DML only). Tests observed errno 1142 on CREATE TABLE as oes_app.

17. **Name the 13 domain tables.**

users, subjects, topics, questions, question_options, exams, exam_questions, exam_attempts, attempt_questions, attempt_option_orders, attempt_answers, results, audit_logs.

18. **Is the schema in 3NF?**

Yes: options and frozen orders are rows; junction keys are full pairs; subject names are not copied; averages are computed.

19. **What Agile sprints were used?**

Seven increments: auth, bank, exams, conduction, evaluation, reports/audit, test/fix. Not the sample’s job-portal sprints.

20. **What is the PERT critical path length?**

A1–A5–A8–A10–A11–A13–A14–A15 totalling about 74.9 working days (~15 weeks), not the sample’s 32-day path.

21. **How many automated tests passed, and what is excluded?**

33 pytest cases in 5.61s on 5 Oct 2026. Browser Playwright/Chrome journeys and the Vite build are separate. There are no Jest tests.

22. **Which APIs lack dedicated pytest cases?**

Token refresh, subject/topic CRUD, question activate (partial), exam unpublish, several list/detail GETs.

23. **What audit actions are stored?**

LOGIN, START, SAVE, SUBMIT, EVALUATE, PUBLISH. Admin list is capped at 500 rows.

24. **Is HTTPS configured in Django?**

No. Development is HTTP. TLS belongs on a reverse proxy in front of Gunicorn.

25. **What is not implemented despite synopsis wording on monitoring?**

Webcam, microphone, screen watch, face recognition, browser lockdown, OTP, CAPTCHA, 2FA, IP allow-list, AI cheating detection, coding judge.

26. **How does rule-based draw fail safely?**

If the active same-subject pool is smaller than requested counts, the API returns 400 and the paper is unchanged.

27. **Can a published exam's questions be edited?**

Paper edits are draft-only. A question already on an attempt cannot change stem/marks/type/options.

28. **What does Faculty see for an in-progress attempt?**

403. Keys must not leak during conduction.

29. **Give the live demo Operating Systems Midterm published totals used in Chapter 8.**

From GET on 5 Oct 2026: 3 registered students, 2 submitted, 1 published result, pass 1 / fail 0, average 7.50 / 10.00. Demonstration data only.

30. **What function-point effort was estimated?**

UFP 225, VAF 1.00, 5 hours/FP → 1,125 hours ≈ 7.0 person-months. No rupee spend is claimed.

31. **Which integrity types are used?**

Entity (PKs/uniques), referential (FK PROTECT/CASCADE/SET_NULL), domain (CHECKs), user-defined (serializers and freeze_paper).

32. **How does the UI autosave?**

React exam room PUT /api/attempts/{id}/answers/ every 15 seconds and on option change.

33. **What production WSGI command is documented?**

gunicorn config.wsgi:application –bind 0.0.0.0:8000

34. **Who is the student and guide on the cover?**

Sushmita Mishra, Enrolment No. 2350025885, guide Shiv Pratap Singh, from the supplied synopsis front page. Year, study centre, signatures remain TO BE PROVIDED.

35. **Why is there no reports table?**

Participation, averages and pass/fail depend on the current published result set. Querying attempts/answers/results avoids a stored average that becomes stale.

36. **Why is Exam–Question many-to-many?**

A reusable bank question may appear on several exams, and each exam selects several questions through exam_questions.

37. **Why are option values rows instead of option_a to option_d columns?**

Rows satisfy first normal form, give each option a stable id, and let the frozen attempt store a shuffled option order.

38. **Which table prevents a refresh from reshuffling options?**

attempt_option_orders, unique by attempt-question+option and attempt-question+display order.

39. **Why is result calculation separate from result publication?**

Calculation records the completed evaluation. Publication is Faculty's deliberate disclosure step, especially after descriptive marking.

40. **What is the difference between SUBMITTED and EVALUATED?**

SUBMITTED may still have descriptive answers with null awarded marks. EVALUATED means all awards and one result are complete.

41. **How is a short draw pool handled?**

The service raises an error inside the transaction. The test confirms the previous draft paper count is unchanged.

42. **Why is the deadline the minimum of two times?**

A student cannot exceed either the duration measured from their start or the institution's overall exam window.

43. **What happens to an unpublished result in Student history?**

The attempt/status is listed, but result status and marks are null. Detail returns 403.

44. **What do Faculty report averages include?**

Only results rows whose published flag is true. Submitted/evaluated counts are reported separately.

45. **Why can submitted be greater than pass plus fail?**

Some submitted attempts can still need marking or publication and therefore are excluded from published outcome counts.

46. **What does "correct" mean in topic analysis?**

It increments only for an MCQ whose awarded marks equal the question marks. Descriptive rows use average award, not a correct count.

47. **What happens when a referenced subject or question is deleted?**

PROTECT prevents deletion where historical meaning would be lost. Deactivation is the normal way to stop new use.

48. **What does the audit log not prove?**

It is an in-application newest-500 extract, not an external write-once compliance archive or proctoring record.

49. **Which acceptance work remains before production?**

HTTPS/headers, secrets, rate limiting, backup restore, monitoring, token-storage review, account governance, eligibility and accessibility/load/security testing.

50. **Why is the report printed on A4?**

MCSP-232 requires A4 paper. This Word file and the PDF were rendered together in LibreOffice from the same A4 layout, so the printed pages match.

Five-minute oral outline: title and three actors; three-tier Django/React/MySQL; frozen paper and server timer; hybrid evaluation and unpublished results; query reports; 33 passing tests and named limits.

Ten-minute outline: add PERT 75-day path, function points, DFD 0/1/2, ER 13 tables, grants, and the access matrix.